

QGRAF 4.0

P. Nogueira

CeFEMA, Instituto Superior Técnico
Universidade de Lisboa (ULisboa)
Lisbon, Portugal

Abstract

This document¹ is a (nearly) unified manual for **qgraf-4.0** that contains the latest, revised and updated versions of previously available material — for instance, an extended discussion on the *generic* sign convention implemented in the program (which does not rely on graphical rules). Some other details should be added later. As usual, a summary of the new features and other changes is included in the Changelog (the last section).

The latest version of the program, namely **qgraf-4.0.6**, is a *release candidate* for version 4.0, which should become stable in (mid) 2026. Meanwhile, further testing and possibly bug-fixing should continue.

¹ This version (February 2026) is part of the **qgraf-4.0.6** pack, and was typeset in T_EX, aided by the **Eplain**, **epsf**, **colordvi** and **manmac** packages.

Contents

0.	Preliminary remarks, current plans	4
----	------------------------------------	---

Part I — Introduction

1.	About the program	9
2.	Brief notes on the terminology and notation	10
3.	Downloading, compiling and installing	11
4.	Data types, internal parameters	15
5.	Basic command-line arguments and options	16
6.	Files, characters and lines	18
7.	Basic types of character strings	20

Part II — The control-file

8.	The basic statements	24
9.	The <code>config</code> statement	26
10.	The <code>model_dir</code> , <code>output_dir</code> , <code>style_dir</code> and <code>separator</code> statements	30
11.	The <code>messages</code> statement	32
12.	Multiple <code>style</code> statements	33
13.	The <code>in</code> and <code>out</code> statements (extended version)	34
14.	The <code>loops</code> statement (extended form)	35
15.	The <code>options</code> statement	36
16.	The <code>partition</code> statement	43
17.	The <code>index_offset</code> statement	44
18.	The <code>count_to</code> statement	45
19.	The <code>zero_momentum</code> statement	46
20.	The <code>elink</code> statement	47
21.	The <code>plink</code> statement	49
22.	The <code>vsum</code> and <code>psum</code> statements	50
23.	Other statements (additional filters)	52
24.	A list of the control-file statement types	54
25.	An example of a modern control-file	55

Part III — The model-file

26.	A basic language for describing models	58
27.	An extended language	62
28.	Duplicate vertices	66
29.	The field (or propagator) commutation number	67
30.	An example of a modern model-file	68

Part IV — The style-file

31.	Intrinsic representation of diagrams	71
32.	Generating an output-file — basic concepts	75
33.	The prologue and epilogue sections	78
34.	The diagram section — the non-local style-keywords	81
35.	The diagram section — the local style-keywords	84
36.	A first look at output styles	90
37.	Non-intrinsic style-keywords	92
38.	Other additions and extensions	94
39.	A list of the intrinsic keywords for the prologue and the epilogue sections	96
40.	A list of the intrinsic keywords for the diagram section	97

Part V — Other topics

41.	The display-output	100
42.	An application programming interface (API)	105
43.	The diagram sign	114
44.	Models with explicit propagator mixing	119

Part VI — Additional information

45.	Automatic downloads, licensing	122
46.	Implementing the new input specifications	125
47.	Backward-conversion of input files	128
48.	Conversion in bulk of input files	130
49.	Obsolescent features	132
50.	Changelog for <code>qgraf-4</code>	133

0. Preliminary remarks, current plans

This manual includes up-to-date examples of a control-file (Section 25) and of a model-file (Section 30). Moreover, Section 3.1 shows how to create an executable file, or binary, for use in auto-mode (the usual mode, in which the program runs ‘autonomously’, supported by the operating system and some installed libraries); note that an additional compiler option has just become necessary. The recently introduced API-mode, in which some other program calls QGRAF by means of an interface, is presented in Section 42, and the respective compilation procedure is discussed in Section 3.3.

In recent years, most of the development has been increasingly focused on usability rather than on diagram related features. The corresponding releases have been closer to development versions than in the past, for that allows adding new features more frequently. Nevertheless, that development phase is nearly over, and the above mentioned trend will now be reversed; new releases are likely to become less frequent and include just the odd new (diagram related) feature. Obviously, there are still various non-trivial features, some of which suggestions, waiting for their turn.

Note that **qgraf-4** relies on revised *input specifications*. Some ancillary tools are now available, to help with the corresponding transition (Sections 46 to 48). The list of changes² follows, in the form of ‘rules’ (a new rule has been added, actually, but it does not appear likely to cause much disruption — more on that below).

- (1) The names of files and directories are constrained — as described in Section 7.3 and Section 10.3, only generally.
- (2) Only ‘%’ and ‘#’ are allowed as annotation-marks (the asterisk has been excluded at last); in any given file, only one such character is allowed as annotation-mark.
- (3) Some ‘void’ control-file statements are no longer accepted (Section 8.2).
- (4) The default external momenta and the default prefix of the integration momenta have been dropped; the respective identifiers have to be declared, should they be needed.
- (5) The default filename for the control-file, ie ‘**qgraf.dat**’, has been deleted; a filename has to be specified, always.
- (6) The style-keyword **<diagram_index>** has been re-defined in the epilogue section, where it should be replaced by **<diagram_counter>** (Section 34.1).
- (7) The style-keywords **<command_loop>** and **<command_line_loop>** have been renamed **<statement_loop>** and **<statement_sub_loop>** (respectively), and the style-keyword **<command_data>** has been replaced by **<sub_statement>**, which is not fully equivalent.
- (8) Constants and functions must be declared (in model-files), and any function involved in a numerical constraint (a **vsum** or a **psum** statement) should be declared **integer**.
- (9) The implicit statement continuation has been dropped, and a statement-continuation mark is required in model-files and control-files, in every appropriate instance.
- (10) Both ‘long integers’ and ‘long rationals’ (Section 7.1) have to be input in encoded form (that is, ‘quoted’, using apostrophes).
- (11) For any input file, the maximum allowed line length is now equal to 120.

² That list includes the input specification changes only; some of the additional changes listed in Section 0.1 may have to be looked into.

It ought to be clear that the option to do a ‘rather good sweeping’ prevailed, which included the deletion of some unnecessary defaults. A second look at the above list ought to suggest that its impact should be much more limited than the one that might be suggested by the number of items alone; it is even likely that a good number of those rules are broken only rarely in actual practice, and that little (if anything) has to be done about them (ie these).

Rule 10 is very likely one such rule, and in any case the conversion tool is able to deal with it (for model-files). That rule was nearly missed and had to be added rather hurriedly, mainly due to the ‘indefiniteness’ of some of the bounds for integers and rationals in the previous versions of the program. Also, it would have been a wasted opportunity not to include it this time, only to introduce it after the announcement (and release) of the next stable version, which will be **qgraf-4.0**. As there have been some delays already, that version *shall* become stable by next August (to be more precise, **2026-Aug-01**). A patch version should be released just days before that date, and include in principle the necessary fixes for any bug detected until then.

An integrated file conversion tool is now able to deal with rules 2 and 6–10 (and, trivially, with ‘non-rule’ 11). What will not be addressed, for obvious reasons, includes the modification of third-party software, dealing for instance with the file naming and the construction of input files. Rules 1 and 5 are the ones falling into that category; rules 3–4 will likely be left out too. Rules 1–5 consist in fairly simple restrictions that could be implemented at once in related third-party software since compatibility with currently available versions of the program would not be affected; then, migrating to **qgraf-4** later on would be simpler. The implementation of rules 4–5 is quite easy (Section 46.1).

That means that both model-files and style-files can be automatically converted but, excluding some minor adjustments, control-files cannot (Sections 46 to 48). Therefore, what is left out reduces to generating a valid control-file and possibly restricting the filenames.

The reasons for these changes include: fixing or improving poor design choices (rule 9, specially), improving compatibility with different operating systems, trying to ensure that input files are easily interpretable, and eliminating potential pitfalls. There are also a few reasons for introducing those changes at this time. One, distributing a single source file (to avoid increasing the number of versions to maintain, for instance) also implies change, mainly at the compilation stage. Two, if they were introduced later on, the recently introduced API (application programming interface) would also have to change. Three, there is QGRAF-R to consider; for example, QGRAF and QGRAF-R should in some ‘near-future’ be able to read exactly the same model-files (including those that use the language extensions introduced with **qgraf-3.6**), as well as control-files that satisfy similar specifications.

Thus, change was induced by a combination of several factors. The ensuing disruption should be minimized by introducing all those changes simultaneously (in a single version, that is) while providing some software to help with the conversion of the input files — as opposed to more than one set of changes, with each set introduced in a different stable version.

As with any new features, users that rely (or plan to rely) on packages that depend on the latest version of the program may have to postpone the effective use of some of those features. Two possibilities suggest themselves. As the first, two distinct directories for model-files might be created — say, ‘models3/’ and ‘models4/’ — with ‘models3/’ storing the files in the format accepted by **qgraf-3.6**, and ‘models4/’ the newer version of those files, to be accepted by **qgraf-4.0**. In fact, that newer version in ‘models4/’ could be created whenever needed from an older version in ‘models3/’, using **qgraf-4.0** (Section 46), and that applies

also to style-files. The second possibility consists in the conversion in bulk of whole class directories (Section 48). Thus, apart from an automatic conversion, current input files could continue to be used for some time. The opposite type of conversion may also be possible if no recent features are used in those files (Section 47).

Regarding the previously announced interface (API) specifications, there are a few (simple) changes and additions (Section 0.2). Although further testing will continue, it should be possible to use the latest version for related software development.

Some of the past developments are due to ‘*suggestion-requests*’ from various users. Concerning the more recent features (ie introduced with **qgraf-3.1.5** and later versions), Lance Dixon suggested something like the diagram option **onshellx**, and Vladyslav Shtabovenko suggested having more than one kind of output-file in the same run, allowing the name of the control-file to be definable, and a file overwriting feature (and, indirectly, allowing the use of some additional characters in input files, in annotations). Regarding run-time optimizations, John Gracey and V. Shtabovenko suggested improving the efficiency of (respectively) the graph generation for ‘large’ orders of perturbation theory, and of ‘additive-filters’ such as **vsum**. The implementation of a formal way to run the program as a sub-program was inspired in a not so recent request of Mikhail Tentyukov. The suggestions to have no display-output (or a very limited one) and to extend the **loops** statement are likely due to Jos Vermaseren (long ago).

0.1 Some other changes relative to qgraf-3.6

- (1) The pack that is distributed includes directories;
- (2) an additional compiler option is required in auto-mode since the source code includes preprocessor directives (Section 3.1);
- (3) a UTF-8 byte-order mark may be added to output files (Section 9.9);
- (4) the rule for the implicit display-mode has been modified (Section 9.7) and a slightly extended display-output has been implemented (Section 41);
- (5) the ‘diagnostic’ display-output is directed to the standard error stream (Section 41.4);
- (6) there is now a temporary buffer for storing messages while the message-file has not been opened;
- (7) a minor speed-up might be observed (rarely) as some partitions may be skipped;
- (8) a few parameters were increased.

0.2 A few API adjustments

Concerning the generic API call, the previous choice of the types of the integer arguments (in **qgraf-4.0.5**) does not seem to be the best one, unfortunately; therefore, those types have been changed (see also the first part of Section 4, however). What should be modified in a calling program includes (and might reduce to) the following: in Fortran,

```
c_int32_t    →    c_int_least32_t
c_int64_t    →    c_int_least64_t
```

and in C,

```
int32_t      →    int_least32_t
int64_t      →    int_least64_t
```

at least for the arguments of the interface (in either case).

The initialization call has been modified, with argument `lint` now playing an input role (p. 108).

Further, `lint`'s return role has been extended for interface calls that ought to return output-blocks but do not (p. 112). This adjustment is unlikely to require changes in the calling program unless that extended role is to be put to use, of course.

Part I — Introduction

1. About the program

QGRAF is a Feynman diagram generator — more precisely, a (Fortran) computer program designed with the aim of generating *symbolic* descriptions of Feynman diagrams in various types of QFT models — with a configurable (ie programmable) output. Its main algorithm, of orderly type, is based on the method described in [1].

This computer program was created to automate the often challenging, error prone and tedious task of writing down symbolic amplitudes for scattering processes, specially when the number of diagrams involved is large. It is aimed mainly at writing the amplitudes in terms of products of the matrix elements that follow directly from the Feynman rules. By presenting the amplitudes in that kind of ‘raw’ form, and also by allowing several types of constraints to be imposed on the generated diagrams, it should hopefully provide a common starting point for various types of computations.

The symmetry factor, a sign (that is, a -1 or $+1$ factor that follows from the anti-commutation relations, of course) and a set of momenta should be provided for each diagram; no other field theoretic computation is performed, however. It should be noted that the *generic* sign convention implemented in the program does not rely on graphical rules.

Neither vacuum diagrams (ie diagrams with no external fields) nor non-connected diagrams are generated. In addition, diagrams must have at least one vertex, which means that tree-level propagators are not ‘generated’ either.

The program was first used in a (then cutting-edge) 3-loop massless QCD calculation, circa 1995. Since then, owing not only to (further) impressive developments in multi-loop computational methods but also to sustained advances in computer technology, quite a few calculations involving many thousands of diagrams have been reported in the literature.

Fig. 1 depicts the evolution of QGRAF, measured in terms of the number of lines of source code. Most of it consists of Fortran code but some preprocessor directives are now intermingled and there is even a small header file (for the API) written in C. These numbers do not tell the whole story, of course — not every line of code survives from one version to the next. Versions released before 2020 were written in Fortran 77 but the more recent ones are Fortran 2008 programs; further, both `qgraf-3.6` and `qgraf-4` are expected to conform also to the Fortran 2018 standard.

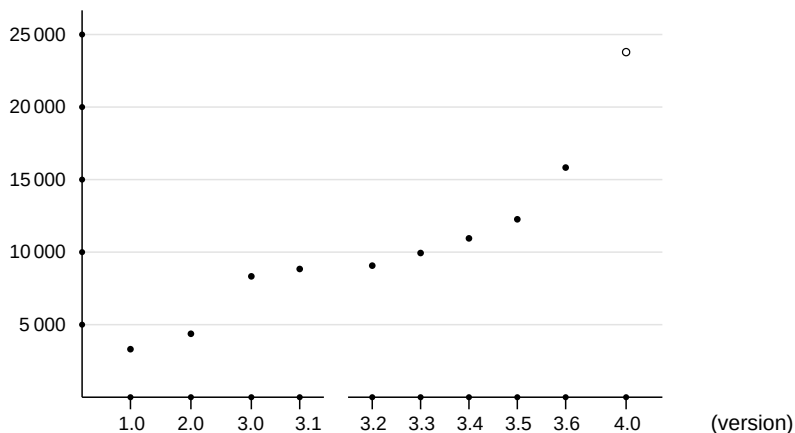


Fig. 1. The approximate number of lines of code.

2. Brief notes on the terminology and notation

In this document, if used with no further specification, the expression *the program* shall mean QGRAF.

The program has now two distinct execution modes — namely *auto-mode* when it is run ‘autonomously’, or as ‘standalone’ (supported by the operating system and some libraries, and typically launched from a script file or from the command-line), and *API-mode* when it is called from another program using the API described in Section 42.

The expression *to display ‘something’* will mean *to send a ‘graphical’ representation of ‘something’* (using ASCII characters, that is) either to the standard output stream (`stdout`) or to the standard error stream (`stderr`); these streams may be re-directed, of course. This type of output will be dubbed the *display-output* since it is often sent by default to the computer monitor, or ‘display’, if the program is launched from a terminal (as opposed to the output that the program may forward directly to some file). Starting with `qgraf-4.0`, diagnostic (ie warning and error) messages are sent to `stderr`, and the ‘normal’ display-output to `stdout` (as is usually the case) — hence it should be possible to separate ‘normal’ from ‘diagnostic’ display-output with the help of the operating system (Section 5.1).

A *display-mode* is a (partial) configuration of the program characterized by a predefined selection of the (sub-)types of display-output that are to be produced; essentially, it is similar to a so-called verbosity level. Further details are given in Section 9.7 and Section 41.

The file types, which reflect the roles of the respective files, are described in Section 6. Next, some internal links to the definitions of a few other technical terms.

- *config option* — configuration option, Section 9
- *class directory* — Section 10
- *output-block* — Section 42

Some abbreviations used in this manual follow.

- *API* — application programming interface
- *f-function* — field-function
- *lhs* — left-hand side
- *p-function* — propagator-function
- *rhs* — right-hand side
- *v-function* — vertex-function

When typing characters strings that include spaces (for this document, in particular), using the symbol $\sqcup$ (sometimes called the *visible space*) often helps clarifying whether a space is indeed part of a character string, or where a line begins, or ends (specially if there are leading and/or trailing spaces).

3. Downloading, compiling and installing

The disclaimer and the general license of the program may be found in the ‘official’ website, currently

`http://cefema-gt.tecnico.ulisboa.pt/~paulo/qgraf.html`

Additional terms may or may not be found in the header of the respective source file(s). The current URL for automatic downloads (Section 45) is

`http://qgraf.tecnico.ulisboa.pt/`

The (old) domain `ist.ult.pt` is likely to become obsolete at some point.

The source files of the latest versions are expected to conform to the Fortran 2008 standard³ (as well as to the Fortran 2018 standard) although testing has been restricted to the GFortran compiler. Note that there are binary/executable versions of GFortran for several operating systems, as described in

`https://gcc.gnu.org/wiki/GFortranBinaries`

although most Linux distros have ready-to-install packages, which might be installed in your computer already (more than one package may be needed). Starting with `qgraf-4.0`, the GNU preprocessor is also required (that program is likely to be a standard part of any Linux distro, though).

The downloaded compressed tar archive can be unpacked by placing it in an empty directory and executing the command

`tar -x -z -f qgraf-4.0.6.tgz`

or an equivalent one, depending on the operating system.

3.1 Compiling and linking (auto-mode, or standalone)

As the source code of `qgraf-4` includes preprocessor directives the compiler option ‘`-cpp`’ will be required from now on (with `gfortran`), to enable preprocessing. Seemingly, that option can be used even with `qgraf-3` — without producing any effect since in this case the source code does not include directives. In a Linux/GNU system one may copy the source file to an empty directory and then execute the commands

```
\mkdir fmodules
gfortran -cpp -o qgraf -Os -J fmodules qgraf-4.0.6.f08
```

to obtain an executable named ‘`qgraf`’ (the correct version numbers should be used, of course). For the latest pack, executing the command

```
\make qgraf
```

in the directory that contains the ‘`Makefile`’ should also produce an executable file.

There is a minor inconvenience in that a number of Fortran module files are created (one file for each module defined in the source code); still, GFortran can place those files in a user-definable directory (option ‘`-J`’) and they can be deleted once an executable file has been created. If the compilation produces an error message like this one

```
Fatal Error: Cannot read module file 'qaski.mod' opened at (1),
because it was created by a different version of GNU Fortran
```

³ Not only that, at least one feature of Fortran 2008 *is* used — hence those versions do not conform to any preceding standard.

that likely means that the compiler found some module files (which it might have ignored, or even deleted) in a format it cannot understand, and then it got a bit confused. In that case, find and delete the problematic module files (their filenames should start with ‘q’ and, as reported by GFortran, end with ‘.mod’), *both* in the working directory and in the directory specified with the option ‘-J’. Better yet, remove *any* such previous Fortran module files, in either directory, before re-compiling.

In fact, the existence of earlier module files (in the same format, but for a different version of the program) can lead to another type of problem, namely compilation failure due to apparent programming errors! So... any pre-existing Fortran module files should be deleted before compiling the source code, or else a new directory (and a subdirectory) should be created for that purpose, as already suggested.

As there are no modules in Fortran 77, the next command can be used as template for compiling older versions (currently, only `qgraf-3.4` is available).

```
gfortran -o qgraf -Os qgraf-3.4.2.f
```

The option ‘-Os’ tells the compiler to try to minimize the size of the executable while still incorporating some optimizations that tend to produce better performing (binary) code. Option ‘-O2’ optimizes a bit more for speed than ‘-Os’, and ‘-O3’ is also a possibility (but ‘-Ofast’ should be avoided).

Compiling with options that rely on somewhat sophisticated code transformations to produce more efficient executable files (this is often called *aggressive optimization*), such as ‘-O3’ in GFortran, should probably be avoided unless the additional speed is really needed — compilers, like other computer programs, are not immune to errors. For relevant computations (ie other than testing), such options should be used only after some preliminary successful testing (with similar but computationally less-demanding cases, say), where the outputs of two or more executables compiled with different optimization levels are compared — for example, an executable with the desired ‘aggressive’ level, and one or more executables with ‘non-aggressive’ levels. Furthermore, it should be kept in mind that changing the program’s environment (either hardware or software) can invalidate a previous test — for example updating the compiler, which one’s computer might do automatically, and then recompiling.

It is perhaps worth mentioning that the program does not make explicit operating system calls to run shell commands; in fact, it is not even aware of which operating system is being run. Thus, neither of the following kinds of statements is used.

```
call system(...)
call execute_command_line(...)
```

The first is a non-standard system call available with GFortran (for Fortran 77), and the other is the standard Fortran call, introduced with Fortran 2003. Naturally, (Fortran) input and output statements are used extensively, providing access to the `stdout` and `stderr` standard streams, to the control-file and to any other file specified in the control-file.

3.2 Installing (auto-mode, or standalone)

The installation is also system dependent but it is not particularly complex. In a Linux/GNU system the executable file produced by the compilation (and linking) procedure may be copied to one of the directories

```
/usr/local/bin/
/opt/bin/
```

(possibly after creating it, should it not exist), and then assigned the correct file permissions. For example, if the executable is in the working directory (and is named ‘qgraf’), one could do the following

```
\cp -i qgraf /usr/local/bin/
\chmod 755 /usr/local/bin/qgraf
```

(as the `root` user, say, or using the ‘`sudo`’ command). After that, if necessary, each user would have to create an appropriate alias for the program, or else add the full name of the installation directory to its own `PATH` (the shell variable). That procedure corresponds to a system-wide installation. Should a single-user installation suffice then the installation directory could be

```
$HOME/bin/
```

Actually, since it is usually convenient to have a dedicated directory for the program (to store all the necessary files, including subdirectories for model-files and style-files) the executable could be moved to (or possibly left in) that directory, and then run from there as

```
./qgraf my_control_file
```

for instance. Packages that rely on QGRAF might dictate specific configurations, however. On the whole, it seems a bit safer to have a dedicated user account to run the program and any related software.

3.3 Compiling and linking (API-mode)

Generating an object file suitable for API-mode is also simple if the commands included in the distributed ‘`Makefile`’ (can be and) are used as templates. The specific commands depend on which interface subroutine is used, which in turn depends on the (Fortran or C) calling program. The names of the available interface subroutines are these:

```
qinterf2008f
qinterf2008c
qinterf2018c
```

A Fortran 2008 conforming program should rely on the modules ‘`iso_c_binding`’ (intrinsic) and ‘`q_api`’ (non-intrinsic, and part of QGRAF), by means of `use` statements; also, it should call subroutine `qinterf2008f`, exclusively. For details, see file ‘`pf08.f08`’ in directory ‘`api_mode`’. The compilation and linking commands might consist in something like this:

```
gfortran (...) -c -O3 -J fmodules qgraf-4.0.6.f
gfortran (...) -o x -O3 -J fmodules qgraf-4.0.6.o x.f08
```

Here, ‘`x.f08`’ is a Fortran program that calls the interface, ‘`x`’ is the combined executable, and each ‘`(...)`’ stands for an appropriate set of additional compiler options (to be found in the ‘`Makefile`’, specifically in the commands that create the binary ‘`pf08`’). The compilation has been split into two steps (QGRAF first, then the calling program plus linking) to try to isolate the source of any problem that might arise; also, the first of those commands will probably have to be executed only once (during the development and testing of the calling program). The file ‘`qgraf-4.0.6.o`’ is the object file produced in the first step.

One may recall that the conformance of a computer program to some (revised) standard depends usually on using neither deleted features nor non-standard extensions. For example,

a program written in Fortran 77 might be a Fortran 2018 conforming program (but possibly not a very good one). As another example, a program written in Fortran 95 should work fine with QGRAF after being ‘polished’ (if necessary) into a Fortran 2008 conforming program. The compilation procedure described in the ‘`Makefile`’ will *likely* produce error messages should the calling program fail to be conforming, but reading (about) that short part of the appropriate Fortran standard(s) is a better option.⁴ Whether or not that polishing step can be skipped by adjusting the Fortran standard of the final compilation, is something that has not been tested; moreover, that possibility (if it exists) might be compiler dependent.

The latest pack includes some actual examples of interfacing (one for each interface subroutine), whose compilation relies on special sets of options. Each example uses the same source file for QGRAF (in fact, only one source file is available for each version) but the compiler options differ. The included ‘`Makefile`’ can deal with each one of those examples; naturally, it is possible to copy (and adapt) any command included in that file.

The earlier warnings about Fortran module files should be kept in mind. Although the ‘`Makefile`’ includes commands for deleting those files, some have been commented out; the deletion of the respective annotation-marks (a leading ‘`#`’ in every case) should be considered once any necessary adjustments have been made.

In the only mixed-programming case to be considered⁵ (and the only one available) the API is called by a C program. The respective compilation procedure will very likely require two steps, the first relying on a Fortran compiler (`gfortran`, say), the other on a C compiler (`gcc`, for instance). The files ‘`pxq08.c`’ and ‘`pxq.c`’ show how to use the two versions of the mixed-programming interface implemented in `qgraf-4.0`. The header file ‘`qgraf.h`’ should be imported by any C program that calls the API, using an ‘`#include`’ directive.

In one of the examples (file ‘`pxq08.c`’), the interoperability between the two programming languages is based only on the standard tool available for Fortran 2008 — namely the ‘`iso_c_binding`’ module, used on the Fortran side (by QGRAF, that is). In this case, the C program should use interface subroutine `qinterf2008c`.

In the other example (file ‘`pxq.c`’), a more recent standard based on *C-descriptors* is also used. In the ‘`Makefile`’, the commands that create the binary ‘`pxq`’ rely on the C17 and Fortran 2018 standards. Generically, the header file ‘`ISO_Fortran_binding.h`’ should be included in the calling program, using another ‘`#include`’ directive, and the interface subroutine `qinterf2018c` should be used exclusively. At present, that method may seem rather superfluous; if the mixed-programming interface is to be used, the method mentioned in the previous paragraph is simpler.

⁴ Free ‘Drafts’ of the Fortran Standards (which do not necessarily describe the definitive standards in every detail, but something *very* close to that) may be available from wg5-fortran.org

⁵ It seems possible to ‘mix’ C with some other programming languages, besides Fortran (although I have not attempted it). In particular, that might allow interfacing QGRAF with programs written in those languages, using C as intermediate.

4. Data types, internal parameters

Concerning Fortran INTEGER type variables, 32-bit integers are required in auto-mode and 64-bit in API-mode; these requirements should be supported by any current compiler.

In API-mode, the data types of the interface parameters include `c_int_least32_t` and `c_int_least64_t`. Should that lead to problems with your compiler (not actually expected but...) then those data types may likely be changed to (respectively) `c_int32_t` and `c_int64_t`.

Changing the default value of an internal parameter is typically not advisable, as there is a real risk of ending up with a defective program (specially for `qgraf-3.1.5` and more recent versions). In particular, increasing the maximum allowed loop number could be a rather risky operation. The only changes that should be ‘safe’ are the ones that consist in increasing the values of one or more of the parameters

```

sbuff0
sibuff0
swbuff0

```

should the program report that they are too small (these parameters control the size of some ‘arrays’ and ‘buffers’ used for storage and input/output). That should not be a frequent occurrence: for instance, an unusually large file would have to be given as input, or the model should have a large number of vertices (or at least some vertices of high degree). One may then double the value of each reported parameter, more than once if necessary, until there is no error (alas, upper bounds also exist); values of the form $2^n - 1$ should be preferred.

5. Basic command-line arguments and options

The program can be launched from the command line (either ‘by hand’ or using a script file) or run as a sub-program (using the API). In the first case, the command-line interface can be used to pass a few basic instructions to the program, as detailed below.

The command-line options include typically two leading hyphen-minus characters (for instance, ‘`--version`’) but it should be possible to use only one such character if the shell dislikes the former possibility.

A ‘normal’ run is characterized by the presence of the name of a control-file as an argument of the command that launches the program; that filename should follow the name of the executable, preceding any option. In a ‘non-normal’ run the name of the executable is followed by an option, dubbed the main option, which defines the type of run.

Some other options that can be used for the conversion of input files are described in Section 46 and Section 47.

5.1 Launching the program

For a normal run in auto-mode, the name of the control-file should be passed as a command-line argument, for instance

```
qgraf a_control-file
```

In this example, the program would try to read the file ‘`a_control-file`’ located in the working directory. The restrictions described in Section 7.3 apply.

5.2 Option `--version`

Executing the command

```
qgraf --version
```

should result in the program outputting its own name and version (typically left-aligned on `stdout` and terminated by a *newline*), and then exiting at once. No additional options nor arguments should be present. For example, the character string produced by the latest version should be ‘`qgraf-4.0.6`’.

5.3 Option `--no_extra_chars`

The character set used by the program is the (‘7-bit’) ASCII set (Section 6.1). With `qgraf-4`, some ‘non-standard’ characters (namely the `Tab` character and printable characters from some 8-bit extensions of ASCII) should be tolerated in *annotations*, in any input file, if any such extension is supported by the compiler (and the operating system). As there exist several ASCII 8-bit extensions, it is likely not a very good idea to distribute files that include such characters (ie those should probably be used only ‘privately’) — `Tab` is an exception since it is a ‘7-bit’ character.

In any case, it is possible to use the program to check whether any input file contains ‘non-standard’ characters (excluding a UTF-8 byte order mark). The command-line option ‘`--no_extra_chars`’ enforces a run-time error if any such character is found, for example

```
qgraf a_control-file --no_extra_chars
```

5.4 Option `--extract`

This option can be used in a special type of run to ‘extract’ a submodel defined in a certain model-file, and save it in a new model-file that does not contain submodels — as the only model defined in the new file, therefore. Currently, the new file does not define sectors nor default images, and does not contain annotations; nevertheless, that may change in future (with the introduction of another option). A template command follows, in which ‘models/model_a’ is the existing model-file, and ‘models/submodel_0’ is the file to create, containing a description of submodel ‘subm0’ only.

```
qgraf --extract models/model_a models/submodel_0 --subm subm0
```

The options that can be used with the file conversion commands (Section 46), namely

```
--hash
--percent
--utf8_bom
--no_utf8_bom
--no_extra_chars
```

can also be used in the present case; the argument sequence is fixed.

6. Files, characters and lines

QGRAF uses with several types of files, whose roles are explained below.

- **control-file**
The input file that contains the top-level instructions for the program.
- **message-file**
The output file where diagnostic messages should be saved.
- **model-file**
The input file that describes the model to be used in the diagram generation.
- **output-file**
The main type of output file, where (one of) the descriptions of the generated diagrams, and possibly some additional information, should be saved (in auto-mode, only).
- **style-file**
An input file that contains the instructions for constructing an output-file (in auto-mode) or the output-blocks (in API-mode).

The hyphen that is part of the term *output-file* is particularly relevant since any file created by the program is an ‘output file’.

The name of the control-file has to be specified when launching the program in auto-mode (Section 5) or (re-)initializing the program in API-mode (Section 42.4) — respectively as a command-line argument, and using one of the interface subroutine arguments. Any other filename should be declared in the control-file using an appropriate statement. There are some restrictions for those filenames, discussed in Section 7.

As there are three types of input files, one might say that QGRAF ‘speaks’ three languages; it certainly parses them. Any control-file and model-file consists of a sequence of statements (and possibly comments) but a style-file looks a bit different.

In the remaining of this section we shall be concerned only with input files, whether created by the user or automatically generated. Moreover, the focus will be on their basic properties, such as the character set to be used, the maximum allowed length for the lines, how to insert annotations (or comments) and so on. In what follows, and depending on the context, the word ‘filename’ may refer to the name of (ie character string defining) a regular file, directory or symlink.

6.1 The character set

First of all, the character set should be the ‘7-bit’ ASCII set (nowadays a very small subset of UTF-8), which includes 95 printable characters and 33 control (‘non-printable’) characters. The printable characters consist of 26 letters (upper and lower case), 10 decimal digits and 33 other symbols, most of it ‘punctuation’. With **qgraf-4**, some other characters are allowed in annotations (or comments), see Section 5.3.

The control characters that remain in use⁶ include (for example) the file and the line terminators, but their exact list depends on the operating system (and possibly even file

⁶ In fact, many of those 33 control characters are no longer used.

system). Typically, the control characters remain in the background so that the user needs not deal with them explicitly, nor even see them (the ‘**Tab**’, discussed in Section 5.3, is an exception); apart from ‘*newline*’, they will usually be ignored in what follows.

6.2 The annotation-mark(s)

In control-files and model-files, any line whose first character is either ‘%’ (percent sign) or ‘#’ (hash mark) is ignored. Thus, that kind of lines (to be referred to as *annotations*) can be used for explanatory notes and other comments. No annotation should ‘break’ a statement extending over two or more lines, however. Style-files may also include annotations but the restrictions are different (Section 32.1).

Only one kind of *annotation-mark* (ie ‘%’ or ‘#’, when employed in the above mentioned role) is allowed in any given input file.

6.3 The statement-continuation mark, wide(r) files

The input files may contain lines with up to 120 characters,⁷ not counting *newline*. Simultaneously, a statement-continuation mark (the backslash ‘\’) has been introduced for model-files and control-files (but not style-files). This feature does not apply to annotations, only statements can be continued. Two examples of control-file statements using the new specification follow.

```
output_dir = \
    '/tmp/my_temporary_dir/' ;
partition = 3^2\
    4^1 ;
```

A backslash character should end each appropriate input line without breaking identifiers, keywords, or other indivisible strings (Section 7); a somewhat special case, ie that of *divisible* strings, is described in (Section 7.3). Each statement should occupy either a single line or a number of consecutive lines; in addition, continuation lines should not be ‘empty’, ie each one has to include some relevant part of the statement, however small (a single punctuation mark, even).

The number of lines occupied by a single statement is also bounded since there is a maximum allowed statement length (of at least 1000 characters).

NB: Although not particularly difficult, the addition of statement-continuation marks to ‘old’ model-files and control-files (that is, compatible with **qgraf-3**) can be done with QGRAF itself (Section 46).

6.4 The space character

Apart from being able to improve readability, the spaces in input files either act as a separator, delimiting character (sub-)strings, or represent real space characters in some ‘encoded’ strings (Section 7.2). Often, they are not even needed as a separator since other punctuation marks (that must be present) already fulfil that role.

⁷ But only up to 80 characters in previous versions.

7. Basic types of character strings

This section discusses some of the types of character strings recognized by the program, namely identifiers, integers, rationals and encoded strings. Naturally, several punctuation marks are also recognized, but those will be presented when needed, without further ado (some of those marks are not single ASCII characters). Moreover, the program recognizes a special type of strings that appear in the style-file(s) (the ‘style-keywords’, to be discussed later), and the characters ‘-’ and ‘+’ (as (anti-)commutation signs in model-files, or as special marks in the **partition** statement). The most basic types of strings are as follows.

- An *identifier* is a string consisting of letters, digits and underscores, with the condition that the first character is a letter. Examples: ‘**spin**’, ‘**csi__**’, ‘**F_0**’.
- An *integer* is a sequence of digits, possibly preceded by either a plus or a minus sign. Examples: ‘-0’, ‘+17’, ‘1234’.
- A *rational* is either an integer or a sequential concatenation of an integer, a slash ‘/’ and a nonzero unsigned integer. Examples: ‘1/3’, ‘-1’, ‘+0/5’.

The characters mentioned in each of the above definitions are the only ones allowed. In particular, spaces are not allowed although they can be used in a different type of character string described in Section 7.2. Moreover, there are strict bounds on the length of any such string (see below). Note also that a string such as ‘-2/1’ is not an integer.

Depending on how the output of the program is put to use, a more restricted class of identifiers may have to be employed. For example, if that output is to be processed by a computer algebra system then the identifiers outputted by QGRAF may likely have to be recognized as such by that other program.

7.1 Constraints on integers and rationals

The program does not support ‘long integers’ (nor ‘long rationals’, therefore). There is then a largest (but not very large) positive integer $M = 46340$ ($\sim 2^{31/2}$) such that any integer (say) z — either read or evaluated by the program, excluding the values of the variables that are employed to either enumerate or index the generated diagrams — has to satisfy

$$|z| \leq M.$$

If the internal parameters are not modified, that value seems to be more than enough for the currently required tasks — for example labelling the various ‘parts’ of the diagram, generating indices specified in the style-file, and evaluating symmetry factors and other weights (for the ‘filters’ **psum** and **vsum**). It is possible to specify larger integers in the model-file, but only in encoded form. In practice, though, they would not be usable in any internal numerical computation. Still, they could be reproduced (unencoded) in the output-blocks and output-file(s).

A rational p/q that does not satisfy the condition

$$|p \cdot q| \leq M$$

should also appear in encoded form (in the model-file); that is not necessary for ‘short rationals’, even if they have no particular use at present.

Concerning control-files, any integer z to be included in an **index_offset** statement or in a **count_to** statement should satisfy $|z| \leq M^3$ ($\sim 9 \cdot 10^{13}$).

7.2 Encoded strings

It should be convenient to be able to define more general character strings, which might even contain sub-strings that could be classified as identifiers, integers, punctuation marks, and so on. For example, suppose that the string

`Quantum_Electrodynamics_in_D=4-epsilon_dimensions`

should be accepted as a single object. A solution to that problem consists in encoding the original string as a string that cannot be mistaken for some other (accepted) type of string, and then supply the encoded string to the program, which will simply decode it. The ASCII character 39 (which represents the apostrophe or right quote) plays a special role in the encoding algorithm that was implemented. Encoding a string is straightforward: every apostrophe must be duplicated (ie replaced by two consecutive apostrophes) and then a further apostrophe added to each end of the string obtained in the first step. The encoded form of the above string is simply

`'Quantum_Electrodynamics_in_D=4-epsilon_dimensions'`

Here are other examples: the empty string is encoded as `''` (a string of length 2), and a single apostrophe (when considered as a string by itself) as `''''` (a string of length 4). This type of encoding (to be dubbed *normal-encoding*) is also used in Fortran programs, as known, but the syntax for string concatenation is different (see below).

As stated earlier, there is an explicit upper bound for the length of any line from any input file. This condition places an effective bound on the length of identifiers, since an identifier is indivisible and cannot be encoded. In contrast, strings presented to the program in encoded form can avoid that bound:⁸ any such string can be decomposed (before being encoded) into two or more components, which may be written sequentially, in encoded form, either on the same line (separated by at least one space!) or on consecutive lines.

For example, the character string

`'f(x)='_1+sin(x)'`

is not the normal-encoding of any (single) string but it will be decoded as

`f(x)=1+sin(x)`

since the concatenation is implicit. By contrast, the string

`'f(x)=''_1+sin(x)'`

will be decoded as

`f(x)='1+sin(x)`

because in this case there is a single component, not two.

A given string may have a large number of (non-normal) encodings; we will say that a string *t* is an encoding of a string *s* if *t* will be decoded as *s*.

7.3 Constraints on filenames and paths

Most of the names of the files and directories (and paths) should be supplied to the program in encoded form; nevertheless, given the constraints described below, that amounts simply to add two apostrophes to each name — ie one leading and one trailing apostrophe.

⁸ There is another bound, of course (namely, the maximum allowed statement length...).

The only exception is the name of the control-file in auto-mode (Section 5.1). The following restrictions apply to the ‘natural’, non-encoded names, whether specified in the control-file or the command-line; more precisely, they apply to each path component except to the path separator, which is restricted in a different way (Section 10.3).

- each (ASCII) character should be either alphanumeric (ie a letter or a digit) or one of three ‘special characters’ — namely the *underscore*, the *hyphen-minus*, and the *dot* (or *period*);
- the first and the last characters should be alphanumeric;
- no two special characters should appear consecutively.

These restrictions may be bypassed at the operating system level with the help of symlinks. The config option **noblanks** (Section 9.3) might be useful for automatic set-ups.

For example, the two following statements (which define the output-directory and an output-file, respectively) are valid.

```
output_dir = '/home/username/qgraf/tmp_dir/' ;
output = 'tmp_file-1.out' ;
```

The filename declared in that **output** statement includes every possible special character, as defined previously.

The string encoding described in Section 7.2, as well as the subsequent string decomposition convention, make it possible to use ‘long filenames’ (path included). As the filenames declared in the control-file must be encoded strings, they can be split into separately encoded components. For example, the statement

```
model = '/home/username/models/qed' ;
```

(which declares the name of the input model-file) can also be written as

```
model = \
    '/home/username/' \
    'models/qed' \
    ;
```

Thus, the length of any such filename can easily exceed the maximum allowed line length. As already mentioned, however, there is also a maximum allowed statement length... (of at least 1000 characters).

Part II — The control-file

8. The basic statements

The control-file contains the main instructions for the program, expressed as *statements* of various kinds. In auto-mode, its name should be an argument of the command that launches the program (Section 5) while in API-mode that filename must be passed in one of the arguments of the interface subroutine (Section 42.4).

8.1 The required statements

Although the control-file may include quite a few different types of statements (see Section 24), let us start off with a ‘minimal’ example that includes only the basic statements, possibly in their simplest form; also, auto-mode is assumed. Any other type of statement is optional (sometimes the `style`, `output`, and `loop_momentum` statements may be absent too).

```
style = 'style_x.sty' ;
output = 'q_list' ;
model = 'qed' ;
in = electron[p1], positron[p2] ;
out = photon[q1], photon[q2] ;
loops = 2 ;
loop_momentum = k ;
```

These statements cannot appear in an arbitrary order; here, with a single exception (the first two could be permuted), their relative ordering is fixed. The generic statement ordering is shown in Section 25. As a general rule, distinct statements should occupy distinct lines.

The `output` statement declares an output-file, which is a file where the descriptions of the generated diagrams should be saved, one after another; this statement is not available in API-mode. Unless it is known in advance that the number of diagrams is not too large, a preliminary run that produces no output should be considered — see the config option `nolist` (Section 9.4) and the `count_to` statement (Section 18).

The `style` statement declares a style-file, which should contain the instructions for generating an output-file.

The `model` statement declares the model-file, which should consist in a description of the model for which the diagrams are required (that is, of the ‘*input model*’).

In the above example, all of the input and output files are in the working directory. It is also possible to declare files in subdirectories or even in some other directory, for instance

```
model = 'models/qed' ;
model = '/home/username/models/qed' ;
```

Moreover, three special directories (the *class directories*) can be declared — one for model-files, another for style-files, and yet another directory for output files (Section 10).

In any statement specifying some input or output file, or some class directory, the respective name should be the encoded form of the actual filename used by the operating system. Moreover, allowed paths and filenames are subjected to the constraints described in Section 7.3 and Section 10.3.

The **in** and the **out** statements declare the incoming and the outgoing fields (which should be defined in the model-file) as well as the respective momenta (which must be distinct identifiers); expressions like **-p1**, **p3-p1**, or **+k** cannot be used. In the previous box, **p1** is the momentum of the **electron** (flowing inwards) and **q1** the momentum of one of the **photon** fields (flowing outwards). Additional details are presented in Section 13. It should be noted that the diagram sign may depend on the exact sequences in which the incoming and the outgoing fields are declared (Section 43).

The **loops** statement specifies the loop number (or number of loops) of the diagrams to be generated. An extended form of this statement also exists (Section 14).

The **loop_momentum** statement declares the prefix of the integration momenta, which should be an identifier. For instance (as in the previous box), if that prefix is chosen as '**k**' and if 2-loop diagrams are requested then the two integration momenta will be '**k1**' and '**k2**'. This statement is optional when that prefix is not needed — for example if the diagrams requested are trees or, in auto-mode, if no output-file is to be produced.

Note that two types of statements are not available in API-mode Section 24.

8.2 Void statements and superfluous statements (in the control-file)

A statement is *void* (or *empty*) if it includes no actual data, for example

```
config = ;
style = '' ;
```

Only the following types of statements are accepted by **qgraf-4**, if void (they are precisely the statements that may include a list of similar 'items', separated by commas).

- **config**
- **in**
- **out**
- **options**

As the **in** and **out** statements are required, there are only two optional statement types that may be empty, of course.

When the control-file is edited 'by hand', any statement can easily be converted into an annotation and back again.

Superfluous statements need not be void; occasionally, removing a certain non-empty optional statement from the control-file does not make any relevant difference on the actual run. Non-void superfluous statements are accepted unless explicitly excluded by some other rule (uniqueness, for instance).

9. The config statement

The `config` statement allows the specification of diagram unrelated options. If present, it must be the first statement of the control-file. The keywords allowed in this statement will be dubbed *config options* (or configuration options), to distinguish them collectively from the other kinds — that is, diagram options and command-line options.

Some of the config options are not available in API-mode.

9.1 Option delete

When the name of a message-file or an output-file is that of an existing file *and* the output-directory is defined (Section 10.2), the config option `delete` instructs the program to erase *any* such obstructing file in that directory. This option should be used with care. An example follows.

```
config = delete ;
output_dir = 'tmp_dir/' ;
messages = 'msg.txt' ;
output = 'q_list' ;
:
:
```

9.2 Option lf

If the operating system is one of those for which a ‘*newline*’ consists of a ‘*line feed*’ character (ASCII control character LF), there is an experimental feature that should speed-up the write operation for the output-file(s) (in auto-mode) and which is enabled by the option `lf`, for example

```
config = lf ;
```

This should lead to a small to moderate (overall) performance increase if the output-file is large enough. Compatible operating systems include Linux/GNU based systems, as described in Wikipedia’s *Newline*⁹ page, Section *Representation*. One may always run the program with and without that option (a single test case might be sufficient) and check whether the output-files are identical.

9.3 Option noblanks

The option `noblanks` instructs the program to delete each and every space character appearing in the names of files and directories (paths included) read from the control-file. For example, this means that the statements

```
config = noblanks ;
output_dir = ' tmp_dir / ' ;
output = ' q_list ' ;
```

⁹ <https://en.wikipedia.org/w/index.php?title=Newline&oldid=863813417>

should become equivalent to

```
output_dir = 'tmp_dir/' ;
output = 'q_list' ;
```

irrespective of the operating system.

9.4 Option `nolist`

In auto-mode, the config option `nolist` disables creating any output-file when one or more files of that kind are declared, for example

```
config = nolist ;
output = 'd_list' ;
style = 'f0.sty' ;

    :
```

This option may be used to predict whether some output-file would be worth generating in a subsequent run using an almost identical control-file, just without that option. The `count_to` statement provides a different possibility, namely a ‘bounded run’ (Section 18).

9.5 Option `flush`

Normally, the program exits without waiting for the operating system to close any open (output) files and make them available in their final form to other processes, although the operating system will eventually do so. The option `flush` tells the program to try to preclude that kind of short-cut and might therefore be useful if some of the output-files are large, for instance. If that option is used, the program will explicitly ask the operating system to perform the above mentioned operation, whose completion should be speeded-up (perhaps slightly, though), and then when QGRAF exits other programs should be able to start reading any output file at once. The same result is likely to be achieved by executing a `sync` command (or a similar command, depending on the operating system) on those files.

There are no output-files in API-mode. Still, if option `flush` is used, the message-file will be ‘flushed’ every time a message is appended to that file (also in auto-mode, in fact).

9.6 Option `no_ntnls`

In API-mode, trailing *newlines* are automatically deleted but non-trailing ones are not. While it is possible to rewrite a style-file formerly designed for auto-mode into another file more appropriate for API-mode, the config option `no_ntnls` offers a different possibility: it will enforce the deletion of non-trailing newlines in any output-block for the diagram section, but only after the output-block has first been built in the usual way. This option applies to every input style-file, however (perhaps it can be used for a bit of debugging); it can be used in auto-mode too.

It should be kept in mind that any *newlines* (trailing or not, possibly to be deleted) are characters of the respective (dynamic) string, and should be taken into account when declaring the maximum allowed lengths for the output-blocks (Section 42.4).

9.7 The display-mode options in auto-mode (noinfo, info, debug, verbose)

Given the current variety of information that can be displayed in auto-mode, a basic method for limiting the display-output has been implemented. Specifically, the user can select one of four available *display-modes*, which may be described as follows.

- **noinfo**

Nothing should be displayed unless an error condition is detected.

- **info**

The program's name and version numbers, the statements from the control-file, and various numbers of generated diagrams (subtotals for each vertex-degree partition, and also the total number) are displayed in that order; when appropriate, the subtotal for each loop order may be displayed too.

- **debug**

In addition to the information displayed in **info** mode, the program displays (nearly) every 'alert' it is able to, as well as a summary of the model consisting of various numbers of propagators, vertices and (when applicable) sectors; as it should be clear, this display-mode may be useful when fixing defective input files, for instance.

- **verbose**

A display-mode almost equivalent to **debug**, but messages about 'non-imminent' obsolescent features may be displayed too ('imminent' obsolescence may be alerted in both **debug** and **verbose** mode).

Additional details are presented in Section 41. The display-mode can be set explicitly with the help of the **config** statement, for instance

```
config = debug ;
```

otherwise it is set implicitly. By default, the display-mode is set to **info** unless the config option **nolist** is used, in which case it is set to **verbose**.

Diagnostic messages are directed to **stderr**, any other display-output to **stdout**. If a run-time error is detected, the respective error message can be either displayed or saved in the message-file (or both), and then the program stops.

9.8 The display-mode option in API-mode (debug)

Usually the program stays quiet in API-mode, saving any diagnostic messages in a message-file if possible, unless it has to pass an error message to the calling program. To facilitate the debugging and thus the development of any program that may call the API, the **debug** display-mode is available also in API-mode although it has a different behaviour in this case — namely it discards any 'normal' display-output that would be forwarded to **stdout** in auto-mode.

- **debug**

The program forwards all the diagnostic messages to the stream **stderr**, producing no other display-output.

The usual method for redirecting the standard streams in Linux/GNU operating systems is illustrated in Section 41.4.

9.9 Options `utf8_bom` and `no_utf8_bom`

Following the introduction of UTF-8, some applications, notably text editors, started creating files with the corresponding *byte-order mark* (hereafter, BOM), which consists of a special 3-byte sequence inserted at the beginning of a file. Although that byte sequence seems not to be strictly required in the case of ASCII text files, the fact is that it became rather common and other applications simply had to adjust to that situation.

QGRAF adapted with `qgraf-3.1.4`, but only for (reading) input files; the files created by the program did not have a BOM. Now, they may have: a UTF-8 BOM will be added if the option `utf8_bom` is used, for example

```
config = utf8_bom ;
```

but not if its dual option `no_utf8_bom` is used. If neither option is used then a BOM will be added whenever the control-file has one.

10. The `model_dir`, `output_dir`, `style_dir` and `separator` statements

A *class directory* is a directory where the program should look for input files of a certain kind (either model-files or style-files), or where the files created by the program (message-files and output-files) should be saved. Three optional statements provide a way to define three class directories; moreover, to enhance the compatibility of the program with different operating systems, the path (or directory) separator might be definable with the help of a further statement.

Recall that the names of files and directories accepted by the program are somewhat restricted (Section 7.3).

10.1 The `model_dir` and `style_dir` statements

The following example shows how to set the class directories for model-files and for style-files; they will be dubbed the *model-directory* and the *style-directory*.

```
model_dir = 'models/' ;
style_dir = 'styles/' ;
output = 'q_list' ;
style = 'style1' ;
model = 'ym/model2' ;
:
:
```

The program prepends the ‘name’ of the model-directory (ie the character string specified in the `model_dir` statement) to the ‘name’ of the model-file (specified in the `model` statement), to construct the actual filename to be passed on to the operating system; a similar convention applies to style-files. In the above example, the program would try to read the following files.

```
styles/style1
models/ym/model2
```

Note the trailing slash in the strings that define the class directories, which is there because the program ignores which operating system is being used (see also Section 10.3, however).

The potential usefulness of these two statements is limited: they allow the full names of the model-file and the style-file to be split into two parts, which might help one to avoid using a statement-continuation mark (the recent increase of the maximum allowed line length may also help); they might also improve the legibility of the control-file. The definition of the other class directory has an additional (and likely more interesting) use.

10.2 The `output_dir` statement

The `output_dir` statement sets the *output-directory*, which should be *reserved* to the files created by the program. The actual filenames of the output-file(s) *and* of the message-file will be obtained by prepending the string defined in the `output_dir` statement to each string specified in either an `output` or `messages` statement.

In this example,

```

output_dir = 'output/' ;
messages = 'msg.txt' ;
output = 'q_list' ;
:

```

the program would ask the operating system to create the files

```

output/msg.txt
output/q_list

```

and, as usual, would make no attempt to create alternative files if anything went wrong.

There is a special feature that is not available for the other class directories, that is, it becomes possible for the program to (try to) *delete* any file in the output-directory if instructed to create one or more files that already exist. For that to happen, however, the configuration option **delete** should be used, for example

```

config = delete ;
output_dir = 'output/tmp_dir/' ;
messages = 'msg.txt' ;
output = 'q_list' ;
:

```

Verifying that the `output_dir` statement is working as expected, before using option **delete**, is strongly recommended.

10.3 The separator statement

The slash '/' is used as a path (or directory) separator by Linux/GNU systems, for instance, but other operating systems may use a different character. That mark might be definable with the help of the **separator** statement, for example

```

separator = '/' ;

```

A single non-alphanumeric character is expected, though 'encoded' (note that the **noblanks** option does not apply to this statement). If a path separator is defined explicitly then the names of the class directories need not end with it — if they do not, the program will automatically append one. On the other hand, if a path separator is not defined explicitly then the names of those directories have to end with a common (and allowed) path separator.

The name of a regular file (or of a symlink pointing to a regular file) should not end with a path separator.

NB: Although **qgraf-4.0.6** accepts only one separator character (ie the slash '/'), other separators may be added if necessary; a detailed request may be sent in by e-mail after confirming that the website's page "**news, alerts**" has no entry about such a request.

11. The messages statement

The `messages` statement declares the name of a file, dubbed *message-file*, to be created by the program for saving diagnostic messages, for example,

```
config = info, lf ;
output_dir = 'tmp/' ;
messages = 'msg.txt' ;
output = 'd_list' ;
style = 'f0.sty' ;

:
```

The message-file is opened quite early on (if declared) but it is not saved at the end of the run if it contains no messages. Since the program stops on any detected error, if an error message is saved then it should be the last message in that file. Moreover, the symbol ‘(w!)’ mentioned at the very end of Section 41.1 will not be displayed if a message-file has been declared.

The `messages` statement is independent of the display-modes and aimed mainly at script based set-ups. Note that an error-message issued before the message-file has been opened will not be stored in that file, but there is now an internal buffer to store a limited number of messages. Problems at that stage should occur only rarely, if at all, unless (say)

- the control-file cannot be read or is defective in some way (for example, if it includes incorrect statements, or statements that lead to errors);
- the message-file cannot be opened, or already exists and cannot be overwritten (see config option `delete`, Section 9.1, however);
- the internal parameters were incorrectly modified.

In auto-mode, it is now possible to separate ‘diagnostic’ from ‘normal’ display-output (Section 41); in API-mode, it is possible to display the diagnostic messages. Those features provide a different and possibly better way of picking up the diagnostic messages.

12. Multiple style statements

Multiple-style configurations enable the concurrent (but separate) output of different kinds of information, including

- the ‘amplitude’ of each (generated) diagram,
- information for determining the ‘topology’ of each diagram,
- information for evaluating special (ie not provided) diagram selection criteria;
- information required to create a graphical representation (ie a drawing) of each diagram.

To create more than one output-file in the same run (in auto-mode), the control-file should include multiple **style** and **output** statements. The style-files and output-files should then be paired by alternating the respective statements, for example

```

config = ;
style = 'f1.sty' ;
output = 'out1' ;
style = 'f2.sty' ;
output = 'out2' ;
:

```

All of those files should be distinct, and there should be at most 7 pairs. It is possible to have zero pairs, although that case is not very useful (still, the number of diagrams may be obtained). Actually, it is also possible to have **style** but no **output** statements (even with a single **style** statement), so that a control-file suitable for API-mode might be cross-checked in auto-mode without changes; the style-files and the model-file will be verified too, and no output-files will be created, of course.

There are no output-files in API-mode, just output-blocks; **style** statements should still appear (at most 7, too) though not **output** statements.

The program tries to open all the input style-files almost simultaneously, and a run-time error will occur if two (or more) of those files are actually the exact same file. For example, an error will occur if the control-file includes the statements

```

style = 'f1.sty' ;
style = 'f2.sty' ;

```

and if ‘f2.sty’ is a symlink to ‘f1.sty’, since in that case the program will try to open the same file twice.

NB: For each output-file, the `<statement_loop>` skips the iterations corresponding to any **style** and **output** statements that refer to another output-file. In API-mode, a similar statement applies to the output-blocks derived from the prologue and epilogue sections of any input style-file: for each such block, the `<statement_loop>` skips the iterations corresponding to those **style** statements that do not refer to that block. Recall that empty **style** and **output** statements are no longer allowed.

13. The in and out statements (extended version)

Occasionally, the external momenta need not be declared and it is possible to state

```
in = electron, positron ;
out = photon ;
```

for instance. If at least one momentum is declared then all momenta must be declared.

Nevertheless, when the output of the program depends on some momentum, internal or external (which very likely means *nearly always*), the external momenta do have to be declared, for example

```
in = electron[p1], positron[p2] ;
out = photon[q1] ;
```

These momenta, enclosed in square brackets, are 4-momenta (or perhaps D -momenta).

Should anti-commuting fields be present, some care should be taken with the relative ordering of the external fields declared in those statements — specially if two or more runs are employed to create the required output-files for the same scattering process (Section 43).

The expressions for the momenta of a diagram are rarely unique, in terms of both the external momenta and the integration momenta. Given that the sum of the incoming momenta must be equal to the sum of the outgoing momenta, it is always possible to avoid including in such expressions a term that depends on a specific external momentum. That can be achieved by altering the respective declaration in a subtle way. Returning to the previous example, the out statement

```
out = photon(q1) ;
```

should be employed instead to exclude any term for $q1$ in the output corresponding to the style-keywords `<momentum_term>`, `<momentum>`, and their duals, but only in the propagator-loop and the ray-loop; that output remains unaffected in both the in-loop and the out-loop.

For n -point functions, $n > 2$, such substitutions tend to increase the average number of terms in the expressions for the internal momenta, but for $n \leq 2$ that is not so.

For $n = 2$, it becomes possible to have the internal momenta depend on only one (fixed) external momentum (besides the integration momenta, of course).

For $n = 1$, the question does not arise except in the ray-loop, and then only for the unique (vertex, ray) combination that corresponds to the leg of the diagram; in this case, that momentum is replaced by the zero-momentum. No internal momentum depends on the external one (which is null by momentum conservation, anyway).

NB: There are no longer default external momenta, as relying on their existence could shorten the path to trouble. Nevertheless, since that earlier convention is still assumed in some of the examples included in this manual, let us review it: the incoming momenta of the fields declared in the `in` statement are noted $p1, p2, \dots$ while $q1, q2, \dots$ denote the outgoing momenta of the fields declared in the `out` statement (always in accordance with the order in which they were declared, as in the above statements).

14. The loops statement (extended form)

From the very start, the `loops` statement allowed users to specify exactly one *loop number* (or loop order, or cycle-rank, or number of loops), for instance

```
loops = 2 ;
```

It is now possible to instruct the program to generate in a single run all the diagrams for a number of consecutive loop numbers, for example

```
loops = 1 to 4 ;
```

or even

```
loops = 4 thru 1 ;
```

These two statements are not fully equivalent, even though the keywords `thru` and `to` can be exchanged with one another. The program starts with the leftmost specified value and then progresses sequentially towards the other value by adding or subtracting 1 in each iteration, as appropriate. The diagrammatic constraints specified by other statements apply to every loop order, obviously.

Usually, the propagators do not contribute directly to the order of perturbation theory (they contribute with a null weight, let us say) while a vertex of degree d (≥ 3) contributes with weight $(d/2) - 1$, which is positive. Thus, it may be more convenient to consider the expansion of the amplitude in some coupling constant(s) (or some other appropriate factors) rather than on the loop number, so that the weights are integer (in which case they can be accepted by a `vsum` statement, for instance). For example, in a simple gauge theory, at tree level, cubic vertices are proportional to some coupling constant g , quartic vertices to g^2 , and propagators do not depend on g .

Nonetheless, the input loop number does not always represent the order of perturbation theory. When (some of) the vertices and/or propagators represent higher order corrections instead of the tree-level interactions and propagators, the loop number becomes an ‘apparent’ quantity, unrelated to the order of perturbation theory. It will then be necessary to deal with propagators and/or vertices with ‘unusual’ weights. Often, such cases can be addressed by performing multiple runs for different values of the loop number, with specially adjusted parameters for each run — including the numerical arguments for the `vsum` and/or `psum` statements. Then, the extended `loops` statement might be able to reduce the number of required runs.

There is a marked difference between dealing with usual or with unusual weights, however. With the usual (implicit) weights, a single run with an extended `loops` statement generates the requested diagrams for all of the corresponding orders of perturbation theory; in the other case, and assuming that the order of perturbation theory can be defined in terms of the number of loops and of a *single* (explicit) weight, each run generates the required diagrams for just one such order.

NB: The current potential of this extended statement is still a bit limited, but that should change when (and if) some other improvements are introduced.

15. The options statement

The `options` statement can be used to specify various diagrammatic constraints, most of which of a ‘topological’ nature. To discard certain types of diagrams (such as 1-particle reducible diagrams, diagrams with tadpoles, and so on) one simply has to include the appropriate keywords separated by commas, for example

```
options = nobridge, noselfloop ;
```

If no such keywords are used, the program will try to generate all the connected diagrams that satisfy the constraints imposed by the control-file statements only.

In (classical) graph theory, there are certain graph transformations that involve the deletion of nodes and/or edges; any edge is indivisible and must be automatically removed if any of its endnodes is deleted. A different possibility, which seems more natural in a QFT context, consists in defining an edge as a pair of linked *half-edges*. Further, a vertex of degree d will have d half-edges of its own, each of which may be linked to some other half-edge (possibly from the same vertex). It will then be possible to *split* or *cut* any edge into its constituent half-edges, thereby preserving the vertices involved; then, *deleting a vertex* will mean cutting every incident edge first, and only then deleting the full (isolated) vertex.

15.1 The keywords for the options statement

The current list of *topological* options follows — each ‘main’ option on the *lhs*, its ‘complementary’ option on the *rhs*, forming pairs of dual options. The main option of a given pair is the one that seems more likely to be used.

bipart	nonbipart
cycli	cyclr
nobridge	bridge
nodiloop	diloop
noedge	edge
noparallel	parallel
norbridge	rbridge
nosbridge	sbridge
noselfloop	selfloop
nosigma	sigma
nosnail	snail
notadpole	tadpole
onepi	onepr
onevi	onevr
onshell	offshell
onshellx	offshellx

These options can be used to impose various types of restrictions on the unlabelled topologies of the generated diagrams, *irrespective* of the fields involved. A few obsolescent options (ie `topol`, `simple` and `notsimple`) will still be available in `qgraf-4`.

There are also a few options of other kinds, shown next.

```
floop          notfloop
new_elinks
new_loops
new_partition
new_topology
```

15.2 The main options

The remainder of this section contains a description of those options, in alphabetical order — but, typically, only for the ‘main’ option of each dual pair. When a dual exists, each option rejects what its dual selects, and conversely; hence there is no need for repeating this part of the definition over and over.

• bipart

This option selects those diagrams whose topology is a bipartite graph.

A bipartite graph is a graph whose node-set can be partitioned into two subsets A and B in such a way that every edge (if any exists) joins a node $u \in A$ to a node $v \in B$. That is the same as selecting the diagrams with no circuit (or ‘loop’) of odd length; in particular, the diagrams must have no self-loops (nor triangles, pentagons, and so forth). Every tree is clearly bipartite.

The dual option (**nonbipart**) requests at least one odd circuit.

• cycli

This option selects those diagrams that have at most one ‘cycle-block’.

Given the usual types of Feynman rules in momentum space, the evaluation of some diagrams involves a factorizable integration, ie decomposable into a product of two or more independent integrations (for 2-loop and higher order diagrams, obviously). Even if the complete integrand does not have such a property it might be possible to decompose it into a sum of factorizable expressions, if the diagram topology is right.

A circuit (or simple cycle, which physicists may call a loop) of a graph G is a non-empty set of edges that, together with their endnodes, define a connected subgraph of G in which every node has degree 2. It follows from this definition that a circuit cannot be decomposed into two or more circuits. Recall that an edge of a graph is either a bridge (if it does not belong to any circuit) or a nonbridge.

Definition: a *cycle-block* (or cycle-component) of a graph G is a non-empty set S of nonbridges such that

- for every circuit C of G , either $C \subseteq S$ or $C \cap S = \emptyset$;
- given any two edges $e_1, e_2 \in S$, there is a circuit containing e_1 and e_2 .

Circuits of length 1 and 2 are allowed: the former case corresponds to self-loops, the latter to pairs of parallel edges (self-loops excluded) as in the topologies depicted in Fig. 2.

There is a simple interpretation of cycle-blocks in terms of sub-diagrams. Let D be a connected l -loop, non-tree diagram, and P_c the set of its nonbridge propagators; also, let $k_1, k_2 \dots k_l$ be the independent integration momenta, and ignore the external momenta (set them to zero, for example). Then, each cycle-block is a minimal (ie non-decomposable into two or more sets of the same kind) non-empty subset S of P_c such that the two sets of momenta flowing through the propagators in S and $\bar{S} = P_c \setminus S$, respectively, can be parametrized independently. The case in which $S = P_c$ and $\bar{S}$ is empty is implicitly allowed, as there may exist a single cycle-component.

The diagrams selected by option `cycli`, which stands for *cycle-irreducible*, are (thus) those whose topology has a non-factorizable cycle space. Since `cycli` ignores bridge-type propagators, it is the following combination

`options = cycli, onepi ;`

which selects sets of diagrams even more ‘primitive’ than those selected by option `onepi` only.

Any self-loop is a cycle-block. Topology T_1 (Fig. 2) has two self-loops and thus two cycle-blocks (hence it is rejected by `cycli`); denoting the momenta of propagators 1 and 2 (as marked) by k_1 and k_2 , one may readily see that (a) those two momenta are independent, and (b) there is no propagator whose momentum has to be expressed as a linear combination in which the coefficients of k_1 and k_2 are both nonzero.

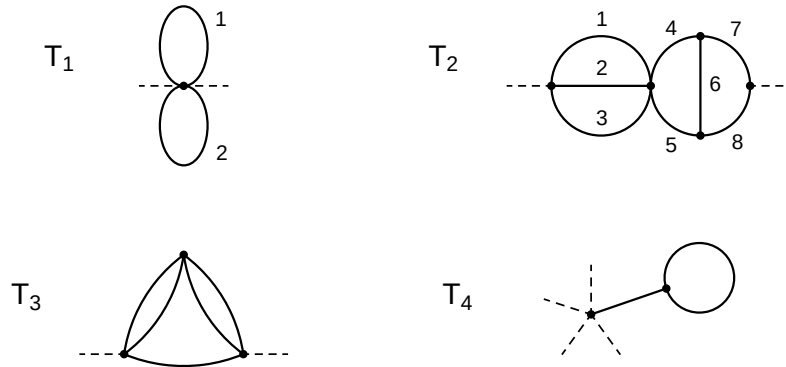


Fig. 2. Illustrating option `cycli`.

Topology T_2 has also two cycle-components, defined by the subsets of propagators $\{1, 2, 3\}$ and $\{4, 5, 6, 7, 8\}$. Ignoring the contributions of the external momenta, here it is possible to express (say) k_3 in terms of $\{k_1, k_2\}$, and $\{k_5, k_6, k_8\}$ in terms of $\{k_4, k_7\}$. Since one may do so for each of the above (disjoint) subsets, T_2 is rejected by `cycli` too. That type of partitioning is not possible for topology T_3 , which is selected. Any tree and 1-loop topology (for instance T_4) is selected by `cycli`.

• `floop`

The diagrams rejected by option `floop` are those that have at least one circuit (ie loop) of odd length for which every propagator is that of an anti-commuting field. In `qgraf-3.2` and later versions the use of `floop` requires that

- there is at least one anti-commuting field;
- every vertex has either 0 or 2 anti-commuting fields.

- **new_elinks**

This option selects only one diagram per labelled topology — more precisely, it selects those diagrams for which `<new_elinks>` produces a ‘1’.

Option `new_elinks` may simplify the task of determining the distinct labelled topologies of the output diagrams (that is, topologies with assigned external fields). It is incompatible with options `new_loops`, `new_partition` and `new_topology`.

- **new_loops**

This option selects only one diagram per loop number — ie it selects the diagrams for which `<new_loops>` produces a ‘1’.

It is incompatible with options `new_elinks`, `new_partition` and `new_topology`.

- **new_partition**

This option selects only one diagram per vertex-degree partition — that is, it selects the diagrams for which `<new_partition>` produces a ‘1’.

It is incompatible with options `new_elinks`, `new_loops` and `new_topology`.

- **new_topology**

This option selects only one diagram per unlabelled topology — that is, it selects the diagrams for which `<new_topology>` produces a ‘1’.

Option `new_topology` may simplify the task of determining the distinct unlabelled topologies of the output diagrams. It is incompatible with options `new_elinks`, `new_loops` and `new_partition`.

- **nobridge**

This option rejects topologies with bridges (thus, `nobridge` is equivalent to `onepi`).

- **nodiloop**

This option rejects topologies with circuits of length 2 (that is, any two distinct vertices may be joined by at most one propagator).

- **noedge**

This option excludes diagrams with edges (or propagators). Thus, it selects only those diagrams that consist of a vertex whose half-edges correspond to the external fields of the input process. More than one diagram may be produced if the input model includes duplicate vertices.

- **noparallel**

This option excludes diagrams with parallel edges (ie no di-loops, and each node can have at most one self-loop).

- **norbridge**

This option rejects diagrams with *regular* bridges.

A bridge is regular if its splitting divides the (connected) diagram into two sub-diagrams, both of which include at least one external field (that is the type of bridge which is *not* associated with tadpoles). Option **norbridge** can be used to generate the diagrams for those correlations functions analogous to 1-particle irreducible correlations functions, but possibly with tadpoles (ie with singular bridges).

- **nosbridge**

This option excludes diagrams with *singular* bridges.

A bridge is singular if its splitting breaks the (connected) diagram into two sub-diagrams such that (at least) one of the sub-diagrams does not include any external field. In other words, a singular bridge is a non-regular bridge, and a regular bridge is a non-singular bridge.

- **noselfloop**

This option excludes diagrams with self-loops.

A self-loop is an edge whose endnodes coincide (ie which forms a circuit of length 1).

- **nosigma**

This option excludes diagrams with ‘*self-energy*’ insertions (ie sub-diagrams that are 2-point functions), anywhere. Since the criterion is topological, it can exclude diagrams that are not higher-order corrections of a tree-level propagator — that is, diagrams with ‘mixed’ propagators are also rejected.

- **nosnail**

This option is an extension of **notadpole** that also rejects diagrams which contain what one might call ‘*tadpoles without tail*’ (or ‘*collapsed tadpoles*’).

- **notadpole**

This option is equivalent to option **nosbridge**.

- **onepi**

This option (which is equivalent to option **nobridge**) excludes diagrams with bridges, usually dubbed *1-particle reducible* diagrams.

A 1-particle irreducible diagram is a connected diagram that cannot be disconnected by cutting any single propagator.

- **onevi**

This option selects those diagrams whose topology has no articulation point, or cut vertex (in the graph theoretical lingo). This means those diagrams that remain connected

upon the removal of any single vertex. By definition, the empty graph is connected; consequently, diagrams discarded by option `onevi` must have at least three vertices. The dual option is `onevr` — `onevi` means *1-vertex irreducible*, and `onevr` means *1-vertex reducible*.

In the case of 1-particle irreducible diagrams there is some overlap between options `cycli` and `onevi`, but they do not coincide. If both bridges and self-loops are excluded, however, then they become identical.

In Fig. 3, T_1 and T_2 are 1-vertex reducible: deleting any vertex that does not link to an external field generates two disjoint components. Topology T_3 is 1-vertex irreducible as it has too few vertices to be reducible. Some of the previous figures provide useful examples too: diagram D_3 in Fig. 6, and diagram T_2 in Fig. 2 are 1-vertex reducible.

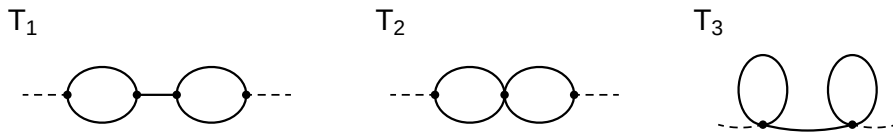


Fig. 3. Illustrating option `onevi`.

There is a class of topologies that are both 1-particle reducible and 1-vertex irreducible, for which T_3 (Fig. 3) may be seen as a kind of template: they have exactly two nodes (u and v , say), joined by a single edge; u and v may have multiple self-loops, depending on the model; some of the legs link up with node u , and the others with v , obviously.

- `onshell`

This option rejects diagrams with ‘*self-energy*’ insertions (ie 2-point functions) on the external lines — where a propagator with a momentum equal to that of an external field is present. As the criterion is topological, those 2-point functions need not be propagators of the classical Lagrangian). See also options `nosigma`, `onshellx`, and the `plink` statement.

- `onshellx`

Option `onshell` discards diagrams that have a propagator whose splitting generates two separate diagrams, provided one of those is a 2-point diagram. In that case there will be a bridge-type propagator whose momentum equals one of the external momenta (as in diagrams D_2 and D_6 (Fig. 4), where the bridge is labelled with the letter b and the corresponding external line with the letter e).

Occasionally, a more extensive elimination may be useful. Option `onshellx` also rejects diagrams for which the above mentioned bridge is absent, as if it had contracted to a single point and its two end-vertices had merged (compare D_1 and D_2 , for example). Diagrams rejected by `onshellx`, but not by `onshell`, will have at least one vertex of degree $d \geq 4$. As those diagrams can be generated by using the combination

```
options = offshellx, onshell ;
```

it is possible to do a basic cross-check in the case where they are thought to evaluate to zero.

Diagram D_1 is rejected by option `onshellx` (but not by `onshell`), whether or not there exists (in the same model) a similar diagram D_2 with the same fields except for the bridge

b. Diagram D_2 is discarded by either option, obviously.

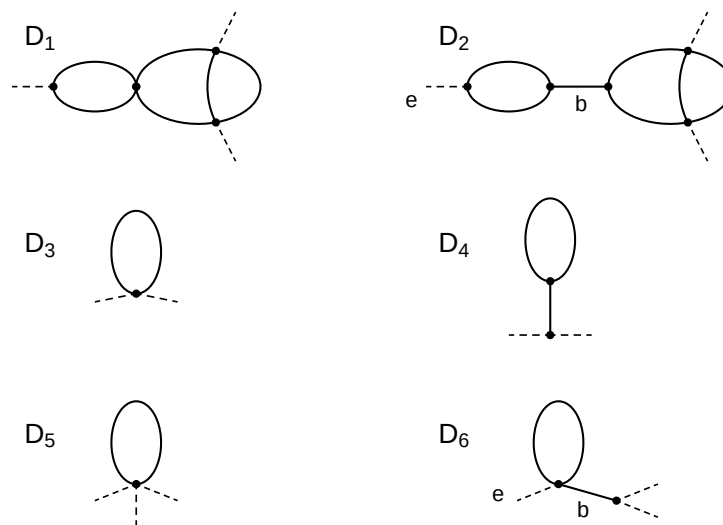


Fig. 4. Illustrating option `onshellx`.

Diagram D_3 is not rejected by `onshellx`, unlike D_5 . Although D_3 can be obtained by contracting the bridge in D_4 , that bridge does not isolate a single leg (and no suitable diagram exists), whilst D_5 can be obtained from D_6 , which has a suitable form. If options `onshellx` and `nosnail` are used simultaneously then all six diagrams will be rejected.

Regarding cross-checks that have been performed so far, the numbers obtained for the combination

```
options = offshellx, nosnail ;
```

agree with several numbers from L. Dixon (private communication) and S. Badger for 1-loop and 2-loop diagrams, respectively. Additionally, a second algorithm has been developed for `onshellx`, and an agreement with the original algorithm was found.

- `simple` (obsolescent)

This option excludes topologies with self-loops and di-loops. The following equivalence applies.

$$\text{simple} \Leftrightarrow \text{noselfloop}, \text{nodiloop}$$

- `topol` (obsolescent)

This option discards diagrams whose unlabelled topology is equal to that of an earlier generated diagram (to be used with a single neutral field, only).

Option `topol` can be replaced by `new_topology`, which is more general.

16. The partition statement

The *vertex-degree partition* of a given diagram is the partition of the number of vertices in that diagram by vertex-degree, represented as $(d_1)^{m_1} (d_2)^{m_2} \dots (d_k)^{m_k}$ with $\{d_1, d_2, \dots, d_k\}$ being the set of (distinct) vertex-degrees of the respective model, and each m_i the number of vertices of degree d_i (in that diagram). Let, conventionally, $d_i < d_j$ for $i < j$.

Typically, the diagram generation extends over the vertex-degree partitions that are compatible with all of the following inputs: the model, the number of external fields of the scattering process, and the order of perturbation theory (in general, that compatibility is not sufficient to ensure the existence of corresponding diagrams). The **partition** statement can be used to restrict the diagram generation to a subset of those compatible partitions. For example, if a model has cubic and quartic vertices only, the statement

```
partition = 3^2 4^1 ;
```

requests that any generated diagram should have precisely two vertices of degree 3 and one vertex of degree 4. Suppose now that the set of vertex-degrees of the input model is $\{3, 4, 5, 6\}$. Then any missing term will be ignored, and the above statement may select more than one partition; in fact, it will select every compatible partition of the form $3^2 4^1 5^a 6^b$, if any, where a and b are ‘free’ (ie not constrained by that statement). Similarly, the statement

```
partition = 3^0 ;
```

rejects diagrams with a (any) cubic vertex. A term d^m should not appear in the **partition** statement unless the model includes some interaction vertex of degree d . To pick a single partition it may be necessary to specify nearly all (or even all) of the possible terms. In the above examples each term sets the exact number of vertices of some degree, but it is also possible to impose bounds instead. For example, the statement

```
partition = 3^(1+) 4^1 5^(1-) ;
```

requests at least one cubic vertex, exactly one quartic vertex, and at most one quintic vertex. As the next example shows, some **partition** statements may be trivial.

```
partition = 3^(0+) ;
```

If the control-file includes a **partition** statement, the numbers of diagrams for excluded partitions are not displayed (nor computed, of course) but the partitions themselves may be displayed. The following display-output excerpt matches the first of the above statements.

-	4^2		
3^2	4^1	...	10
3^4	-		
			10

It should be fairly obvious that the **partition** statement may be employed as a crude ‘parallelization’ method — that is, to divide some diagram generation task, unequally, among a few CPU-cores or a few computers.

17. The `index_offset` statement

When combining the output-files of two or more runs into a single file, it may be useful not to have the first diagram of every output-file being assigned the diagram index ‘1’. The `index_offset` statement instructs the program to add a non-negative integer to the default diagram index, for instance

```
index_offset = 1071 ;
```

The following examples show possible applications of the `index_offset` statement (more complex examples can be devised, of course). Let us suppose in the first place that the diagram selection criteria involve not a conjunction like

```
true = A ;
true = B ;
```

where A and B represent valid expressions, but some other logical connective, for example

$$(\text{ true } = A) \vee (\text{ true } = B).$$

Although this type of statement is not accepted, there is a way out: that inclusive disjunction can be split into three mutually exclusive cases that can be run separately, namely (case 1)

```
true = A ;
false = B ;
```

then (case 2)

```
false = A ;
true = B ;
```

and finally (case 3)

```
true = A ;
true = B ;
```

Similarly, the exclusive disjunction can be divided in two non-overlapping cases, the equivalence $A \Leftrightarrow B$ in two cases also, and so on.

As a second example, suppose that one wishes to generate a set of diagrams satisfying the following two conditions: (i) the selection criteria can be expressed as a conjunction of valid statements (hence there is in principle no need to split the run into multiple runs) but (ii) the set of diagrams should be partitioned into two subsets, each of which to be processed in a distinct way (or on different computers). Then, if that partitioning can be expressed as a single constraint (A , say), one may proceed as if there was an additional exclusive disjunction with two cases — namely (case 1)

```
true = A ;
```

and (case 2)

```
false = A ;
```

Obviously, this kind of ‘trick’ can be employed to divide a set of diagrams into two or more subsets even when the `index_offset` statement is not needed.

18. The `count_to` statement

The `count_to` statement (available in auto-mode only) sets a limit on the number of diagrams to be generated, automatically preventing the creation of any output-file. More precisely, the program stops as soon as one of the following targets is reached:

- the number of generated diagrams becomes equal to the positive integer declared in the `count_to` statement;
- no further diagram can be found.

That type of statement is intended solely as a means of obtaining some preliminary information about the size of a certain set of diagrams, for instance to decide whether or not the respective output-file(s) should be created in a subsequent run. If no other information is available at the time, the exact number of diagrams will not be known at the end of the run unless it is strictly smaller than the value declared in that statement. For example, if the statement

```
count_to = 46340 ;
```

is included in the control-file, the program will generate at most 46 340 diagrams. If there are fewer diagrams, the total number reported should be the correct number, otherwise merely a lower bound. In the latter case, this uncertainty will be reflected in the display-output in two different ways: by prefixing the (usual) string `'total'` with `'min_'`, and marking the partial diagram numbers that may be in doubt (ie the counts for the last vertex-degree partition and the last subtotal). An example of that kind of display-output is given in Section 41.1, near the end. Naturally, not every (compatible) partition and loop number is necessarily displayed.

Those modifications of the display-output apply also when the full number of diagrams is equal to the limit declared in the `count_to` statement, of course. While in a normal run the program would try (and fail) to generate a further diagram, the presence of this type of statement will prevent that attempt; consequently, the program will not detect that the diagram generation is actually complete.

Although the `count_to` statement is not available in API-mode there is in that case a different method, based on the available interface, to instruct the program to perform the same task.

19. The `zero_momentum` statement

Once momentum conservation is enforced at every vertex, the momentum of any propagator associated with a singular bridge (a “tadpole’s tail”) will be equal to zero. That zeroness may lead to minor problems (or inconveniences, let us say) for the amplitude generation since the null momentum is, by default, represented by the expression ‘0’. The `zero_momentum` statement, for example

```
zero_momentum = k0 ;
```

makes it possible to declare an identifier that will become the redefined null momentum. That momentum (in the above statement, `k0`) must be distinct from the other basic momenta, namely external and integration momenta.

The sole consequence of that redefinition is this: when a null internal momentum is found and the respective expression must be produced, the program outputs that identifier instead of ‘0’. For example, it is now possible to output an expression like ‘`k0_{mu4}`’ instead of ‘`0_{mu4}`’.

The `zero_momentum` statement may help simplify the implementation of a proper ‘space-time index algebra’ (in those cases where null momenta can be produced), with every 4-momentum (or perhaps D -momentum) carrying its own space-time index. In addition, regarding the symbolic manipulation of the amplitudes, dealing with the actual zeroness of that momentum can then be postponed to some later step of the computation, if convenient.

The natural use of this statement is associated not only with the `<momentum_loop>` construct (Section 35.4) but also with the `<momentum>` style-keyword.

20. The `elink` statement

The `elink` statement provides a way to restrict the configuration of the external fields — that is, diagrams can be selected or rejected depending on whether or not some external fields are attached to the same vertex or (more generally) to a certain number of vertices. Some constraints of this type, to be dubbed external linking conditions, can be simulated by constructing an extended model — with new **external** fields and new vertices — but that approach can be exacting.

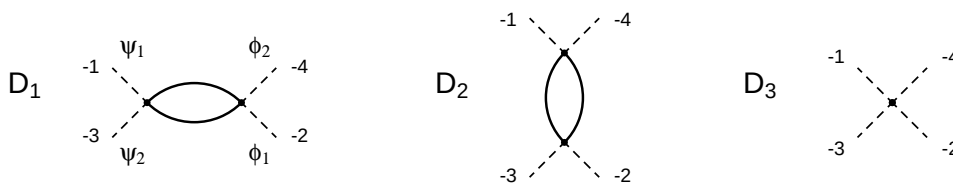


Fig. 5. Illustrating the `elink` statement.

Let us take a look at some examples, assuming the generic scattering process

$$\psi_1 \psi_2 \rightarrow \phi_1 \phi_2.$$

The field-indices of those external fields are equal to -1 , -3 , -2 , -4 (from left to right). The statement

```
true = elink[ -1, -3, incl, 1, 1 ];
```

selects those diagrams in which ψ_1 and ψ_2 link up with the same vertex, *inclusively* — meaning that the other external fields can be attached to any vertex, as far as this statement is concerned. For example (see Fig. 5), diagrams like D_1 and D_3 would be validated, and D_2 rejected. This other statement

```
true = elink[ -1, -3, excl, 1, 2 ] ;
```

selects those diagrams in which ψ_1 and ψ_2 link up (in total) with either one or two vertices, this time *exclusively* — ie ψ_1 and ψ_2 may or may not be attached to the same vertex, but in either case the other external fields (ϕ_1 and ϕ_2) should link up with *other* vertices. This condition rejects diagrams D_2 and D_3 , for instance.

A generic `elink` statement includes three types of arguments. Each argument of the first type should be the field-index of an external field, and therefore a negative integer (no repetitions are allowed). Then comes a non-numerical argument, either `excl` or `incl`, to specify whether the linking condition is exclusive or inclusive. The third set of arguments consists of two positive integers (a and b , say) which define the range for the number of vertices involved in the linking condition — that is, for the number of vertices that link up with at least one of the external fields whose field-index is an argument of that `elink` statement. If m denotes the number of arguments of the first type and n the number of legs then the inequalities

$$1 \leq a \leq b \leq m \leq n, \quad n \geq 2,$$

should hold, otherwise an error will occur. Here are some more examples: the statement

```
false = elink[ -1, excl, 1, 1 ];
```

requests ψ_1 to link up with some other external field(s), whilst the statement

```
false = elink[ -1, -2, incl, 1, 1 ];
```

requests ψ_1 and ϕ_1 not to link up with the same vertex (a condition which diagram D_3 fails to satisfy). For the above mentioned process, the next statement requests that no two external fields link up with the same vertex (actually, in this example `excl` would also do).

```
true = elink[ -1, -2, -3, -4, incl, 4, 4 ];
```

Then, in the special case of 1-loop diagrams, that statement selects ‘squares’, or ‘boxes’ — if, as usual, the diagrams are connected and the input model does not include vertices of degree smaller than 3. That possibility can be generalized to select diagrams with a circuit of length k , or k -circuit, from a set of n -leg 1-loop diagrams ($n \geq k$) — a triangle, a square or a pentagon, for example (even a 2-circuit and a 1-circuit). In this case (using the above convention) one should have an inclusive statement with $m = n$ and $a = b = k$.

Some `elink` statements are trivial. The next three statements provide examples of that assertion, still in the case of the above mentioned process.

```
true = elink[ -1, incl, 1, 1 ];  
true = elink[ -1, -3, incl, 1, 2 ];  
true = elink[ -1, -2, -3, -4, excl, 1, 4 ];
```

Their negation rejects every diagram, of course.

21. The plink statement

This statement addresses the case in which the selection criterion consists in either the absence or existence of a bridge-type propagator with a specific nonzero momentum (this requires two or more legs). The ability to either disallow or require the existence of such propagators may help select diagrams whose amplitude has some type of resonance, or tree diagrams contributing to the s -, t - and u -channels, for example; in the case of non-tree diagrams it also provides a way to have some external fields ‘on-shell’ and others ‘off-shell’ (as a kind of selective `onshell` option).

If a connected diagram D has a regular bridge then splitting that edge will produce two connected sub-diagrams. Let X_1 and X_2 be the non-empty subsets containing the external fields of D in each of those sub-diagrams; also, let $\mathbf{p}_i$ and $\mathbf{q}_j$ denote the incoming and the outgoing momenta. The momentum $\mathbf{P}$ flowing through that bridge may be then written as

$$\mathbf{P} = \sum_i a_i \mathbf{p}_i - \sum_j b_j \mathbf{q}_j$$

with $a_i, b_j \in \{0, 1\}$; the orientation will be ignored (in any case, the relative signs of the coefficients in the above relation are fixed). As the external momenta satisfy the momentum conservation relation $\sum_i \mathbf{p}_i = \sum_j \mathbf{q}_j$, it is clear that $\mathbf{P}$ can be expressed as a linear combination of the external momenta in two different ways.

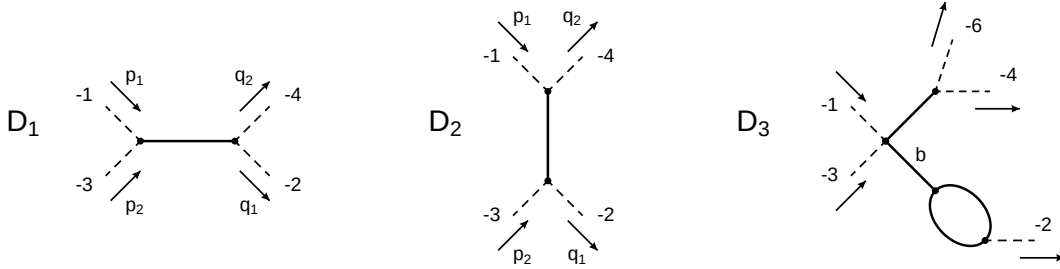


Fig. 6. Illustrating the `plink` statement.

The arguments of the `plink` statement should be the field-indices of the external fields in either X_1 or X_2 . The number of arguments cannot be equal to zero nor equal to the number of legs, as $\mathbf{P}$ would then be null. Let us take a look at some examples. The statement

```
true = plink[ -1, -3 ];
```

selects those diagrams that have at least one propagator with momentum $\mathbf{p}_1 + \mathbf{p}_2$, such as D_1 (Fig. 6). Diagram D_2 , which has a propagator with momentum $\mathbf{p}_1 - \mathbf{q}_2$ (or $\mathbf{q}_1 - \mathbf{p}_2$), would be selected by either of the next two statements.

```
true = plink[ -1, -4 ];
```

```
true = plink[ -3, -2 ];
```

The `plink` statement can also be used to set only part of the external fields ‘on-shell’. For instance, the statement

```
false = plink[ -2 ];
```

rejects any diagram with a bridge separating the external field with index -2 from every other external field, such as diagram D_3 (which has one such bridge, labelled with a ‘ b ’).

22. The `vsum` and `psum` statements

The ability to define functions in the model-file — namely parameters associated with fields, propagators, and vertices (Section 26.3) — automatically suggests a few diagram selection criteria. The `psum` and the `vsum` statements make it possible to impose some numerical constraints involving integer functions (either p-functions or v-functions). To be clear, such functions have to be declared in the model-file with the `integer` specifier.

22.1 The `vsum` statement

The implementation of a method to restrict the sum(s) of the powers of the coupling constant(s) (for example) seems particularly relevant, specially in models with two or more independent coupling constants. Partial radiative corrections based on subsets of diagrams defined by such constraints have been routinely considered in the particle physics literature.

Let `g_power` be a v-function mapping each vertex to an integer equal to the power of some coupling constant g in the Feynman rule for that vertex. To select the diagrams whose contribution to the amplitude is proportional to a certain power of g , a statement similar to

```
true = vsum[ g_power, 4, 4 ] ;
```

may be employed. In general, the first argument of `vsum` is a v-function and the other two arguments are integers, possibly negative, the first not larger than the second. These integers (n_1 and n_2), say, define the allowed range for the summation of the values of that v-function over the vertices of the diagram. If the keyword `true` is used, the allowed range consists of the integers z such that $n_1 \leq z \leq n_2$. If the keyword `false` is used instead, the previously allowed range becomes the excluded range. Thus, in the above example, the sum of the values of `g_power` has to be exactly 4. Since the function values have to be integer, definitions like

```
g_power = '1/2'
g_power = ' +2'
```

will not be accepted. In principle, it is possible to convert a constraint involving rationals into another constraint depending on integers only; in practice, such integers should not be too large (Section 7.1).

The `vsum` statement can be used for purposes other than to select the powers of coupling constants, obviously. For example, let `one` be a v-function that maps every vertex to '1'; if `one` is used as the first argument of a `vsum` statement then the number of vertices in each diagram will be restricted. Note that such a function can be completely defined with only two statements, namely

```
[ integer v_function :: one ]
[ ;; one= 1 ]
```

independently of the number of vertices of the model. Here is another example: let V_1 be a subset of the interaction vertices, and `vbinary` a v-function that maps vertices in V_1 to '1' and vertices not in V_1 to '0'. Then, the statement

```
false = vsum[ vbinary, 0, 0 ];
```

selects diagrams containing at least one vertex in V_1 , while the statement

```
true = vsum[ vbinary, 0, 0 ];
```

selects diagrams that contain no vertex in V_1 . In the latter case, defining submodels may be considered as an alternative method.

22.2 The `psum` statement

There is an analogous statement for p-functions. For example, the statement

```
true = psum[ pweight, -1, 1 ] ;
```

restricts the sum of the values of the p-function `pweight` taken over the diagram propagators. Still, there is a nontrivial difference between the `psum` and `vsum` statements, apart from the fact that they work with distinct types of functions. Given that the program does not produce tree-level 2-point functions, any generated diagram has at least one vertex; therefore, any implicit summation for `vsum` includes at least one term. Nevertheless, there exist tree-level diagrams without propagators — namely the diagrams that consist of the tree-level vertices of the model, which we may call *stars*. There are also tree-level scattering processes with diagrams of ‘mixed-type’, ie some diagrams are stars while others contain one or more propagators.

Conventionally, a null term is assigned for the stars, but the user should check whether that is indeed the desired action and, if not, make any necessary adjustments. It might be necessary to generate the stars in a different run (if they are required, but are excluded by some `psum` statement which must be present to deal with the non-stars), or else the stars may have to be excluded by some additional statement (if stars are to be discarded, but are allowed by the current statements). The diagrammatic options `edge` and `noedge` might help.

22.3 Additional comments

It is possible to define a selection range of non-consecutive integers by using multiple statements, with distinct bounds and keywords. For example, the allowed range defined by the statements

```
true = vsum[ f1, -4, 11 ];
false = vsum[ f1, -2, 2 ];
false = vsum[ f1, 8, 10 ];
```

is equal to $\{-4, -3, 3, 4, 5, 6, 7, 11\}$.

Asymptotically, these ‘filters’ — ie `vsum`, for a large number of vertices, and `psum`, for a large number of propagators — do not (moreover, cannot¹⁰) have efficient implementations if they are general enough. In a few particular cases, though, some speed-ups may be implemented. In fact, both `qgraf-3.5` and (specially) `qgraf-3.6` brought noticeable improvements; apart from that, in some cases the impact of this (in)efficiency issue can be lessened by defining submodels (and more rarely, using the `partition` statement).

¹⁰ At least assuming the widely held belief that the algorithmic complexity classes P and NP do not coincide (ie $P \neq NP$).

23. Other statements (additional filters)

Some other constraints can be imposed on the edges and propagators, with the help of a further set of five statements. These optional statements should be the very last ones to appear in the control-file, and they are of the form

$$(\text{logical}) = (\text{operator}) [(\text{arg}_1), (\text{arg}_2) \dots, (\text{arg}_k)] ;$$

where ‘(logical)’ should be replaced by either **true** or **false**, and ‘(operator)’ by one of the following identifiers.

```
bridge
chord
iprop
rbridge
sbridge
```

The number of arguments (ie k) should be at least 2, and the last two arguments should be non-negative integers (which are subject to the bounds stated in Section 7.1) such that the last argument should not be smaller than the other. The remaining arguments (if any) should be fields.

For example, to restrict the number of propagators of a certain field **phi**, a statement of the following kind can be used.

```
true = iprop[ phi, 1, 3 ] ;
```

This statement selects diagrams for which the number of **phi** propagators is at least 1 and at most 3. If **true** is replaced by **false** then the diagrams selected are the ones for which the number of **phi** propagators is either less than 1 or greater than 3. It should not matter whether a propagator is represented by the particle or the anti-particle.

On other occasions one might be interested in propagators with certain topological properties. The ‘filters’ **chord** and **bridge** restrict the number of propagators that belong (respectively, do not belong) to a *circuit* (or *loop*); thus, ‘chord’ constrains non-bridges and the respective propagators. For example, the statement

```
true = chord[ photon, 0, 0 ] ;
```

requires that there is at least one **photon** propagator in some circuit. All of these operators may have several fields as arguments, for example

```
true = bridge[ photon, electron, 2, 2 ] ;
```

in which case the sum of **photon** and **electron** bridges is constrained, or even no field at all as in

```
true = chord[ 1, 2 ] ;
```

where the total number of chords is restricted.

The **rbridge** and **sbridge** statements are similar, and they constrain the propagators attached to a *regular bridge* and a *singular bridge* (respectively), where the following terminology applies. If a propagator is attached to a bridge then its momentum does not depend on the integration momenta; a bridge is singular if that momentum is identically null, irrespective of the external momenta (as in a tadpole with a ‘tail’), and is otherwise regular.

It may be observed that the above types of statements may obviate the need for the optional keywords `notadpole` and `external` in the model-file. In fact, statements like the following produce (respectively) the same effect.

```
true = sbridge[ photon, 0, 0 ] ;  
true = iprop[ Phi, 0, 0 ] ;
```

The model-file is intended as something fairly permanent, not as something one should change only temporarily for a single calculation, specially if the same result can be achieved with the control-file. Both possibilities exist, however.

24. A list of the control-file statement types

The `model_dir`, `output_dir`, `style_dir`, `separator`, `zero_momentum` and `count_to` statements are the latest additions. Below, statements marked with an open circle are not available in API-mode. The strictly required statements follow, in alphabetical order (these statements should appear exactly once).

- `in`
- `loops`
- `model`
- `out`

Still, the next three statements are typically required too (except the `output` statement, in API-mode). The `loop_momentum` statement is required if some output that depends on the integration momenta is to be produced. The `output` and the `style` statements, which may appear more than once, are needed to create the output-file(s); in API-mode, one or more `style` statements are necessary for constructing the output-blocks.

- `loop_momentum`
- `output`
- `style`

The set of optional statements that may appear at most once includes the following.

- `config`
- `count_to`
- `index_offset`
- `messages`
- `model_dir`
- `options`
- `output_dir`
- `partition`
- `separator`
- `style_dir`
- `zero_momentum`

Additionally, there exist the optional statements that involve one of the ‘filters’ listed below, and which may appear more than once.

- `bridge`
- `chord`
- `elink`
- `iprop`
- `plink`
- `psum`
- `rbridge`
- `sbridge`
- `vsum`

25. An example of a modern control-file

```
##    the config options

    config = delete, noblanks ;

##    the path-separator

    separator = '/' ;

##    the class directories
##    (in any relative order)

    output_dir = 'tmp_dir/' ;
    style_dir = 'styles/' ;
    model_dir = 'models/' ;

##    the message-file

    messages = 'msg.txt' ;

##    the style-file(s) and (possibly) the output-file(s);
##    alternating statements if more than one pair exists;
##    no "output" statements in api-mode

    style = 'f1.sty' ;
    output = 'q_list.1' ;

##    the main required statements
##    (in the predefined order)

# model = qed2 // 'qedx' ;

    model = 'qed' ;
    in = mu_minus[p1], mu_plus[p2] ;
    out = photon[q1], photon[q2] ;
    loops = 0 ;
```

(continues on the next page)

```

##   other statements that may appear just once
##   (only the next 6 types, in any relative order);
##   no "count_to" statement in api-mode

loop_momentum = k ;
zero_momentum = k0 ;
options = onshell, nosbridge ;

# partition = 3^5 4^2 ;
# index_offset = 257 ;
# count_to = 524288 ;

##   other constraints --- examples
##   (these statements may appear in any relative order)

# true = vsum[ v_weight, 2, 4 ];
# false = psum[ p_weight, 0, 1 ];
# true = elink[ -1, -3, incl, 1, 1 ];
# false = plink[ -2 ];

```

Part III — The model-file

26. A basic language for describing models

The input model should be described in the *model-file* (to be supplied by the user, or made available in some package). This section discusses only (the basic) part of the language that can be used to describe input models; several other statement types are presented in the next section.

A model-file can be divided into implicit sections (or *zones*), of which only two are always required. If the model-file includes only the basic statement types described in this section then, apart from any annotations, the first required zone consists of the propagator statements, and the second of the vertex statements; these zones correspond in fact to the fifth and sixth generic zones. For example, a model-file for Quantum Electrodynamics (QED) might look like this:

```
%  propagators
  [electron, positron, -1]
  [photon, photon, +1]
  
%  electromagnetic vertex
  [ positron, electron, photon ]
```

The symbol is a *visible space*, which is used here to render the lines clearer, and any leading ‘%’ is an annotation-mark. In this example, there are three fields (denoted by the identifiers *electron*, *positron* and *photon*), two propagators and one (cubic) vertex.

As no keywords such as **propagator** and **vertex** are present, both propagator and vertex statements may be considered implicit statements; there is no **field** statement, the fields are declared (implicitly) in the propagator statements.

Distinct statements should occupy distinct lines. A statement can occupy several consecutive lines provided the following conditions are met:

- neither annotations nor (effectively) empty lines exist between the first and the last line occupied by the statement;
- the line breaks are consistent — that is, any ‘indivisible’ string (keywords, identifiers, and so on) is contained in a single line;
- the character ‘\’ (backslash) ends any line, except the last, containing only part of the statement (Section 6.3).

26.1 Propagators

The basic propagator statement is of the form

$$[\phi_1, \phi_2, q]$$

where ϕ_1 and ϕ_2 denote fields (ie identifiers) and q the commutation number (‘+1’ for fields satisfying commutation relations, ‘-1’ for anti-commutation relations); the plus sign should not be omitted. It is possible to use just ‘+’ and ‘-’, however, if done consistently (that is, only one convention can be used in each model-file). The field ϕ_1 is the conjugate of ϕ_2 , and conversely. If ϕ_1 and ϕ_2 are identical identifiers then we have a *neutral* or *self-conjugate* field,

else ϕ_1 and ϕ_2 are dubbed *charged* fields. Sometimes ϕ_1 and ϕ_2 are not really particle and anti-particle, as in the case of ghost fields; still, ϕ_1 and ϕ_2 will be called (respectively) the *particle* and the *anti-particle*, in an absolute sense.

The propagator represents a non-trivial contraction — ie vacuum expectation value of the time ordered product of two fields. Graphically, it is simply a type of (possibly oriented, (bi-)coloured) edge that can be used to construct diagrams. The above propagator statement represents the contraction $\langle \phi_1 \phi_2 \rangle$, not $\langle \phi_2 \phi_1 \rangle$, and this is specially relevant in the case of anti-commuting fields. In the above example, and denoting the **electron** field by ψ , the propagator ‘[**electron**, **positron**, -1]’ corresponds to the usual $\langle \psi \bar{\psi} \rangle$ contraction.

26.2 Vertices

The basic vertex statement for defining a vertex of degree n is of the following form.

$$[\phi_1, \phi_2 \dots, \phi_n];$$

Every vertex should have an even number of anti-commuting fields, obviously. Interactions are often cubic and/or quartic, but in some cases (exotic gauges and effective models, for instance) there will be interactions of higher degree. The program will accommodate for most of that: degrees in the range 3–8 are accepted by default.

The field ordering should not be neglected here either. Each vertex statement should represent a term (or set of terms) of the Lagrangian density in which the fields appear in that same order $\phi_1 \phi_2 \dots \phi_n$ (from left to right, with or without derivatives), and the Feynman rule to be used when processing the amplitudes should match that exact ordering too. In the previous example (QED), the vertex ‘[**positron**, **electron**, **photon**]’ corresponds to the usual $\bar{\psi} \psi A_\mu$ ordering.

NB: Any mismatch between propagator and vertex statements (on one side) and Feynman rules (on the other) is potentially disastrous, see in particular Section 43.

26.3 Functions (parameters) and constants

The syntax presented up to this point is the basic, minimal syntax. It lets us set the combinatorial description of the model (ie what kind of edges exist and how they are allowed to meet at the vertices of the diagrams), and whether the fields satisfy commutation or anti-commutation relations. Only an extended syntax may provide the ability to define quantities like spin, electric charge, mass, or even more complex objects representing Feynman rules, for example.

It is possible to define functions for fields, propagators and vertices. Each such function — which maps either fields, propagators, or vertices to character strings — is represented by its own (unique) identifier. A function f with a finite domain can be defined by a list of assignments of the form $x_k \rightarrow f(x_k)$, without using a generic ‘formula’ (not unlike a tabular definition), and that is essentially the method that is used. Moreover, it is also possible to define *constants*, which have a fixed ‘value’. Section 30 presents an example of a ‘modern’ model-file where the relative positions of nearly all the statement types is made obvious.

The constants should be declared in zone 2, for example

```
[ constants :: c1, c2, model ]
```

while the functions should be declared in zone 3. As the previous paragraph might suggest, functions come in three types, and there is a distinct statement for declaring each type. For

example, the statements

```
[ f_function :: ff1 ]
[ p_function :: pf1 ]
[ v_function :: vf1 ]
```

declare `ff1`, `pf1`, and `vf1` as (respectively) a field-function, a propagator-function, and a vertex-function. These are generic functions, for which any image is treated as a character string. The `integer` qualifier should be added when declaring functions that might be used in some constraint (implemented in terms of the statements `vsum` and `psum`), for example

```
[ integer v_functions :: vf2, vf3 ]
```

A suitable modification of the model-file presented earlier will be used to illustrate these features. Here is the new version, this time without visible spaces.

```
% constants
[ constants :: model ]
[ model = 'QED' ]

% functions
[ f_function :: C ]
[ p_function :: m, s ]
[ integer v_function :: g_power ]

% propagators
[ electron, positron, -1 ; C = ('-1', '+1'), m = 'me', \
  s = '1/2' ]
[ photon, photon, +1 ; C = ('0'), m = 'm0', s = '1' ]

% electromagnetic vertex
[ positron, electron, photon; g_power = '1' ]
```

In this example there is a single constant (ie `model`, whose value is `'QED'`), and there are three functions, one of each type. The f-function `C` maps `electron` to `'-1'`, `positron` to `'+1'`, and `photon` to `'0'` (one may think of `C` as the electric charge); the p-function `m` (which might stand for 'mass') maps the fermionic propagator to `'me'`, and the bosonic propagator to `'m0'`, while the p-function `s` (the 'spin', let us say) maps those propagators to `'1/2'` and `'1'`, respectively. Finally, the v-function `g_power` maps the single vertex to `'1'` (this function might represent the power of the coupling constant appearing in the Feynman rule).

A v-function can be defined by including, in each vertex statement, the image of that vertex under that function (see below for the exact notation), and a p-function can be defined in a similar way. Defining an f-function is just slightly less trivial: every propagator statement should include either a single image or a pair of images, according to the number of distinct fields in that propagator. The images should be defined on the *rhs* of the propagator and/or vertex statements, which should be separated from the *lhs* by a semicolon. As it should be clear by now, the syntax for defining the images of vertices and/or propagators is

```
function_id = S
```

where `function_id` is the function identifier and `S` is the image — an encoded string or, if allowed, an unencoded string (see below). In the case of f-functions the syntax is either

```
function_id = ( S1 , S2 )
```

if the propagator fields are charged, or

```
function_id = ( S )
```

in the opposite case (`S`, `S1`, and `S2` denote encoded strings). Clearly, a p-function is a special case of an f-function, and can always be rewritten as such.

The following simplification is allowed in the definition of functions and constants: an image can be written unencoded if it is a valid identifier, integer, or rational (length allowing). Note that unencoded numbers will not be simplified; for example, the definition '`s= +001`' is not equivalent to '`s= 1`'. Thus, some of the statements of the previous model description can be simplified as follows.

```
[ model = QED ]
[ electron, positron, -1 ; C= (-1, +1), m= me, s= 1/2 ]
[ photon, photon, +1 ; C = (0), m= m0, s= 1 ]
[ positron, electron, photon; g_power= 1 ]
```

Here are also some examples of definitions where that kind of simplification is not permitted: `x = ''`, `s = ' 1'`, `sum= '2+1'`, `key= 'a b'`, `v = '(0)'`, `rp = ''`.

26.4 The keywords `external`, `internal` and `notadpole`

There are three optional keywords that serve to further characterize the fields of the model. The keyword `notadpole` prevents the generation of diagrams containing one-point insertions of the field(s) declared in the statement where that keyword is used. For example, if a propagator is defined as

```
[ photon, photon, +1, notadpole ; C = (0), s = 1 ]
```

then diagrams with tadpoles of the field `photon` will be systematically suppressed (this means that no such propagator will have an identically null momentum).

The keyword `external` can be used to declare fields that represent external sources or to suppress propagators with very large masses. For example, the statement

```
[ Phi, Phi, +1, external ]
```

leads to the exclusion of the diagrams with propagators of the field `Phi`. Objectively, such statements do not declare propagators, only fields.

In contrast, the keyword `internal` specifies that the fields to which it applies should never appear as external fields of the scattering process (or correlation function) given as input. For example, if the model-file includes the propagator statement

```
[ phi1, phi2, +1, internal ]
```

then `phi1` and `phi2` should both be excluded from the `in` and `out` statements, otherwise an error condition will occur.

The keyword `external` is incompatible with the other two.

27. An extended language

The input-model description has been overhauled (mainly) with **qgraf-3.6**, and it is now possible to define *sectors*, *submodels*, and *default images* (or default ‘values’) for functions. What can be achieved with this extended language includes (i) a more compact description of ‘complex’ models (that is, models with many propagators and/or vertices, and for which one or more functions have been defined), and (ii) a combined description (in a single file) of a given model and some of its submodels and/or extensions. Creating a combined model-file can be a bit more time-consuming than creating separate files but, should some changes be required later on (adding new functions, or fixing some definition, for example), it would then be possible to update all of those models by editing just one file.

A ‘modern’ model-file consists of up to six *zones* (or implicit sections) though some of them may be empty. An illustrative example of such a file — that may be useful in inferring the sequence in which the different types of statements should appear — can be found in Section 30.

NB: The declaration of various types of identifiers (namely submodels, sectors, constants and functions) no longer has to employ a minimal number of statements. Also, a submodel block may now contain multiple **include** or **exclude** statements (but not statements of both types, nor redundant statements).

27.1 Sectors

A *sector* is a ‘piece’ of the model-file delimited by two statements of the form

```
sector[ sector1 ]
      :
end[ sector1 ]
```

Here, **sector1** is the name of the sector, which in general must be an identifier. Not every type of statement may appear in a sector block; in fact, there are only two types of sectors, each of which includes at most two other types of statements. A *propagator-type* sector includes one or more propagator statements, possibly preceded by other statements (to be discussed later) that define default images for some f-functions and/or p-functions, for instance

```
sector[ sector1 ]
  [ ; pf1= -1 ]
  [ phi, phi, +1 ; pf2= 'A' ]
  [ psi, psi, -1 ; pf2= 'B' ]
end[ sector1 ]
```

Likewise, a *vertex-type* sector should include one or more vertex statements, and may include additional statements that define default images for some v-functions, for example

```
sector[ sector4 ]
  [ ; vf1= 0 ]
  [ phi, phi, phi ]
  [ psi, psi, phi ]
end[ sector4 ]
```

Although propagator and vertex statements need not be part of a sector block, propagator statements should still precede vertex statements and (thus) propagator-type sectors should precede vertex-type sectors. The sectors are declared in zone 1, using one or more appropriate statements for each type of sector, for example

```
[ p_sectors :: sector1, sector2 ]
[ v_sectors :: sector3, sector4, sector5 ]
```

27.2 Sub-models

The existence of sectors makes it possible to define *submodels*, namely by instructing the program to treat some of the sectors as an integral part of the input model and the other sectors as alien. The propagator and vertex statements that are not part of any sector are always part of the definition of every submodel. The submodels are declared at the very top of the model-file (zone 1), and then defined one by one in zone 4. For example, the statement

```
[ submodels :: subm1, subm2 ]
```

declares two submodels, and the submodel block

```
submodel[ subm1 ]
  [ include :: sector1, sector3 ]
end[ subm1 ]
```

defines submodel **subm1** by listing the sectors that it comprises (their order of appearance in the **include** statement being immaterial). A submodel can also be defined by listing the sectors that are not part of that submodel, for instance

```
submodel[ subm2 ]
  [ exclude :: sector5 ]
end[ subm2 ]
```

A submodel that comprises every sector of a model-file can be defined by either listing all the sectors (using one or more **include** statements) or using a ‘void’ **exclude** statement, ie

```
[ exclude :: ]
```

Such a submodel *has* to be defined if (i) the complete model-file describes a (sub)model that the program should be able to use and (ii) at least one other submodel is defined.

Concerning the control-file, one may have either the usual kind of **model** statement if no submodel is defined in the model-file, for example,

```
model = 'qed' ;
```

or else an extended statement that also specifies the submodel to be selected (in the next statement, submodel **qed1**), as follows.

```
model = qed1 // 'qedx' ;
```

27.3 Non-global constants

Constants were introduced earlier. Still, if the model-file describes more than one submodel, it may be useful to define *submodel-dependent* constants. For example, one may wish to assign to each submodel a distinct string representing the name of that submodel, using the same identifier. While the definitions of the submodel-independent (or *global*) constants

should follow the respective declaration(s), still in zone 2, the submodel-dependent (or *non-global*) constants should be defined in every submodel block, in zone 4, unless global default images are defined for them (this alternative method is described in the next subsection). In the next example, `c1` is global but `c2` is not.

```
[ constants :: c1, c2 ]
[ c1= 'g' ]
    ⋮
submodel[ subm1 ]
  [ include :: sector1, sector3 ]
  [ c2= 'qed3' ]
end[ subm1 ]

submodel[ subm2 ]
  [ exclude :: sector5 ]
  [ c2= 'qed1' ]
end[ subm2 ]
```

The first statement in a submodel block should be an `include` or an `exclude` statement; these are the only types of model-file statements that can be *void*.

27.4 Default images (or simply ‘defaults’)

Suppose one wishes to define a function that has many identical images — in other words, a highly non-injective function. In that case, it might be useful to define defaults for that function. Default images can be defined globally and/or per sector, with the sectorial defaults overriding the global ones; sectors need not be defined (just) for this purpose when all the defaults are global.

For each function, it is possible to define one global default and one default per sector. Defaults for distinct functions (and constants) should be defined in distinct statements. For example, the statement

```
[ ;; vf2 = 1 ]
```

(which should appear in zone 3, after declaring `vf2` as a vertex function, say) defines a global default for function `vf2`. As a result, that function no longer has to be defined for every vertex — only those images which differ from that default have to be set explicitly, in the usual way. Additionally, if the model-file includes sectors then sectorial defaults can be defined too, whether or not global defaults exist, for instance

```
sector[ sector3 ]
  [ ; vf2 = 0 ]
    ⋮
end[ sector3 ]
```

It then follows that `vf2` will be equal to ‘0’ for every vertex in sector `sector3` unless that default is overridden by explicit definitions in the vertex statements. Having statements with

either a single or a double semicolon may serve as a reminder that the corresponding defaults can be overridden at one or two other stages, respectively. Defaults for propagator-functions can be defined similarly, but for field-functions there are more possibilities. For example, the statement

```
[ ;; ff1= (0), (-1,1) ]
```

defines a global default for `ff1`, both for neutral and charged fields. Still, since not every model (nor sector) has fields of both types, partial defaults can be defined, for instance

```
[ ;; ff1= (1) ]
[ ; ff1= (0,1) ]
```

Within each sector block, the precedence rules are as follows: statements defining defaults for f-functions or p-functions should precede propagator statements, and statements defining v-function defaults should precede vertex statements. Thus, the last statement of any sector (excluding the `end` statement that closes the sector block) should be either a propagator statement or a vertex statement.

Furthermore, it is possible to define defaults for submodel-dependent constants, even though that feature is probably not very useful. In any case, if the statements

```
[ constants :: c2 ]
[ ; c2= 'qcd' ]
```

are included in zone 2 then `'qcd'` becomes the global default for constant `c2`. That default can be overridden in any submodel block, as follows.

```
submodel[ subm1 ]
[ include :: sector1, sector3 ]
[ c2= 'qcd1' ]
end[ subm1 ]
```

27.5 Additional remarks

Defining submodels requires introducing sectors too, but not conversely. Note also that sectors and submodels should use distinct identifiers.

Let us dub a sector as *active* when it is part of the submodel specified in the control-file, and as *alien* otherwise; if no submodel is defined then any sector is active. The active sectors are those that the program will focus on, of course; they are fully checked for syntax and content whilst the alien sectors are only partly checked. To properly debug a model-file it is necessary to run the program for every (sub)model defined in that file.

Although it is easy to create rather cryptic model-files with this extended language, that is likely not a very good idea; for instance, model-files may have to be edited (long) after coming into existence, possibly by someone who did not create the original file. The proactive measures to avoid creating such files include adding an adequate amount of commentary, avoiding cryptic identifiers, defining the sectors carefully, and grouping together those propagator (and vertex) statements that are not part of any sector — or even creating all the necessary sectors to preclude the existence of such statements.

NB: Should you decide to update your 'old' model-files, please note that they may still be used as input not only for some previous version of the program but also for QGRAF-R (until it is updated, at least).

28. Duplicate vertices

A model has *duplicate vertices* if its description includes at least two vertex statements with the exact same fields (with the same multiplicities, of course). Each vertex may in fact have several copies, and although the corresponding statements may be identical that needs not be the case — the images of the v-functions and even the field ordering may differ.

Duplicate vertices had been tolerated for some time but typically QGRAF generated more diagrams than necessary (with appropriate ‘symmetry factors’, though, so that the corresponding calculation could have been carried out). This has been addressed with the release of `qgraf-3.3` — additional symmetries are taken into account, so that there are usually fewer diagrams now.

29. The field (or propagator) commutation number

The usual propagator statement includes a (‘plus’ or a ‘minus’) *commutation sign*, for instance

```
[ phi, phi, + ]
[ psi, psi, - ]
```

which may be dubbed the *field-sign* or the *propagator-sign*. Nonetheless, to make this type of description look a bit more ‘algebraic’, it is now possible (from **qgraf-3.6** onwards) to replace that sign by a signed *commutation number*, for example

```
[ phi, phi, +1 ]
[ psi, psi, -1 ]
```

The plus sign should not be omitted. Also, there must be some consistency — in any given model-file, either commutation signs or commutation numbers should be used (but not both).

30. An example of a modern model-file

```

%      zone 1
%
% the submodels,
% and then the sectors

[ submodels :: subm1, subm2 ]

[ p_sectors :: sk1 ]
[ v_sectors :: sk2, sk3 ]

%      zone 2
%
% the constants,
% the definitions of the global constants
% and the defaults of the non-global constants

[ constant :: c1 ]
[ constant :: c2 ]
[ c1 = 'f' ]

%      zone 3
%
% the functions
% and their global defaults

[ integer f_functions :: ff1 ]
[ p_functions :: pf1, pf2 ]
[ ;; ff1= (-1,1), (0) ]
[ ;; pf1= 1/2 ]
[ integer v_function :: vf1 ]

%      zone 4
%
% the submodel's blocks

submodel[subm1]
  [ include :: sk2 ]
  [ c2 = 'scalar submodel' ]
end[subm1]

```

(continues on the next page)

```

submodel[subm2]
    [ exclude :: ]
    [ c2 = 'full model' ]
end[subm2]

%      zone 5      (required)
%
% the propagators and the propagator-type sectors

[ phi, phi, +1 ; pf1= 0, pf2= 'A' ]

sector[sk1]
    [ ; pf2= 'B' ]
    [ psi, psibar, -1 ]
    [ lambda, lambda, -1 ; ff1= (1) ]
end[sk1]

%      zone 6      (required)
%
% the vertices and the vertex-type sectors

[ phi, phi, phi ; vf1= 0 ]

sector[sk2]
    [ phi, phi, phi, phi ; vf1= 1 ]
end[sk2]

sector[sk3]
    [ ; vf1= -1 ]
    [ psibar, psi, phi ]
    [ psibar, psi, lambda, lambda ]
end[sk3]

```

Part IV — The style-file

31. Intrinsic representation of diagrams

This section presents a number of technicalities that will be needed for understanding and controlling the output of the program. To begin with, let us present some terminology. Apart from Feynman diagrams — viewed as pure combinatorial objects — sometimes we will consider also the underlying graphs, as if the Feynman diagrams had been deprived of their fields and associated labellings. When referring to those graphs we will use the terms *node* and *edge*. The external nodes are the nodes of degree one associated with the external fields, while the remaining nodes are called internal nodes. Similarly, an external edge is an edge that is incident to an external node, and any other edge is called internal. When referring to a Feynman diagram we will use the terms *vertex* and *propagator*; these terms are the analogue of internal node and internal edge, respectively, but they are meant to include the information about the attached fields.

The representations of a Feynman diagram that can be put to use to create an output-file are based on several types of indices that label the basic ‘components’ (or ‘parts’) of the diagram. The program assigns one or more labels (integer numbers, let us stress) to each of the following ‘parts’: external and internal fields, propagators and vertices (for instance). Such sets of labels will be dubbed by terms such as ‘vertex-index’ and ‘propagator-index’.

A critical issue should be clarified at once: the objects that are labelled are (strictly) not the ones defined in the model-file. In that file, one may find a certain number of fields, propagators and vertices that are considered to be distinct if either the strings that define them are distinct (in the case of fields) or if they are not defined in the same statement (in the case of propagators and vertices, whether or not different sets of fields are involved). The labels we have just mentioned are given to *embedded* objects, ie they are attached to some ‘part’ of the diagram.

Let us see in more detail how one can define the embedding of fields, propagators, and vertices. Most of those cases are obvious: propagators are attached to internal edges, vertices to internal nodes, and external fields to external nodes. What about the internal fields? Recall that in the perturbative expansion the interaction vertices supply the fields that are to be contracted in pairs, creating propagators. Hence internal fields should be attached to objects surrounding the internal nodes. One could, for example, insert two auxiliary nodes into every internal edge and then attach the internal fields to those auxiliary nodes (as illustrated in Fig. 8a). There is no need for auxiliary nodes in the external edges — the external nodes will do.

The above mentioned method of field embedding is not unique. One could attach them to edges instead of nodes, that is, one could insert an auxiliary node into every internal edge (therefore sub-dividing every such edge into two) and then attach the internal fields to the resulting ‘half-edges’. External fields would be attached to external edges, or possibly half-edges (if the external nodes were dropped). Hereafter we will take for granted that the field embedding is properly defined, without relying too much on the actual method.

31.1 The basic indices (or labellings)

If a diagram has V internal nodes and P internal edges then QGRAF numbers its vertices from 1 to V , and its propagators from 1 to P — see the examples given in Fig. 7, diagrams

(a) and (b). Those labellings define the *vertex-index* and the *propagator-index*, respectively.

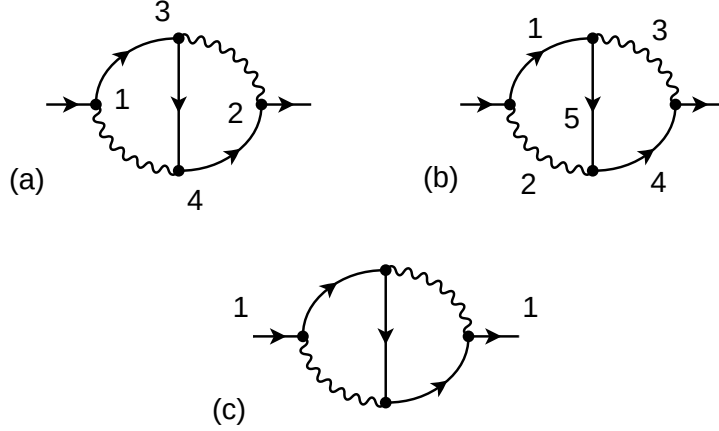


Fig. 7. Some indices for a simple diagram: (a) the vertex-indices, (b) the propagator-indices, and (c) the leg-indices.

The embedded external fields — or legs — should be labelled too. There are two different leg-indices, one for incoming fields (the *in-index*) and another for outgoing fields (the *out-index*). If a diagram has r incoming legs and s outgoing legs then the former get the labels $1, 2, \dots, r$ and the latter the labels $1, 2, \dots, s$. The label that is chosen for each leg follows automatically from the order in which the external fields were declared in the control-file. For example, if the external fields are declared by means of the following statements

```
in = positron[p1], electron[p2] ;
out = higgs[q1], muon_minus[q2], muon_plus[q3] ;
```

then the leg `positron` is assigned the in-index 1, and the leg `electron` the in-index 2; also, the leg `higgs` is assigned the out-index 1, the leg `muon_minus` the out-index 2, and the leg `muon_plus` the out-index 3.

Six basic labellings (indices) are defined. The field-index and the ray-index — the last two types of indices to be presented — are defined over the set of embedded fields.

The field-index uses the propagator and the leg indices. If a propagator has propagator-index k then its two fields receive the field-indices $2k-1$ and $2k$ (see Fig. 8b); if the particle differs from the anti-particle then the former gets the field-index $2k-1$ and the latter the field-index $2k$. An external field is assigned a negative index related to the leg-index of the corresponding leg. Specifically, the field-index of an incoming field whose in-index is equal to j is defined as $-2j+1$, while for an outgoing field with out-index j the field-index is defined as $-2j$. This means that the incoming fields get odd indices ($-1, -3, \dots$) and the outgoing fields get even indices ($-2, -4, -6, \dots$). Although this labelling may seem unnatural at first, it allows one to distinguish external fields from internal ones — as well as incoming from outgoing fields — without reference to any other quantity.

The last type of labelling will be called *ray-index* because, visually, one might associate an edge emerging from a vertex with a ‘ray’. For every vertex, the ray-index labels the

surrounding vertex fields with the numbers $1, 2, \dots, D$ (where D is the degree of the interaction), an example of which is given in Fig. 8c. In contrast to other labellings, here labels differ only within each vertex; globally, repeated labels exist in any diagram with at least two vertices. The ray-index is not totally arbitrary: the index of an embedded field always coincides with the position (or one of the positions) of the field name in the definition of the respective vertex given in the model-file. For instance, if there is a vertex of the form

[positron, electron, photon]

then, for vertices of this type, the field **positron** will always be assigned the ray-index 1, the field **electron** the ray-index 2, and the field **photon** the ray-index 3, which means that in this case the labelling is unique — see Fig. 8(c). If the vertex includes repeated fields, some arbitrariness remains.

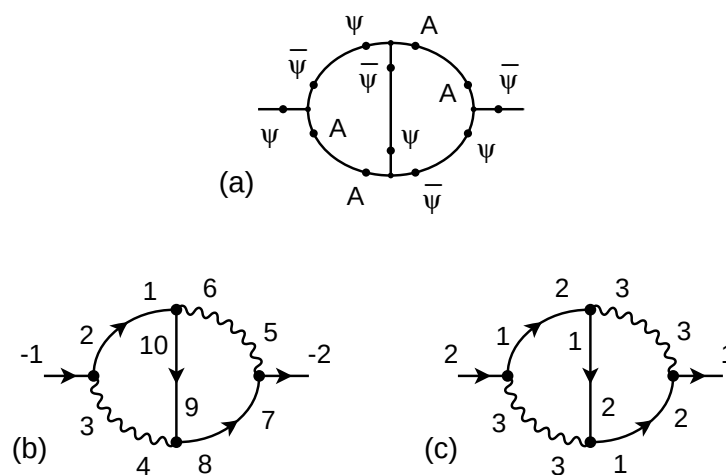


Fig. 8. Revisiting the diagram presented in Fig. 7: (a) a way of embedding the internal fields using auxiliary nodes, (b) the field-indices, and (c) the ray-indices.

We now have at our disposal two different notations for the embedded fields. The first of these is a single-index notation: Φ_i denotes the field with *field-index* i . There is also a two-index notation: $\Phi_{i,j}$ denotes the field that belongs to vertex i (ie the vertex whose *vertex-index* is equal to i) and whose *ray-index* equals j . In the case of Fig. 8 one has $\Phi_{-1} = \Phi_{1,2} = \psi$ and $\Phi_6 = \Phi_{3,3} = A$.

Some of the indices presented in this section (the vertex-index, for instance) are not completely determined in terms of a specific, complete rule. Hence one cannot predict the vertex-index of the vertices of most diagrams, relying on this manual only. Should the undocumented rules used by the program change in the future, no problem should arise provided no special property is assumed other than the generic ones herein presented.

31.2 The field-sign and the field-type

The *field-sign* is defined as ‘+’ for commuting fields and ‘-’ for anti-commuting fields. This quantity may also be referred to as the *propagator-sign*.

Necessarily for embedded fields, we will also define the *field-type*. It takes only three values, namely 1 (for incoming fields), 2 (for outgoing fields), and 3 (for internal fields).¹¹

¹¹ There is now a more ‘natural’ version of the field-type, which uses short ‘names’ (see Section 38.3).

31.3 The propagator momentum (and orientation)

The program also provides a set of symbolic expressions for representing the momentum flow throughout the diagram, a feature that may be rather handy. Obviously, to specify that flow, a reference orientation will have to be assigned to each propagator. What will be called the *propagator momentum* is the momentum flow corresponding to that orientation.

Suppose that the field embedding is properly defined, and that the field-index of every embedded field is known. The actual rule for defining the propagator orientation is as follows: the program picks the direction in which, travelling along the propagator, the embedded field with field-index $2i$ is reached before the one with field-index $2i-1$. This orientation coincides with the particle flow whenever the particle and the anti-particle differ.

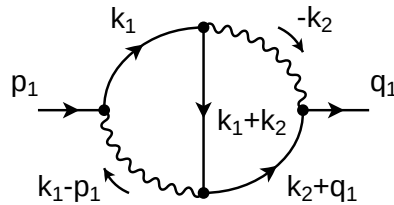


Fig. 9. A momentum-flow for the diagram presented in Fig. 7.

Fig. 9 shows a set of momenta for the diagram presented earlier. It may be verified at once, by taking a look at Fig. 8(b), that the orientation of the propagators (indicated with arrows) is in agreement with the above mentioned rule that depends on the field-index.

32. Generating an output-file — basic concepts

In the initial versions — before the release of `qgraf-2.0` — there was just a short list of predefined output formats that could be used to generate the output-file. Having a fixed number of predetermined formats is a self-limiting approach. Since every program that may be used to process the output of a diagram generator is most likely bound to have its own notation, the number of formats, and thus the number of corresponding sub-programs, may have to increase over time. A possible alternative is that users write their own conversion programs, one for each output processing program they may wish to use. Another problem with the fixed format approach is that it lessens the potential ability of the program to incorporate into the output the parameters (represented by functions and constants) that are part of the definition of the input model. If one wishes to have (i) a model-file where the parameters are chosen by the user and (ii) an output containing a selection of those parameters, using a notation also chosen by the user, then the fixed format approach is inadequate.

In later versions there is a lot more flexibility, since the output of the program can be shaped with the help of a simple programming language. It is not just a matter of choosing the type of delimiters, spacing and other punctuation — one can also choose what kind of information should appear in the output (within some limits, of course). In practical terms, it goes like this: to get a new ‘format’ a user has to provide a file (the style-file) containing a fairly short *program*, and that is all there is to it. Users can have a small collection of such files, and use different formats on different occasions or even on the same run.

Typically, it seems much easier to write and (specially) modify one such file than a specialized conversion program. Although it may not be possible to write a style-file that formats the output exactly as desired, it should be possible to write a style-file such that the respective output not only includes all the necessary information but can also be processed directly by one’s favourite computer algebra system.

32.1 Style-keywords, and the sections of the style-file

The output-file consists of (up to) three parts, to be called *sections*. A typical output-file starts with a *prologue* that may include, for instance

- the name and version of the program that generated that file, and the approximate time the file was created (that is, opened);
- information that can be used to identify the input scattering process and the type of diagrams described in the file (for example, the statements included in the control-file);
- extra information to be used by another program that will read the output-file (for instance, special marks or strings to signal the beginning of the diagram list).

After that, in the diagram section, some information about each diagram (a symbolic amplitude, for example) is included sequentially, following a specific pattern. Finally there is a third section, dubbed the *epilogue*, whose basic purpose is to mark the end of the list and/or the end of the file. For example, it is important to know that the program did not exit prematurely; there is also a second opportunity to pass on additional information to the program(s) that will process the output-file. The blueprint for constructing each one of those three output sections is the corresponding section of the style-file.

The syntax of the style-file is completely different from any syntax discussed so far.

Apart from annotations — see below — the style-file contains ‘text’ (ie printable ASCII characters that, with a few exceptions, are taken literally) and ‘keywords’ (*style-keywords*).

An intrinsic style-keyword is a reserved string that starts with ‘<’ and ends with ‘>’, for example <end>. The following keywords, which delimit the specification of the output sections, must appear in any style-file.

```
<prologue>
<diagram>
<epilogue>
<exit>
```

Those four keywords should always appear in that specific order, and only once, each one on its own line (containing no further characters) and starting at the leftmost position; they are the only style-keywords that must appear in any style-file. Apart from syntactic requirements every other style-keyword is optional — although a style-file containing no keyword other than those four is not terribly useful. Note that a string such as <exit> is not a style-keyword if it is part of some annotation.

The prologue specification starts on the line following the <prologue> keyword and ends on the line that precedes the <diagram> keyword. Analogously, the diagram section specification is enclosed by the lines containing the keywords <diagram> and <epilogue>, and the epilogue specification should appear in between the lines that include the keywords <epilogue> and <exit>. What comes either before the line with the keyword <prologue> or after the line containing the keyword <exit>, if anything, must consist of empty lines or annotations, but the latter have to conform to some rules (Section 6.2); no annotations are allowed elsewhere (ie in any of the three proper sections of the style-file).

The program reads the style-file(s) before it starts generating diagrams, and stores internally what it read. It can then build the character strings to be written to the output-file (or passed on to the program that calls the API). Each such string, initially empty, has a fixed end (the ‘left-end’, so to say, which is a kind of starting point) and a non-fixed end (the ‘right-end’) that changes as a result of executing the instructions described in the respective section of the style-file. An output string is built by following a sequence of two basic operations: appending some characters to the non-fixed end of the current string and, less frequently, deleting one or more characters from that same end. As the possibility of deleting characters exists, no such string is output before the section that defines it has been fully executed.

Nearly every bit of a style-file section that is parsed as ‘text’ (namely everything that is neither a keyword nor part of some annotation) is simply copied verbatim to the output-file (or output-block, in API-mode). The exceptions are described in Section 32.3.

The style-keywords can be divided into two main classes. One class contains what one may call control-keywords; these serve to delimit the output sections, to define the programming constructs etc, but do not add useful information to the output string(s) by themselves. The other class includes the keywords that instruct the program to append some characters to the output string(s); these will be called data-keywords.

32.2 Style-loops and local keywords

Some data-keywords are needed to output the entries of one- and two-dimensional arrays (ie sets of indexed strings). For example, for each diagram, the names of the fields that

correspond to the computed field-indices constitute such an array (whose entry k is the name of the field to which the field-index k is assigned). Therefore, some *programming loops*, to be called *style-loops*, are needed to output that kind of information in a suitable way. These constructs are defined by style-keywords too, a list of which follows.

```

<end>
<in_loop>
<momentum_loop>
<out_loop>
<propagator_loop>
<ray_loop>
<statement_loop>
<statement_sub_loop>
<vertex_loop>

```

Each of those keywords, except the first, declares a style-loop whose scope ends with the appearance of the matching `<end>` keyword. Not all style-loops may appear in every section of a style-file. Some style-loops should be nested within other style-loops. Additional details will be given later.

Data keywords may be split into local and non-local keywords. Local keywords are those that must be used in a programming loop, while non-local keywords have no such restriction. Non-local keywords do not have to be constants, for example the keyword `<diagram_index>` produces different strings at different stages.

32.3 Exceptional printable characters

A few printable characters have to appear duplicated in the style-file if they are to be accepted as ‘text’. Those exceptional cases are as follows (the ASCII characters that have to be ‘encoded’ are on the *lhs*, and their respective ‘encodings’ on the *rhs*).

<code><</code>	→	<code><<</code>
<code>></code>	→	<code>>></code>
<code>[</code>	→	<code>[[</code>
<code>]</code>	→	<code>]]</code>

With this convention it is always possible to distinguish ‘text’ from keywords. The first two exceptions are related to the use of the characters ‘<’ and ‘>’ in the delimitation of intrinsic keywords. The square brackets play a similar role in the case of non-intrinsic keywords, which are derived from the identifiers of user-defined constants and functions.

In fact there is an additional special case, namely that of a trailing space character, which will be discussed in the next section. That case is a bit different, however, since it is about always reading and writing a certain character rather than having to encode it.

33. The prologue and epilogue sections

For the sake of illustration consider a fictitious style-file whose prologue section consists of the lines shown in the box below. To keep the discussion as simple as possible, let us assume that there is a *newline* character whose purpose is to mark the end of a line and implicitly the beginning of the subsequent line, should there be one.

```
#
#_generated_by_<program>
#_<back>
#_on_<full_time>
#
<statement_loop><statement_sub_loop>#_<sub_statement>
<end><end>#
#
_tsum_:=_0
```

This description is processed as follows. The program starts with an empty output string, as already mentioned, and then line 1 tells it to add a '#' as well as a *newline*, which is always added when the end of a line is reached. Line 2 instructs the program to add another 15 characters (shown), then to perform the action determined by the keyword <program>, and finally to add another newline. That keyword outputs a string that describes the name and the version of the program; what that string is can be deduced from the output (shown below) generated from the above example and from a control-file whose contents may be inferred from that same output.

```
#
#_generated_by_qgraf-4.0.6
#_
#_on_2025/04/17_01:15:14.387_+0100
#
#_output_='_d_list'_;
#_style_='_sum.sty'_;
#_model_='_qed'_;
#_in_='_electron[p1]_';
#_out_='_electron[q1],_photon[q2]_';
#_loops_=_3;
#_loop_momentum_=_k;
#_options_='_floop,_onepi_';
#
_tsum_:=_0
```

The keyword <back> deletes the rightmost character from the current output string,

which may produce an error if the string is empty at that point. That keyword may appear in any of the sections of the style-file but there should be no interference between different sections — nor, in the case of the diagram section, interference between different diagrams.

Trailing spaces are ignored, no matter what type of input file is being read — there is usually some difficulty not only for a user to see them but also for Fortran to read them. Still, it is possible to output trailing spaces, as exemplified by line 3 of the previous style-file: after adding one hash character and two spaces, the program will delete the rightmost space character and then add a newline, leaving a trailing space. The keyword `<back>` can also be used to concatenate consecutive lines from the style-file. For example, the two lines

```
This_is_a
<back>_single_line!
```

will be concatenated (omitting the keyword), since `<back>` will delete the newline that separates them (if the character ‘<’ is in the leftmost position). Thus, in the style-file, a long line can be split into two or more lines without introducing additional lines in the output-file. As there is a maximum allowed line length for any input file (Section 6.3), that kind of splitting may be necessary.

Next, line 4 instructs the program to add another 5 characters and then, owing to the presence of the keyword `<full_time>`, write the current date and time (the ones provided by the operating system, that is) as detailed in Section 33.1.

Lines 6–7 from our example are more interesting. The keyword `<statement_loop>` tells the program to perform a loop¹² (ie a programming loop); whatever is in between that keyword and the matching keyword `<end>` is to be executed once for (nearly) every statement found in the control-file, in the same sequence. There is one exception, however: in the case of multiple-style configurations (Section 12), the only **style** and **output** statements that may appear in an output-file are those associated with that file (which are unique). Empty lines and annotations (in the control-file) are always ignored.

There is a second style-loop nested within the first and delimited by the keywords `<statement_sub_loop>` and the corresponding `<end>`. The inner loop is executed once for each line occupied by the statement that the outer loop defines implicitly (depending on the iteration step), following the line sequence of the control-file. The keyword `<sub_statement>` tells the program to add the character string that corresponds to the statement line defined implicitly by the outer and inner loops — one input line in each iteration of the inner loop. That character string includes neither a newline, nor a statement-continuation mark, nor trailing spaces.

In our example, each statement from the control-file takes up a single line. Nevertheless, if that file were rewritten so that at least one statement occupied more than one line, the output-file might include something like this:

```
#_output
#_=_
#_''d_list';
```

That may or may not be what one wishes to obtain; it should not be difficult to program a different kind of output (such that not every line begins with an annotation-mark, say).

To present another example, the next lines of code will reformat statements that occupy

¹² This type of construct, of which the present kind is not the only example, will be dubbed a *style-loop*.

more than one input line into a single output line, with ‘#’ as annotation-mark.

```
<statement_loop>#<statement_sub_loop>_<sub_statement><end>
<end>#
```

To sum up, the statements from the control-file can be copied to the output-file(s) by having two nested style-loops and including the keyword `<sub_statement>` in the inner loop; the formatting is rather free. It is no longer possible to reproduce the control-file statements using a single style-loop; still, anything that could be done with the single loop construct should also be possible with the two loop construct.

33.1 The style-keywords `<full_time>` and `<raw_time>`

The output produced by `<full_time>` should consist of the *date*, *time*, and *time zone*, in ASCII, as provided by the operating system; obviously, the correctness of this type of output depends on having an appropriately configured environment. That output should look like this:

```
2023/08/14_01:15:14.387_+0100
```

The date format uses the sequence year/month/day, and the time is provided to the millisecond; the slashes and the colon signs are added by the program. Alternatively, the keyword `<raw_time>` produces a similar output without added characters, for example

```
20240915_151401.783_+0100
```

The obvious usefulness of these keywords consists in making it possible to register the creation time in the output-file itself — the time saved in the file-system can be changed inadvertently if one is not careful enough (when copying the file, for instance). In fact, two distinct times can be registered, namely the time when the output-file was opened and the time it was completed, typically to a very good approximation.

A quite good estimate of the *real time* (or ‘*wall clock time*’) taken by the execution of the program can be obtained by inserting one of those keywords in both the prologue and epilogue sections of the style-file (or one of the style-files) and then computing the elapsed time from the respective outputs. That elapsed time might not fully include the period of time taken by the operating system in copying the output-file to the computer’s disk; that file is usually created in the computer’s (random access) memory and seldom saved at once, specially if it is a large file.

33.2 The epilogue section

The epilogue section may include every keyword allowed in the prologue section, and no other. Therefore, all the information about the input statements and the version of the program may be included in the epilogue section instead, or even in both sections. Clearly, the output produced by the keywords `<diagram_counter>` and `<diagram_index>` depends on which of the sections they are used.

Recent additions to the scope of the prologue and epilogue sections are discussed in Section 38.2. The style-keywords allowed in these sections are listed in Section 39.

34. The diagram section — the non-local style-keywords

The diagram section is obviously the most relevant. There are many keywords (most of which data-keywords) that are allowed in that section, as well as six types of style-loops. The non-local keywords, ie those data-keywords that do not have to appear in a style-loop, are presented in this section, and the other (ie local and control) keywords in the next.

34.1 The basic non-local keywords

The ‘basic’ non-local keywords allowed in the diagram section, which follow, produce the output indicated. Other non-local keywords that require a lengthier explanation are listed and described separately.

- `<diagram_counter>` — a positive integer equal to the number of diagrams generated up to that point (ie ‘1’ for the first diagram, ‘2’ for the second diagram, etc)
- `<diagram_index>` — a positive integer equal to the sum of the outputs produced by the keywords `<diagram_counter>` and `<index_offset>`
- `<index_offset>` — the non-negative integer declared in the `index_offset` statement, else zero
- `<legs>` — the number of external fields (of the current diagram)
- `<legs_in>` — the number of incoming fields
- `<legs_out>` — the number of outgoing fields
- `<loops>` — the loop number (of the current diagram)
- `<loops1>` — the first (and possibly only) value declared in the `loops` statement
- `<loops2>` — the second value declared in the `loops` statement, else the only value
- `<loop_momentum>` — the identifier declared in the `loop_momentum` statement
- `<minus>` — equal to `<sign>` (see below) if the diagram sign is ‘-’, else an empty string
- `<propagators>` — the number of propagators (internal edges) of the current diagram
- `<sign>` — the diagram sign (either a plus or a minus sign) that follows from the anti-commutation rules
- `<symmetry_factor>` — the diagram symmetry factor (either ‘1’, if there are no symmetries, else a fraction of the form $1/m$ with m a positive integer greater than 1)
- `<vertices>` — the number of (internal) vertices of the current diagram
- `<zero_momentum>` — the identifier declared in the `zero_momentum` statement, else ‘0’

34.2 Some style-keywords related to the symmetry factor

The first additional set of non-local keywords consists of the following keywords.

- `<local_symmetry_number>`
- `<nonlocal_symmetry_number>`
- `<symmetry_number>`

The keyword `<symmetry_number>` produces a positive integer equal to the reciprocal of the symmetry factor. That integer can be written as a product of two others, however. Let $\text{Aut}(D)$ be the automorphism (ie ‘symmetry’, permutation) group of diagram D , and $\text{Aut}_p(D)$ the subgroup of $\text{Aut}(D)$ consisting of those automorphisms that leave every vertex fixed — that is, those symmetries that only involve permuting parallel propagators or ‘flipping’ self-loops. These are the kind of symmetries that may be called *local*.

The keyword `<local_symmetry_number>` produces the order of $\text{Aut}_p(D)$ and the other keyword, `<nonlocal_symmetry_number>`, the order of the quotient group $\text{Aut}(D)/\text{Aut}_p(D)$. Those two integers provide some basic information about the origin of the symmetry factor: if $|\text{Aut}_p(D)| > 1$ then (non-trivial) local symmetries exist, and if $|\text{Aut}(D)/\text{Aut}_p(D)| > 1$ then there are (non-trivial) *non-local* symmetries (these two cases may co-exist, obviously).

34.3 Non-standard, non-local style-keywords

Four additional non-local keywords, namely

- `<new_elinks>`
- `<new_loops>`
- `<new_partition>`
- `<new_topology>`

depend not only on the ‘current’ diagram but also on the preceding diagram, if any; they always produce a ‘1’ for the first diagram of every run. Note also the implicit hierarchy (below), which is a consequence of the diagram generation algorithm.

The keyword `<new_loops>` may be useful when the `loops` statement requests diagrams for more than one loop number. A ‘1’ is produced in the following cases (otherwise, a ‘0’ is produced):

- if the current diagram is the first generated diagram of the current run;
- else, when the loop number of the current diagram differs from that of the preceding diagram.

The keyword `<new_partition>` produces a ‘1’ in the following cases (else, a ‘0’):

- when `<new_loops>` produces (or would produce, if it were present) a ‘1’;
- else, when the vertex degree partition of the current diagram differs from that of the preceding diagram.

The output produced by `<new_topology>` specifies whether or not the topology of the current diagram differs from that of the preceding diagram, where ‘*topology*’ means unlabelled topology, without fields and associated labels. Thus, a ‘1’ is produced in the following cases (else a ‘0’ is produced):

- when `<new_partition>` produces a ‘1’;
- else, when the unlabelled topology of the current diagram differs from that of the previous diagram.

Finally, `<new_elinks>` refers to labelled topologies, ie to topologies with the external fields in place. This keyword produces a ‘1’ in the following cases (else, a ‘0’):

- when `<new_topology>` produces a ‘1’;
- else, when the configuration (or ‘layout’) of the external fields in the current diagram differs from the corresponding configuration in the preceding diagram.

As an example of a potential application, one may point out that `<new_topology>` and `<new_elinks>` can be employed to speed-up the topology identification. In addition, the information that the same topology is shared by multiple (specific) diagrams can often be exploited to improve the efficiency of the computations that involve large numbers of diagrams (sometimes even when no topology has been precisely determined).

On the other hand, if the model has a single (self-conjugate) field then `<new_elinks>` becomes irrelevant, as it always produces a ‘1’. Also, the usefulness of these keywords might be reduced to some extent, or even completely, if (say) the output diagrams were arbitrarily divided among several files or, worse still, if those diagrams were sorted in a different way.

35. The diagram section — the local style-keywords

This section presents the types of style-loop allowed in the diagram section and the respective local keywords. Each of the keywords

```
<in_loop>
<out_loop>
<propagator_loop>
<vertex_loop>
```

announces one such type of loop, and the keyword `<end>` closes them. Those style-loops are executed as many times as there are (respectively) incoming particles, outgoing particles, propagators and vertices (in the current diagram). A further type of style-loop, defined by `<momentum_loop>`, is discussed separately (Section 35.4).

During the execution of a style-loop the program examines the main class of objects accessible in that loop (propagators or vertices, for example) and possibly other adjacent objects, and then prints some information about them if requested to do so. Note that the definition of a local keyword depends on the type of style-loop where it appears. A fifth style-loop type, defined by the keyword `<ray_loop>`, should always appear nested within the vertex-loop, like this:

```
<vertex_loop> ... <ray_loop> ... <end> ... <end>
```

The ray-loop is needed to tell the program to examine every line incident with the vertex that is iteratively defined by the vertex-loop. Thus, the number of times that the ray-loop is executed is equal to the degree of the vertex being examined. If a keyword can be used in the vertex-loop then it can also be used in the ray-loop, although the information it represents will remain constant while the ray-loop is executed.

Those five style-loops form the basic tool to access the local information that defines a diagram, where *local* means that it refers either to the object (or ‘component’) being examined, or to some adjacent object. For instance, if that object is a vertex v_i then some information regarding the vertices adjacent to v_i is also available at that time, as is the information on any propagators and external lines incident with v_i . Nevertheless, at that same time, no information about other (more remote) objects is available, with the obvious exception of the information provided by the non-local keywords.

What remains to be explained here is which local keywords may be used inside which style-loops (this information is summarized in the table given in Section 40), as well as what the keywords stand for.

35.1 The propagator-loop

The propagator-loop is executed as follows: when the keyword `<propagator_loop>` is found the program assigns the value 1 to the loop counter, which coincides with the propagator-index, and then it outputs the information requested about the respective propagator (and possibly some other information, or fixed characters) until the matching keyword `<end>` is reached. At that point it increments the loop counter to 2, and the rest is easily guessed; the loop terminates as soon as loop counter exceeds the number of propagators.

To infer what kind of information is available during the execution of the propagator-

loop let us take a look at Fig. 10, which has been obtained by grouping together the figures included in Section 31 but retaining only part of the original diagram, namely propagator 5 and its neighbourhood.

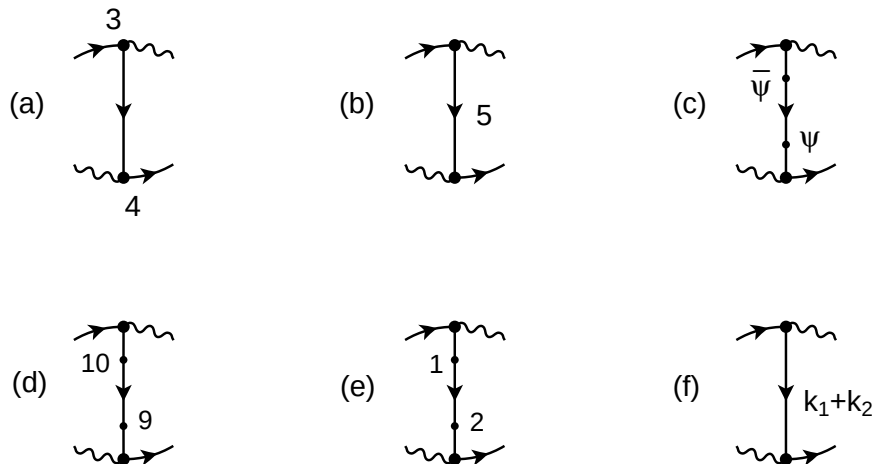


Fig. 10. Some local information available in the propagator-loop: (a) vertex-indices, (b) propagator-index, (c) propagator fields, (d) field-indices, (e) ray-indices, and (f) propagator momentum.

A list of the keywords allowed in the propagator-loop is given below. Each entry includes a keyword, the respective meaning, and an output string generated by that same keyword. That string (which follows the arrow symbol) is the one that would be obtained in the description of the propagator shown in Fig. 10; in this case one should identify ψ with **electron** and $\bar{\psi}$ with **positron**.

Recall that if a propagator is assigned a propagator-index i then the field-indices of the respective fields (to be noted Φ_{2i-1} and Φ_{2i}) will be equal to $2i-1$ and to $2i$. Generically, let $\hat{v}_j$ denote the vertex to which Φ_j belongs (hence j is a field-index, but not a vertex-index).

- `<dual-field>` — the name of Φ_{2i} ($\rightarrow$ **positron**)
- `<dual-field_index>` — the unsigned integer $2i$ ($\rightarrow$ **10**)
- `<dual-momentum>` — the opposite of `<momentum>` ($\rightarrow$ **-k1-k2**)
- `<dual-ray_index>` — the ray-index of Φ_{2i} (in its vertex) ($\rightarrow$ **1**)
- `<dual-vertex_degree>` — the degree of $\hat{v}_{2i}$ ($\rightarrow$ **3**)
- `<dual-vertex_index>` — the vertex-index of $\hat{v}_{2i}$ ($\rightarrow$ **3**)
- `<field>` — the name of Φ_{2i-1} ($\rightarrow$ **electron**)
- `<field_index>` — the unsigned integer $2i-1$ ($\rightarrow$ **9**)
- `<field_type>` — the field-type of Φ_{2i-1} ($\rightarrow$ **3**)
- `<field_sign>` — the sign of propagator i ($\rightarrow$ **-**)
- `<momentum>` — the momentum of propagator i ($\rightarrow$ **k1+k2**)
- `<propagator_index>` — the unsigned loop counter i ($\rightarrow$ **5**)
- `<ray_index>` — the ray-index of Φ_{2i-1} (in its vertex) ($\rightarrow$ **2**)

- `<vertex_degree>` — the degree of $\hat{v}_{2i-1}$ ($\rightarrow$ 3)
- `<vertex_index>` — the vertex-index of $\hat{v}_{2i-1}$ ($\rightarrow$ 4)

35.2 The leg loops

There are two types of leg loops, the `<in_loop>` and the `<out_loop>`, which are executed once for each incoming (respectively, outgoing) field. The legs of the diagram are, rather obviously, the main objects that are accessed in those style-loops.

Let r and s be the number of incoming and of outgoing fields. Those fields appear in the control-file in a certain sequence, and the n^{th} incoming (or outgoing) field will be noted Φ_n^{in} (respectively, Φ_n^{out}).

To illustrate the keywords available for the leg loops, we shall make use of a diagram that has been used before (Fig. 7), whose legs are specified by the following statements.

```
in = electron[pp1] ;
out = electron[qq1] ;
```

As (in our example) $r = s = 1$, each keyword produces a single string (below). Consider first the `<in_loop>`, where the loop counter i runs from 1 to r . Let v_k be the vertex that leg i is incident to, and j the ray-index (with respect to that vertex) of Φ_i^{in} . The field Φ_i^{in} can then be identified with $\Phi_{k,j}$.

- `<dual-field>` — the conjugate of `<field>` ($\rightarrow$ positron)
- `<dual-momentum>` — the opposite of `<momentum>` ($\rightarrow$ -pp1)
- `<field>` — the name of Φ_i^{in} ($\rightarrow$ electron)
- `<field_index>` — the field-index of Φ_i^{in} ($\rightarrow$ -1)
- `<field_sign>` — the sign of Φ_i^{in} ($\rightarrow$ -)
- `<field_type>` — the field-type of Φ_i^{in} ($\rightarrow$ 1)
- `<in_index>` — the unsigned loop counter i ($\rightarrow$ 1)
- `<leg_index>` — the same as `<in_index>` (in this type of loop) ($\rightarrow$ 1)
- `<momentum>` — the momentum flowing into the diagram through leg i ($\rightarrow$ pp1)
- `<ray_index>` — the ray-index of $\Phi_{k,j}$ (ie the unsigned integer j) ($\rightarrow$ 2)
- `<vertex_degree>` — the degree of vertex v_k ($\rightarrow$ 3)
- `<vertex_index>` — the index of vertex v_k (ie the unsigned integer k) ($\rightarrow$ 1)

Some of the above definitions have to be changed for the `<out_loop>`. The loop counter i goes from 1 to s while the leg-index goes from $r+1$ to $r+s$. The keyword `<field>` now produces outgoing fields, and the momentum direction points outwards. Thus $\Phi_{k,j}$ is now the *conjugate* of Φ_i^{out} .

- `<dual-field>` — the conjugate of `<field>` ($\rightarrow$ positron)
- `<dual-momentum>` — the opposite of `<momentum>` ($\rightarrow$ -qq1)
- `<field>` — the name of Φ_i^{out} ($\rightarrow$ electron)
- `<field_index>` — the field-index of Φ_i^{out} ($\rightarrow$ -2)
- `<field_sign>` — the sign of Φ_i^{out} ($\rightarrow$ -)

- `<field_type>` — the field-type of Φ_i^{out} ($\rightarrow 2$)
- `<leg_index>` — the unsigned integer equal to $r+i$ ($\rightarrow 2$)
- `<momentum>` — the momentum leaving the diagram through leg i ($\rightarrow$ qq1)
- `<out_index>` — the unsigned loop counter i ($\rightarrow 1$)
- `<ray_index>` — the ray-index of $\Phi_{k,j}$ (ie the unsigned integer j) ($\rightarrow 1$)
- `<vertex_degree>` — the degree of vertex v_k ($\rightarrow 3$)
- `<vertex_index>` — the index of vertex v_k (ie the unsigned integer k) ($\rightarrow 2$)

35.3 The vertex-loop and the ray-loop

The vertex-loop tells the program to examine each vertex of the diagram, and the ray-loop to examine each edge (or possibly half-edge) incident with each vertex. The counters associated with these loops coincide with the vertex-index and the ray-index; in the following they will be denoted by i and j , respectively.

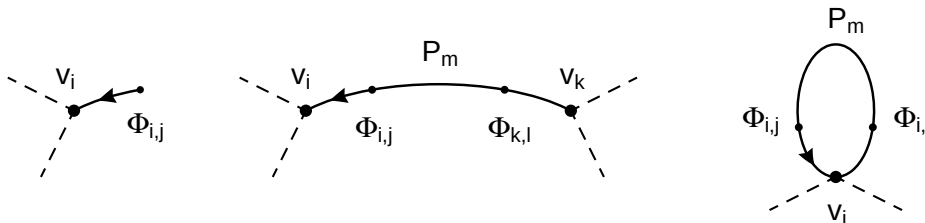


Fig. 11. Basic notation used in describing the vertex-loop and the ray-loop.

A vertex v_i of degree d_i comprises the fields $\Phi_{i,j}$, for $j=1, \dots, d_i$. Fig. 11 illustrates the three possible cases: $\Phi_{i,j}$ can be an external field (left), an internal field that is part of a propagator joining two distinct vertices (centre), or an internal field that is part of a propagator built from two fields of the same vertex (right). If, for a given value of j , $\Phi_{i,j}$ is an internal field then there is a propagator P_m connecting v_i to another vertex v_k (or else connecting v_i to itself, in which case we just set $k = i$); that propagator also contains another field — belonging to v_k — which is the conjugate of $\Phi_{i,j}$ and that will be denoted by $\Phi_{k,l}$. Note that v_k and $\Phi_{k,l}$ are both undefined whenever $\Phi_{i,j}$ is an external field.

In the ray-loop, the keyword `<momentum>` refers to the momentum flowing along the edge to which $\Phi_{i,j}$ is attached, in the direction that is shown graphically in Fig. 11 by means of arrows. This graphical rule gives the momentum flowing into vertex v_i coming from that edge, except that if $\Phi_{i,j}$ is an internal field and $i = k$ then this wording must be made more precise.

Fig. 12 includes part of the same diagram presented earlier — now it shows vertex 1 and its neighbourhood — and will also be used for illustrating the meaning of the keywords presented below. This time the output of the program given with each entry can be divided in two cases. For the keywords that do not require the use of the ray-loop the output string contains a single label, namely the one that corresponds to vertex 1. For the other keywords the output string is composed of three sub-strings (since the degree of vertex 1 is equal to 3), one for each ray-index; that means that the execution of the ray-loop is simulated, but the execution of the vertex-loop is not.

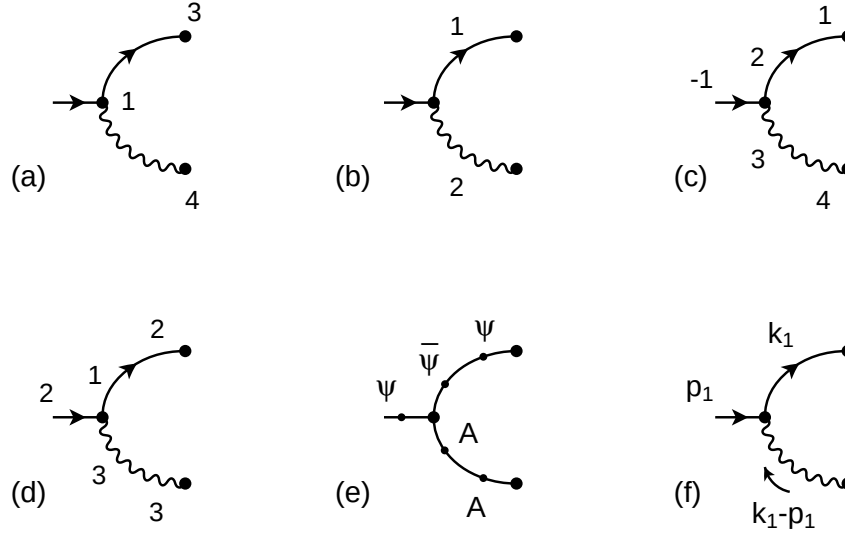


Fig. 12. Basic information available in the vertex-loop.

- `<dual-field>` — the name of the conjugate of field $\Phi_{i,j}$
($\rightarrow$ electron positron photon)
- `<dual-field_index>` — the field-index of $\Phi_{k,l}$ (Fig. 11) if that field exists, otherwise zero ($\rightarrow$ 1 0 4)
- `<dual-momentum>` — the opposite of `<momentum>` ($\rightarrow$ k1 -p1 -k1+p1)
- `<dual-ray_index>` — the ray-index of $\Phi_{k,l}$ (ie the unsigned integer l) if that field exists, else zero ($\rightarrow$ 2 0 3)
- `<dual-vertex_degree>` — the degree of vertex v_k , if that vertex exists, else zero ($\rightarrow$ 3 0 3)
- `<dual-vertex_index>` — the index of vertex v_k (ie the unsigned integer k), if it exists, otherwise zero ($\rightarrow$ 3 0 4)
- `<field>` — the name of $\Phi_{i,j}$ ($\rightarrow$ positron electron photon)
- `<field_index>` — the field-index of $\Phi_{i,j}$ ($\rightarrow$ 2 -1 3)
- `<field_type>` — the field-type of $\Phi_{i,j}$ ($\rightarrow$ 3 1 3)
- `<field_sign>` — the sign of $\Phi_{i,j}$ ($\rightarrow$ - - +)
- `<momentum>` — the momentum flowing into vertex v_i coming from the edge with ray-index j ($\rightarrow$ -k1 p1 k1-p1)
- `<propagator_index>` — the propagator-index of P_m (Fig. 11), ie the unsigned integer m if it exists, else the field-index of the external field $\Phi_{i,j}$ ($\rightarrow$ 1 -1 2)
- `<ray_index>` — the unsigned integer j ($\rightarrow$ 1 2 3)
- `<vertex_degree>` — the degree of vertex v_i ($\rightarrow$ 3)
- `<vertex_index>` — the unsigned integer i ($\rightarrow$ 1)

Five keywords — `<dual-field_index>`, `<dual-ray_index>`, `<dual-vertex_index>`, `<dual-vertex_degree>` and `<propagator_index>` — are defined in a peculiar way when the ray-index j defines an external line. This is done for two main reasons: it either ensures compatibility with earlier pre-defined formats or it provides more information than a straightforward definition would. The core issue is the same: either one does not accept any of those keywords as valid, or else one must define them *ad hoc* for the cases where a natural definition fails to exist (ie for vertices incident with external lines). An additional keyword, `<dual-field>`, is defined in a way that is not consistent with the meaning of the prefix ‘dual-’ as used in other keywords; in addition, if it were consistent in that regard then it would suffer from the same problem that the other five keywords do. This problem will hopefully reach an acceptable status in a future release, as other features become available.

35.4 The momentum-loop

The keyword `<momentum_loop>` defines a style-loop over the terms of the expression that would be produced by `<momentum>`, enabling a more complex output than the one provided by the latter keyword. The momentum-loop can be nested within either the ray-loop or the propagator-loop. The number of iterations is equal to the number of (nonzero, reduced) terms in the expression for the respective momentum if that momentum is nonzero, and is otherwise equal to 1.

Generically, the style-keywords `<momentum>` and `<dual-momentum>` can be replaced by the following constructs, respectively.

```
<momentum_loop><momentum_term><end>
<momentum_loop><dual-momentum_term><end>
```

For example, the keyword `<momentum_term>` produces one term in each loop iteration, as if extracted sequentially from the output of `<momentum>`. Therefore, those terms can now be ‘decorated’. With the help of the `zero_momentum` statement, the following kind of code

```
<momentum_loop><momentum_term>_{mu<field_sub>}<end>
```

can produce consistent output, even for null momenta, avoiding the construction of dubious expressions such as ‘0_{mu2}’. For example, when the style-keywords `<momentum>` and `<field_sub>` produce the expressions ‘k1-p1’ and ‘7’, the last style-loop should produce

```
k1_{mu7}-p1_{mu7}
```

Also, when a null internal momentum is found, that same code should produce (say)

```
k0_{mu4}
```

assuming that `k0` is the (declared) zero-momentum. Therefore, the momentum-loop can be used to generate expressions in which every momentum term has a ‘space-time’ index. Expressions for tensors with similar indices, such as $g^{\mu_i \mu_j}$, can be produced by simpler means, of course. Although the lengths of the generated amplitudes will tend to increase, the overall impact should be negligible unless the number of diagrams is quite large and the output-blocks have to be stored in a file — but a new approach is emerging (Section 42).

For now, the only loop-dependent style-keywords allowed in the `<momentum_loop>` are `<field_index>`, `<field_sub>`, `<momentum_term>`, their duals, and (if also nested within the propagator-loop) `<propagator_index>`. Other style-keywords might be enabled later, should they be useful.

36. A first look at output styles

The set of available data keywords is certainly not a minimal one. Also, from a strict point of view, not all of the existing style-loops are needed. The advantage of this abundance is that it is possible to choose the ‘best’ features for each task; for example, generating some types of redundant information that are available ‘instantly’ could require some extra programming if a minimal feature set were used.

The main issue concerning the choice of output style is the processing of the expressions constructed by the program. The output styles defined by the ancillary files ‘`sum.sty`’ and ‘`array.sty`’ serve to illustrate two different approaches. Although those output styles are just ‘toy’ styles, a brief analysis can still be profitable if it provides a basis for building more useful style-files.

The first approach consists in combining all the symbolic ‘amplitudes’ into a single expression, that is, to add them up. Further processing may be problematic if the number of diagrams is large. In the second approach an array is defined, with each entry storing a symbolic expression for a single diagram; then the processing program may read the whole array and manipulate all the expressions as appropriate. The examples presented below refer to the diagram shown in Fig. 13.

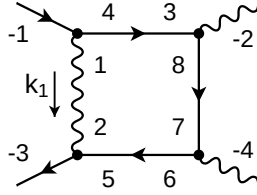


Fig. 13. A diagram from QED, with the corresponding field-indices
(the embedded fields are implicit).

The contents of the diagram section of style-file `array.sty` are as follows (or possibly something very similar to that).

```
<back>
a(<diagram_index>):= (<sign><symmetry_factor>)*
<in_loop> pol(<field>(<field_index>,<momentum>))*
<end><out_loop> pol(<field>(<field_index>,<momentum>))*
<end><propagator_loop> prop(<field>(<field_index>,<momentum>))*
<end><vertex_loop> vtx(<ray_loop><field>(<field_index>,
<back><momentum>),<end><back>)*
<end><back><back>;
```

Each entry of the array `a`, indexed by the diagram index, will be a symbolic expression that includes the diagram sign, the symmetry factor, and the output generated by five distinct style-loops; for simplicity, the momentum-loop is absent. The kind of output that can be obtained when the style-file ‘`array.sty`’ is used looks like this:

```

a(1):= (+1)*
pol(e(-1,p1))*
pol(p(-3,p2))*
pol(A(-2,q1))*
pol(A(-4,q2))*
prop(A(1,-k1))*
prop(e(3,-k1+p1))*
prop(e(5,-k1-p2))*
prop(e(7,-k1+p1-q1))*
vrtx(p(4,k1-p1),e(-1,p1),A(1,-k1))*
vrtx(p(-3,p2),e(5,-k1-p2),A(2,k1))*
vrtx(p(8,k1-p1+q1),e(3,-k1+p1),A(-2,-q1))*
vrtx(p(6,k1+p2),e(7,-k1+p1-q1),A(-4,-q2));

```

The numbers that appear in the above expression as arguments of the fields are the field-indices. The propagators have a single ‘argument’; in the absence of explicit propagator mixing, the second argument would not contain new information (the second field would be the conjugate of the first, the second index would be equal to the successor of the first, and the momentum would be the opposite of the first).

Concerning the style-file `sum.sty` let us remark that its prologue and epilogue sections include (respectively) the following lines.

```

_tsum:=_0

```

```

;

```

The sum of the partial ‘amplitudes’ is represented by the variable `tsum`, defined in the prologue as zero, but using a ‘non-terminated’ expression; that will allow `tsum` to be well defined if no diagrams are generated. Then, in the diagram section, each amplitude is defined by a *signed*, non-terminated expression so that a sum is produced out of the partial amplitudes. Finally, the semicolon in the epilogue terminates the expression that defines `tsum`.

37. Non-intrinsic style-keywords

A non-intrinsic style-keyword is a keyword that derives from some (user-defined) function or constant. A few of the preceding sections discuss intrinsic style-keywords and the kind of output that they produce. As it is possible to define functions for the fields, propagators and vertices (Section 26.3), however, one might wish the diagram description to include some of their images; ditto for the ‘value’ of some of the constants defined in the model-file. The subject of the present section is on how to define non-intrinsic keywords and use them in a style-file.

37.1 Keywords derived from functions

Consider once more QED, described as follows.

```
% propagators
[ electron, positron, -1 ; prop= 'S', m= 'me' ]
[ photon, photon, +1 ; prop = 'P', m= 'm0' ]

% electromagnetic vertex
[ positron, electron, photon ; gpow = '1' ]
```

Suppose that `prop` represents the (name of the) ‘propagator function’, and `m` the ‘propagator mass’. Coupling the above description with the following code,

```
<propagator_loop>_ [prop] (<momentum>, [m]) *
<end>
```

which is assumed to be part of the diagram section of the style-file, leads, in the case of the diagram shown in Fig. 9, to the following output:

```
_S(k1,me)*
_P(k1-p1,m0)*
_P(-k2,m0)*
_S(k2+q1,me)*
_S(k1+k2,me)*
```

This simple example shows how to obtain an output where each propagator type has its own name, and each propagator has an appropriate mass parameter. As seen earlier, intrinsic style-keywords are ‘select’ identifiers¹³ enclosed in angle brackets (the ASCII characters ‘<’ and ‘>’, in fact). Now we see that a function may give rise to a non-intrinsic (local) keyword by enclosing its name in square brackets. This precludes any interference between the two types of style-keywords, so that any valid identifier can be used to name a function (though it is probably a good idea not to abuse that possibility).

It might be worth stressing that the domain of a function is a set of objects from the model-file, not a set of diagram embedded objects. For example, when the program outputs

¹³ Possibly with an added ‘dual-’ prefix, that is.

an image of `prop`, it looks only at the field names that appear in the propagator, not at the propagator-index; hence `[prop]` produces the same result for all the diagram propagators sharing exactly the same field names.

When appropriate definitions exist, non-intrinsic keywords may appear in more than one kind of style-loop; in addition, they may be prefixed with ‘`dual-`’ unless they derive from p-functions. The following table shows which kind of keywords are allowed in which kind of loops — an entry with a full circle denotes permission, and an empty entry denotes interdiction. The strings `ff` and `vf` denote generic f-functions and v-functions, respectively; they map each field ϕ and each vertex ν from the model into some strings $\text{ff}(\phi)$ and $\text{vf}(\nu)$. The string `pf` denotes a generic p-function.

	i/o-loop	p-loop	r-loop	v-loop
<code>[ff]</code>	•	•	•	
<code>[dual-ff]</code>	•	•	•	
<code>[pf]</code>	•	•	•	
<code>[dual-pf]</code>				
<code>[vf]</code>	•	•	•	•
<code>[dual-vf]</code>		•		

The definitions of the generic keywords listed in the above table depend on the style-loop they appear in, but it is possible to present unified definitions using a few intrinsic loop-dependent keywords. Here they are:

- `[ff]` — outputs the string $\text{ff}(\phi)$ where ϕ is the field defined by `<field>`
- `[dual-ff]` — outputs $\text{ff}(\phi)$ where ϕ is the field defined by `<dual-field>`
- `[pf]` — outputs the string $\text{pf}(\phi)$ where ϕ is either of the fields defined by `<field>` and `<dual-field>`
- `[vf]` — outputs $\text{vf}(\nu)$ where ν is the vertex with vertex-index `<vertex_index>`
- `[dual-vf]` — outputs $\text{vf}(\nu)$ where ν is the vertex defined by `<dual-vertex_index>`

Note that using `[dual-vf]` in the ray-loop would lead to problems. The issue is linked to the fact that some intrinsic keywords have abnormal definitions in this type of loop, and will hopefully be resolved in a later release.

37.2 Keywords derived from constants

The constants declared in the model-file give rise to style-keywords too, although non-local ones, and these may appear in any section of the style-file. For example, to the constant `model` corresponds the non-intrinsic keyword `[model]`.

38. Other additions and extensions

38.1 The style-keyword `<field_sub>` and its dual

The keyword `<field_sub>` (where *sub* stands for *subscript*) is a variant of the keyword `<field_sub>`, only it produces always a positive integer. It may help simplify the generation of space-time indices for fields and momenta,¹⁴ for instance. The set of values output by this keyword is seldom a collection of consecutive integers, but that should hardly be a problem. The keywords `<field_sub>` and `<field_index>` are allowed in the same style-loops.

Let r and s denote the number of incoming and of outgoing fields of the input process, and $t = \max(r, s+1)$. For an external field, `<field_sub>` produces the absolute value (ie the opposite) of the output of `<field_index>`. For an internal field, `<field_sub>` produces the result of the ‘sum’ $2t + \text{<field_index>}$.

The output produced by the dual keyword `<dual-field_sub>` is the result of the ‘sum’ $2t + \text{<dual-field_index>}$ unless `<dual-field_index>` produces a zero, in which case the former output is a zero too (this may occur only in the ray-loop). Thus, `<dual-field_sub>` may produce a zero in the ray-loop, but not in the propagator-loop.

The expected usage should probably be as follows: `<field_sub>` in the ray-loop and possibly in the leg loops (in-loop and out-loop); `<field_sub>` and `<dual-field_sub>` in the propagator-loop, to ‘link’ the indices produced in the other loops (thereby producing summable indices). Some of those indices will be covariant, others contravariant.

Although it would have been possible to have a slightly more compact range for the output of `<field_sub>`, there appear to be two minor advantages also, at least for debugging purposes. One, in every instance, the parity of the outputs of `<field_sub>` and of `<field_index>` is the same, and that also holds for their duals. Two, one can tell from the complete set of outputs of `<field_sub>` which fields are internal or external (and even incoming or outgoing), given that the subscripts of the internal fields (if any) form a special set of natural numbers — namely a set of even size consisting of those consecutive subscripts with the largest values, separated from the rest by a ‘gap’ (of size one or two). If there are no internal fields then either there is no such gap or there is a single number above the top gap. Anyhow, the identification of the external fields can then be based on the individual outputs of `<field_sub>` — this is similar to the corresponding identification based on the output(s) of `<field_index>` since it is just a matter of multiplying the output values by -1 .

38.2 More general prologue and epilogue sections

Restricted versions of the style-loops `<in_loop>` and `<out_loop>` are now allowed in the prologue and epilogue sections. Naturally, in these sections the style-keywords allowed in those loops cannot depend on any diagram, hence some keywords have to be excluded (Section 39).

The basic definition of the input scattering process can then be restated in a format recognized by the program (computer algebra system, say) that is going to process the output-file. Some additional information about the external fields, present in the model-file, can also

¹⁴ Many users must have been doing something of that kind already, of course (as well as the kind of indexing enabled by the momentum-loop construct, described next).

be produced. Specifically, the following information can be added or reformatted:

- the input process — ie the external fields and their momenta;
- other properties of the external fields — index, sign, type, values of f-functions and of p-functions, for instance.

There are a few more possibilities not related to fields:

- the values appearing in the `loops` and `index_offset` statements;
- the identifier(s) declared in the `loop_momentum` and `zero_momentum` statements.

Recall also that the constants of the input model can be used in any section of the style-file.

38.3 The style-keyword `<field_stype>`

The output produced by `<field_stype>` is equivalent to that of the style-keyword `<field_type>`, only it consists of a short ‘symbolic’ string (instead of an integer), to make it more ‘user-readable’. Specifically, it outputs ‘`incg`’, ‘`outg`’ and ‘`intl`’ (for an incoming, outgoing, and internal field, respectively) instead of ‘1’, ‘2’ and ‘3’.

39. A list of the intrinsic keywords for the prologue and the epilogue sections

The control-keywords:

```
<back>
<end>
<in_loop>
<out_loop>
<statement_loop>
<statement_sub_loop>
```

The non-local data-keywords:

```
<diagram_counter>
<diagram_index>
<full_time>
<index_offset>
<legs>
<legs_in>
<legs_out>
<loop_momentum>
<loops1>
<loops2>
<program>
<raw_time>
<zero_momentum>
```

The local data-keywords:

```
<dual-field>
<dual-momentum>
<field>
<field_index>
<field_sign>
<field_stype>
<field_sub>
<field_type>
<in_index>
<leg_index>
<momentum>
<out_index>
<sub_statement>
```

40. A list of the intrinsic keywords for the diagram section

The control-keywords:

```
<back>
<end>
<in_loop>
<momentum_loop>
<out_loop>
<propagator_loop>
<ray_loop>
<vertex_loop>
```

The non-local data-keywords:

```
<diagram_counter>
<diagram_index>
<index_offset>
<legs>
<legs_in>
<legs_out>
<local_symmetry_number>
<loop_momentum>
<loops>
<loops1>
<loops2>
<minus>
<new_elinks>
<new_loops>
<new_partition>
<new_topology>
<nonlocal_symmetry_number>
<propagators>
<sign>
<symmetry_factor>
<symmetry_number>
<vertices>
<zero_momentum>
```

The local data-keywords, namely those that have to be used in some style-loop, are listed on the next page.

The local data-keywords that do not have a dual:

<field_sign>	<leg_index>
<field_stype>	<out_index>
<field_type>	<propagator_index>
<in_index>	

The local data-keywords that have a dual, at least sometimes:

<field>	<dual-field>
<field_index>	<dual-field_index>
<field_sub>	<dual-field_sub>
<momentum>	<dual-momentum>
<momentum_term>	<dual-momentum_term>
<ray_index>	<dual-ray_index>
<vertex_degree>	<dual-vertex_degree>
<vertex_index>	<dual-vertex_index>

Next, a table showing which local keywords can be used inside which style-loops. A given keyword may be used in a certain loop type if and only if the respective table entry is marked with a full circle. The loop types have been abbreviated — `i_loop` stands for in-loop, `r_loop` for ray-loop, and so on. Additional style-keywords might be allowed in the momentum-loop (in some future version of the program) should that turn out to be useful.

	i_loop	o_loop	p_loop	v_loop	r_loop	m_loop
<dual-field>	•	•	•		•	
<dual-field_index>			•		•	•
<dual-field_sub>			•		•	•
<dual-momentum>	•	•	•		•	
<dual-momentum_term>						•
<dual-ray_index>			•		•	
<dual-vertex_degree>			•		•	
<dual-vertex_index>			•		•	
<field>	•	•	•		•	
<field_index>	•	•	•		•	•
<field_sign>	•	•	•		•	
<field_stype>	•	•	•		•	
<field_sub>	•	•	•		•	•
<field_type>	•	•	•		•	
<in_index>	•					
<leg_index>		•				
<momentum>	•	•	•		•	
<momentum_term>						•
<out_index>		•				
<propagator_index>			•		•	•
<ray_index>	•	•	•		•	
<vertex_degree>	•	•	•	•	•	
<vertex_index>	•	•	•	•	•	

Part V — Other topics

41. The display-output

In what follows, it will be assumed that the program is run in auto-mode and that the config option `noinfo` is not used, otherwise the display-output would typically be suppressed. The exception would be the displaying of an error message, which is not suppressed in auto-mode even when option `noinfo` is used.

The ‘normal’ display-output is the one sent to the stream `stdout`.

41.1 The normal display-output

Suppose first that the program runs uneventfully — ie that it stops after concluding the required tasks, with no output being dispatched to `stderr`. In that case, the first part of the display-output shows the version of the program and the statements found in the control-file. After that, but only in `debug` and `verbose` display-mode, some elementary information about the input model-file and the input (sub)model is displayed, for example

```
submodels:  [3]
sectors:   4   ...   (P)  1+ [2]   (V)  3 [5]
propagators: 8   ...   (N+) 2   (C+) 3   (N-) 1   (C-) 2
vertices: 17   ...   ( 3^10  4^7 )
```

Fig. 14. The second part of the display-output in `verbose` mode.

The first line in Fig. 14 shows the submodel count, the second line the sector counts; each of these lines will be suppressed whenever the respective total count is null. The numbers enclosed in square brackets refer to the complete model-file, which in this example defines 3 submodels and 7 sectors (ie 2 plus 5). The input submodel includes 4 sectors, namely 1 propagator-type sector and 3 vertex-type sectors (plus some ‘loose’ propagator statement(s), as explained next); a plus sign may follow each of the these two sector counts, indicating that there exist (respectively) propagator or vertex statements that are not in any sector block (such statements are part of the description of any submodel defined in the respective model-file). If no submodels are defined, but at least one sector is, then a simplified output will be displayed, for instance

```
sectors:  7   ...   (P) 2+   (V) 5
```

since in this case every sector is part of the input model.

The first two lines in Fig. 14 include some information about the complete model-file but the other two refer to the input (sub)model only. The third line shows the number of propagators, as well as the nonzero subtotals per ‘*propagator-type*’, according to the following rules:

- the letter ‘N’ refers to ‘*neutral*’ propagators (ie those for which the particle is equal to the anti-particle);
- the letter ‘C’ refers to ‘*charged*’ propagators (ie those for which the particle and the anti-particle differ);
- the signs ‘+’ and ‘-’ denote whether the propagators are defined in terms of commuting or anti-commuting fields.

For example, the propagators of Dirac fermions (and of some ghost fields) contribute to the count of (ie the number that follows) ‘C-’ while the propagators of Majorana fermions contribute to the count of ‘N-’. In the above example (Fig. 14), the (sub)model includes a total of 8 propagators (5 ‘bosonic’ and 3 ‘fermionic’). Two of those bosonic propagators are neutral and the other three are charged; only one fermionic propagator is neutral.

The program differentiates between ‘true-propagators’ and ‘non-propagators’. If a statement includes the keyword **external** then it counts as a ‘non-propagator’, otherwise as a ‘true-propagator’. An extra line may then be displayed, for instance

```
non-propagators: 1    ...    (N+) 1
propagators: 7    ...    (N+) 1    (C+) 3    (N-) 1    (C-) 2
```

Those numbers are mutually exclusive — in this case the definition of the input (sub)model includes 8 propagator statements too.

The fourth displayed line (Fig. 14) shows the vertex count of the input (sub)model and the respective subtotals per vertex-degree; there is a total of 17 vertices, of which 10 are cubic and 7 are quartic.

The last part of the display-output (see the next ‘box’) shows the numbers of diagrams generated for each compatible vertex-degree partition and, in the ‘**subtotals**’ column, for each loop number indicated in the **loops** statement. Lastly, the number of generated diagrams is shown. The ‘**subtotals**’ column may be omitted when the additional numbers that could be displayed are completely redundant — ie when the same exact numbers already appear in the ‘**#diagrams**’ column.

#loops	v-degrees		#diagrams		subtotals
1	-	4 ¹	...	1	
	3 ²	-	...	2	
					3
2	-	4 ²	...	2	
	3 ²	4 ¹	...	12	
	3 ⁴	-	...	6	
					20
total		...	23 connected diagrams		

Fig. 15. The last part of the display-output.

In this example, it is implicit that the respective control-file contains a statement of the form

```
loops = 1 to 2 ;
```

and that the input model includes only cubic and quartic vertices. There are two possible

partitions for 1-loop diagrams: 3^2 (diagrams with exactly two vertices, both cubic) and 4^1 (diagrams with exactly one vertex, but a quartic one). The corresponding numbers of diagrams are 2 and 1, and the 1-loop subtotal equals 3 (as displayed). The same kind of information is then shown for 2-loop diagrams.

Part of the display-output may look a bit garbled when there is a ‘large’ number of distinct vertex degrees. To improve the readability, one may then have to increase the width of the terminal or redirect the standard output to a file. Further, when a `count_to` statement (Section 18) is used, the ‘normal’ output (illustrated in Fig. 15) is modified to emphasize the premature termination of the program (should it occur), for instance

#loops	v-degrees		#diagrams		subtotals
1					
	-	4^1	...	1	
	3^2	-	...	2	
					3
2					
	-	4^2	...	2	
	3^2	4^1	...	5 **	
					7 **
min_total		...	10 connected diagrams		

41.2 Diagnostic messages

The program stops prematurely if an error is detected in auto-mode, usually after displaying the corresponding error message, for example

```
=====
input error: output-file #1 already exists
=====
```

‘Alerts’ (or ‘warnings’) are displayed differently, for instance

```
####  check 'config' statement
```

and in this case the execution of the program continues. Both kinds of messages are directed to `stderr` (the standard error stream). In fact, the output of the program sent to `stderr` consists exactly of the diagnostic messages (if any), including their accompanying ‘graphical’ symbols, as shown in the previous examples.

In `info` display-mode, if the diagram generation finishes but some diagnostic messages

(which would have been shown in either `debug` or `verbose` display-mode) were suppressed, a warning sign is then added to the line that shows the total number of diagrams, for instance

```
total    ...    12343 connected diagrams  (w!)
```

The following error types have been defined for use with error messages.

```
input error
internal error
I/O error
```

41.3 When the diagram generation is obviated

Sometimes the program is able to detect that there are no diagrams, either for certain vertex-degree partitions or even for all such partitions, without having tried to generate a single diagram (work in progress). Those situations can be created by some combinations of options and/or statements. In such cases (those detected, that is) the display-output looks a bit different. If the program detects a complete absence of diagrams (for instance, if the diagrams are required to be trees with self-loops), it will simply display

```
total    ...    0 connected diagrams  **
```

without listing any partial result; otherwise, for each ‘flagged’ partition (that is, a partition for which the program has determined in advance that there can be no diagrams), the respective line shows only that partition, for example

```
3^2  4^1
```

In either case, the appearance of a simplified display-output implies that the usual diagram generation was not attempted.

One other possibility of that kind consists in detecting whether the input process breaks some particle number conservation rule or, equivalently, some (perturbatively) conserved charge — where *conserved* means conserved in the input model, obviously. Still, for now, there is a separate program (QGRAF-R) that can be used for that purpose.

41.4 Output redirection

With `qgraf-4`, the display-output is split between `stdout` and `stderr` in the usual way; the `stderr` stream will get the diagnostic messages, and `stdout` any other display-output. By default, if the program is launched on a terminal then both of those streams will typically be directed to that terminal. It is now possible to separate those two kinds of output if that type of operation is supported by the shell (it usually is). The following

examples refer to **bash**.

```
qgraf a_control-file 1> normal.log 2> error.log
```

```
qgraf a_control-file 1> /dev/null 2> diagnostic.log
```

With each of these commands, **stdout** is redirected to the file that follows ‘1>’, and **stderr** to the file that follows ‘2>’. These redirections may be useful to either secure or discard a copy of (one of the types of) the display-output; also, when running more than one instance of the program simultaneously, it is possible to save (each of the types of) the respective display-outputs in a different file (or in different files).

42. An application programming interface (API)

We will say that QGRAF is running in *API-mode* if it is being controlled by another program (the *calling program*, which should be written in Fortran or in C) that calls the application programming interface (API) described in this section.

One of the main advantages of using that API, as compared to using the auto-mode, consists in the ability to distribute the ‘amplitudes’ produced by the program among any number of computer processes — not only in real-time but also in a versatile way, since the calling program is restricted only by the sequence in which those amplitudes are supplied. As a second advantage, there is the ability to select the diagrams in real-time, without having first to store them in a file to be read by another program.

An *output-block* is a (possibly long) character string resulting from *one* full execution of the instructions included in a single section of one of the input style-files; in particular, output-blocks may contain newlines. The *index* of an output-block is the index (ie 1,2,...) of the **style** statement that declares the corresponding style-file. In a typical run, one output-block will be produced for each prologue and epilogue section; further, one output-block will be built for each diagram section *and* for each generated diagram. If a run produces n_d diagrams, and if n_s style-files are declared, then that run may produce up to $n_s(n_d+2)$ non-empty output-blocks. The output-blocks are accessible in real-time to the calling program, which decides what to do with them.

42.1 QGRAF as a sub-program

The API consists mainly in a module comprising some definitions and subroutines, including three interface subroutines (not to be used jointly), each of which providing a way to access in real-time some of the quantities and structures computed by the program — namely by having the respective (set of) ‘values’ written on the output-blocks. This interface may not only be employed to run QGRAF step-by-step (eg diagram by diagram), treating it as a sub-program, but serves also as a kind of buffer that isolates most of the its code from the inner code of any calling program (both of which may continue to evolve independently).

An object file suitable for API-mode can be obtained from the same downloaded source file, though compiled with a different set of preprocessor related options (Section 3.3); also, there is a header file to be imported by any calling C program. Using the aforementioned interface consists essentially in calling repeatedly one (fixed) interface subroutine, which depends on the calling program (Section 3.3). For simplicity, it will be assumed in what follows that there is a single interface subroutine, fictitiously named **q_interface**. Each Fortran call should be of the form

```
call q_interface(sgnl1,sgnl2,ccs,csl,lint)
```

where **sgnl1**, **sgnl2** and **lint** are integer variables (the last one a ‘long’ integer), **csl** is an integer array (whose size should not be smaller than the number of input style-files), and **ccs** is a character string, long enough to hold the generated output-blocks (one per style-file), that is treated as an array of character strings (possibly of different lengths). For example, the first call should tell QGRAF to initialize itself, read the control-file and the input files declared therein, storing any relevant information, and then return to the calling program. A call in C looks a bit different, of course; for one of the available mixed-programming interfaces, it should be of the following form.

```
q_interface(&sgnl1,&sgnl2,ccs,csl,&lint)
```

The value of `sgnl1` should be an *input-signal*, to specify the computational task to be performed. The array `cs1` is multi-purpose: on entry, it may specify either the maximum allowed lengths of the various output-blocks, or which output-blocks to construct; on return, it usually contains the lengths of the output-blocks generated during the call, if any were requested, although it may also contain the length of an error message returned in `ccs` if an exception occurs. Sometimes `ccs` and `lint` are used as input arguments too.

The arguments `sgnl2`, (occasionally) `lint`, and (very often) `ccs` and `cs1` are set by QGRAF and returned to the calling program. These arguments should be over-writeable (or possibly pointers to such variables). The value returned in `sgnl2` is a *return-signal*, which provides some feedback on the call's success or lack thereof. Unless an exception occurs, if a call requests some output-block(s) then these blocks and their lengths should be returned in `ccs` and `cs1`, ordered by their indices. The argument `lint` may return the number of generated diagrams, messages, or style-files.

42.2 Basic input-signals

The input-signals tell QGRAF what kind of computation to perform (of which there exist only a few predefined types). The following list should include the most common (in C, the character '%' should be replaced by a '.' to match the notation for structure member).

- `qisgnl%init`

This is the signal that instructs QGRAF to 'start-up' (or possibly re-start) and then read the input files; it should be the input-signal of the first call.

- `qisgnl%count`

Requests the total number of (connected) diagrams that match the control-file statements, if `lint` is set to zero (on input); a potentially long call could then take place, but a 'bounded' call can be requested by setting `lint` as a positive integer (this is similar to what could be achieved in auto-mode with a `count_to` statement).

- `qisgnl%msg_count`

This signal requests the number of messages created up to that point (since the last initialization).

- `qisgnl%prologue`

Requests the output-block(s) for the prologue section of each style-file.

- `qisgnl%diagram`

The signal that requests that a new diagram be generated, if possible (even for the first diagram), and the respective output-block(s) returned.

- `qisgnl%epilogue`

Requests the output-block(s) for the epilogue section of each style-file.

- `qisgnl%stop`

Instructs QGRAF to close any open file (the message-file, typically) and terminate the current job. It can be used at any time after initialization.

Whenever output-blocks are requested, each entry of `cs1` should be an input-signal, though one of a slightly different kind. The next two signals, which can be interpreted as ‘yes’ and ‘no’, are the only values allowed.

- `qisgnl%y`, `qisgnl%n`

The output-block of index n (that is, of the type defined by `sgnl1` and derived from the n^{th} style-file declared in the control-file) is computed if and only if `cs1(n)` equals `qisgnl%y`.

42.3 The return-signals

A *return-signal* serves to report whether or not the call was successful, and possibly what kind of error occurred. There are two return-signals, given next, that can be produced in a non-eventful call (that is, when none of the exceptions described further below occurs).

- `qrsgnl%ok`

This is the signal returned when the call is successful. If some output-block ought to have been produced then the contents of `ccs` and `cs1` should be valid.

- `qrsgnl%end`

This return-signal means that there are no more output-blocks of the type requested — or, if returned on the first call of its type, that there are no such output-blocks at all (the latter situation should not happen except for a call with `qisgnl%diagram`).

There are also three *error-signals*, defined below, to let the calling program know not only that some run-time exception occurred but also the generic kind of that exception. Additional details should be provided in a corresponding error message; its text should be stored in `ccs(1)`, and its length in `cs1(1)`. Only the messages for (detected) run-time errors are transmitted to the calling program (at most one message per ‘run’, obviously); the warning messages should be saved in the message-file, once (and if) it has been opened.

- `qrsgnl%input_error`

This is the signal returned when QGRAF detects an error in some input file or in some call to the interface subroutine.

- `qrsgnl%q_error`

The signal returned when QGRAF detects some inconsistency with its current state, likely due to a problem with its own code.

- `qrsgnl%os_error`

The signal returned when QGRAF experiences any type of I/O problem.¹⁵

These error-signals should occur rarely. Once problems such as wrong file permissions and lack of disk space are excluded, finding a returned `qrsgnl%os_error` should be a very rare event since it will likely mean that a serious software or hardware error occurred.

¹⁵ This means input/output problem, and includes reading from files and writing to files.

42.4 Some examples

For interoperability's sake, and also to avoid some duplication, the Fortran types of the variables shared by (or with) QGRAF are interoperable C-types — even when the calling program is a Fortran one. Specifically, `sgn11`, `sgn12` and `cs1` are of type `c_int_least32_t` while `ccs` is of type `c_char` and `lint` of type `c_int_least64_t`. The corresponding C types are `int_least32_t`, `char` and `int_least64_t`, obviously. It should be noted that it is not that difficult to generate a set of diagrams of cardinality not representable with `int32_t` — currently, less than one day (ie several hours) of CPU time should suffice, at least if only their count is requested. The type chosen for `lint` should preclude the need for future adjustments.

The lines of code given next show how to initialize QGRAF using the Fortran interface. The ancillary files `'pxq.c'` and `'pxq08.c'` include a C version of most of the examples given below. Recall that the name `q_interface` is fictitious; the real names of the interface subroutines can be found in Section 3.3 and in Section 42.7.

```
sgn11= qisgn1%init
ccs(1:121)= "'qgraf.dat' " // c_null_char
cs1(:)= int(4096,c_int_least32_t)
lint= int(len(ccs),c_int_least64_t)
call q_interface(sgn11,sgn12,ccs,cs1,lint)
```

The encoded name of the control-file, terminated by a `c_null_char` character, should fit into the first 120 positions of `ccs`. Space characters at either end (ie not in the sub-string delimited by the single quotes) are automatically deleted; other spaces can be deleted with the help of the config option `noblanks`.

The length of `ccs` (or possibly a *lower* value, to be taken as the effective length) is passed in argument `lint` and should not exceed the value of parameter `CCS_L0` (ie 46 080). If the definition of `lint` in the above example cannot be implemented, owing to the particular way in which `ccs` itself is implemented, an alternative statement should be found.

In that call, the entries of `cs1` specify the maximum allowed lengths (one per style-file) for the output-blocks to be produced in subsequent calls; their sum should not exceed the length declared for `ccs`. In the above example, every maximum is equal to 4 096. More generally, each declared maximum should be comprised between 128 and 16 384 (ie 2^7 and 2^{14}) inclusive, but the *effective* maximum will be equal to the declared value *minus* 8. These reserved positions exist as a very minimal precaution against potential 'overflows', and also to allow for one or two control characters. The non-reserved positions must be able to store any respective output-string at any stage of its construction ('newlines' included), not just the final output-block. Those limits can be redefined in any subsequent re-initialization (only).

Still in the same type of call, if `qrsgn1%ok` is returned in `sgn12` then the following information should be returned too: QGRAF's version (in `ccs`, starting at position 1 and terminated by a null character); the respective string length (in `cs1(1)`). That control character does not contribute to the (formal) length of the character string.

When an exception occurs, the respective error message should be returned in `ccs`, and its length in `cs1(1)`. A re-initialization should still be allowed if the return-signal is equal to `qrsgn1%input_error`. In every other case, any subsequent call is returned at once with

the original error-signal and the run will very likely have to be abandoned.

Every call to `q_interface` should be followed at once by a check on `sgn12`, say

```
if( sgn12 /= qrsgn1%ok )then
  if( sgn12 /= qrsgn1%end )then
    call my_error_checking(sgn11,sgn12,ccs,cs1)
  end if
end if
```

where subroutine `my_error_checking` should decide what to do in case of error.¹⁶ That subroutine might have the following structure.

```
if( sgn12 == qrsgn1%os_error )then
  :
else if( sgn12 == qrsgn1%input_error )then
  :
else if( sgn12 == qrsgn1%q_error )then
  :
end if
```

To do a ‘counting run’ one may proceed as follows.

```
sgn11= qisgn1%count
lint= 0
call q_interface(sgn11,sgn12,ccs,cs1,lint)
```

Setting `lint` to zero makes the call ‘unbounded’ (ie the program will try to generate every possible diagram), but setting `lint` as a positive integer would automatically make the call ‘bounded’, with `lint` as an upper bound for the number of diagrams to be generated. That would be the equivalent of having a `count_to` statement in auto-mode (Section 18). The actual number of generated diagrams is returned in `lint`. This kind of call should be made before requesting any output-block for either the diagram section or the epilogue; if it is made, QGRAF will have to be re-initialized (possibly using the same control-file) before producing any other output-block.

Once re-initialized, QGRAF should once again be able to construct any requested output-block(s). In the case of the diagram section, there should be a call per diagram until the signal `qrsgn1%end` is returned. Meanwhile, any requested set of output-blocks will be returned in `ccs` as if this string had been divided into sub-strings with fixed lengths,

¹⁶ Possibly notify the job owner and then stop...

specified in the latest initialization; the actual lengths of the output-blocks will be returned in `cs1`. The following lines of code show how to request (all of) the output-blocks for the prologue(s).

```
sgnl1= qisgnl%prologue
cs1(:)= qisgnl%y
call q_interface(sgnl1,sgnl2,ccs,cs1,lint)
```

To obtain the output-blocks for the diagram section, one might do the following.

```
sgnl2= qrsgnl%ok
do while( sgnl2 == qrsgnl%ok )
  sgnl1= qisgnl%diagram
  cs1(:)= qisgnl%y
  call q_interface(sgnl1,sgnl2,ccs,cs1,lint)
  if( sgnl2 == qrsgnl%ok )then
    :
  else if( sgnl2 == qrsgnl%end )then
    :
  else
    call my_error_checking(sgnl1,sgnl2,ccs,cs1)
  end if
end do
```

The first case of the above `if` block corresponds to a successful call (in which a new diagram has been generated), and the respective code should describe what to do with each output-block. The second case corresponds to the natural end of a diagram generation job.

The next example shows how to obtain the number of diagnostic messages created up to that point, whether or not saved in the message-file. That number is returned in argument `lint`.

```
sgnl1= qisgnl%msg_count
call q_interface(sgnl1,sgnl2,ccs,cs1,lint)
```

Not all three sections of the style-file(s) have to be made use of, but the usual relative precedence should be observed. For example, the output-block(s) for the epilogue(s) can be requested (once) at any time but subsequently it is not possible to request output-blocks for any of the other sections without re-initializing.

One last example shows how to instruct QGRAF to close any open files(s). If no exception occurs, only the message-file may be open when that call is made.

```
sgnl1= qisgnl%stop
call q_interface(sgnl1,sgnl2,ccs,csl,lint)
```

The number of diagnostic messages created since the previous initialization is returned in `lint`. After returning, QGRAF will wait for the next re-initialization, if any. As always, it is necessary to check whether some error occurred.

42.5 Advanced input-signals

A second set of input-signals allows for a more efficient diagram searching, assuming that it is possible to utilize QGRAF's algorithmic structure — ie the fact that the diagram generation involves four hierarchical levels (loop number, vertex-degree partition, unlabelled topology and labelled topology). These signals should not be needed in most situations; they are intended for demanding, CPU-intensive cases, typically with unusual selection criteria.

Suppose one is interested in finding a set S of 'rare' diagrams satisfying some special criteria not implemented in the program — that is, a set S that cannot be obtained by simply using the available options and statements, and for which only supersets of S much larger than S itself can be obtained in that way. In such cases, as the calling program will have to check a potentially huge number of diagrams only to discard most of them, it would be very convenient to avoid generating as many irrelevant diagrams as possible. For example, if the topology of the latest diagram cannot be used to generate another (or a) relevant diagram, one should then wish not to generate any further diagram with that topology. That is the basic idea behind the following signals, which may be chosen at will, one at a time (see also Section 34.3). Still, the only signal that can start the diagram generation is `qisgnl%diagram`.

- `qisgnl%elinks`

This signal instructs QGRAF to ignore any further diagram with a labelled topology¹⁷ equal to that of the latest diagram, and thus to find the first subsequent diagram for which `<new_elinks>` produces a '1'.

- `qisgnl%topology`

An input-signal similar to the previous one, except that it refers to the unlabelled topology; it requests the first subsequent diagram for which `<new_topology>` produces a '1'.

- `qisgnl%partition`

Requests the first subsequent diagram for which `<new_partition>` produces a '1'.

- `qisgnl%loops`

Requests the first subsequent diagram for which `<new_loops>` produces a '1'.

One additional signal exists, as detailed next.

- `qisgnl%hold`

This input-signal tells QGRAF to construct additional output-blocks for the latest diagram — according to the (new) entries of `csl` — instead of trying to generate a new diagram.

¹⁷ This refers not only to the underlying graph but also to the external field configuration.

This last signal allows the calling program to make more than one call for the same diagram, with different (typically ‘orthogonal’) `cs1` arrays as input. This possibility may sometimes be used to lower the total number of output-blocks to be constructed. For example, if the information required (by the calling program) to apply the selection criteria does not depend on every output-block, it might be more efficient to delay the construction of some output-block(s) until the latest diagram is found to satisfy those criteria — and to skip their construction otherwise.

42.6 Further details

In API-mode, there are some constraints on the control-file: neither the statements

```
count_to
output
```

nor most of the config options related to the display-mode and (strictly) to output-files, ie

```
info
lf
noinfo
nolist
verbose
```

are available. Moreover, the following options should not appear in the `options` statement if any of the advanced input signals is to be used (`qisgnl%hold` excepted).

```
new_elinks
new_loops
new_partition
new_topology
topol    (obsolescent)
```

To render cross-checking and debugging a bit easier, additional information may be returned in argument `lint`, including: the number of style-files, on an successful initialization call (that number is probably known to the calling program since the control-file must be created in advance); the number of generated diagrams, on a call that returns `qrsgnl%end` in `sgnl2`. Further, if a call fails to generate some output-block due to lack of space (ie when the respective maximum allowed length is not sufficient), the index of that block (or of one of the problematic blocks) is returned in `lint`. In addition, `lint` is set to zero whenever its return role is neither defined nor relevant.

```
call q_interface(sgnl1,sgnl2,ccs,cs1,lint)
```

In API-mode, the control-file should include one or more (consecutive) `style` statements but no `output` statement, for example

```
messages = 'm.txt' ;
style = 'f1.sty' ;
style = 'f2.sty' ;
style = 'f3.sty' ;
:
:
```

Thus, ‘*multiple-style*’ configurations are possible in this mode too. The maximum allowed number of **style** statements is exactly the same in auto-mode and API-mode (Section 12).

The output-blocks may or may not contain ‘newlines’, depending on what the style-files specify. Long output-blocks without such characters can be constructed; a section of the style-file may consist of many lines but inserting the keyword **<back>** at the start of each line (except the first) leaves only one newline, ie a trailing one. Moreover, in API-mode any trailing newlines (plural) are erased automatically, and a trailing **null** character is added for compatibility with C. Also, the configuration option **no_ntnls** can be used to delete any non-trailing newlines (Section 9.6).

Unlike newlines, the added **null** control characters do not contribute to the lengths returned in **cs1**. At least for now, a ‘*newline*’ is (and counts as) a single character, namely LF (line feed), even if the operating system uses another character or a byte sequence. Thus, some minor adjustments may be required before saving the output-blocks in a file.

The style-keywords

```
<diagram_counter>
<diagram_index>
```

are available also in API-mode but they may become irrelevant if the calling program rejects some of the generated diagrams.

Although the names of the arguments of the generic interface call used in this manual are (in principle) internal to QGRAF, they may appear in some of the messages returned.

42.7 The ancillary files

The latest pack includes three examples of interfacing (in directory ‘**api_mode**’), with source files, one written in Fortran and two in C. The names of the source files and of the corresponding interface subroutines are as follows.

```
pf08.f08    ↔    qinterf2008f
pxq08.c     ↔    qinterf2008c
pxq.c       ↔    qinterf2018c
```

Hopefully, some insight will be gained by perusing their source code and utilizing (one of) them as template(s). It is very important to pick the appropriate subroutine for the calling program, of course; further details are given in Section 3.3. C programs should include the header file ‘**qgraf.h**’ by means of a preprocessor directive, for instance

```
#include "qgraf.h"
```

As already mentioned, use is now made of a few simple types of preprocessor directives and of the GNU preprocessor. With the help of the **make** command, executed in the directory that contains the ‘**Makefile**’, a binary (incorporating QGRAF) can be created for each of the above examples, for instance

```
\make pf08
```

Once created, they can be run on a terminal, for example

```
./pf08
```

43. The diagram sign

The *diagram sign* has been perhaps QGRAF's least understood feature. This state of affairs seems to derive mainly from the program's ignorance about (or rather, avoidance of) *graphical rules* — the most well known being ‘for each fermion loop, multiply the amplitude by -1 ’, of course. Then, what does the program *do*?

The explanation for there being a possible relative minus sign between two distinct diagrams (for the same scattering process) follows directly from Wick's theorem in the presence of anti-commuting fields, and that is precisely the approach implemented in the program. To compute the sign of a Feynman diagram D (hereafter, it will be assumed that D depends on some anti-commuting fields), the program starts by placing side by side (as a product) the vertices into which D can be decomposed. For example,

$$F_1 = (\bar{\Psi}_2 \Psi_{-1} A_3) (\bar{\Psi}_{-2} \Psi_7 A_5) (\bar{\Psi}_{10} \Psi_1 A_6) (\bar{\Psi}_8 \Psi_9 A_4)$$

is the vertex product for the diagram shown in Fig. 16, assuming that the field ordering in each vertex coincides with that of the respective vertex statement given in the model-file; the subscripts, which match the labels in that diagram, are the field-indices computed by the program. Those vertices have been reduced to a product of ‘plain’ fields, as the other attributes (coefficients, space-time indices, and so on) are not needed here.

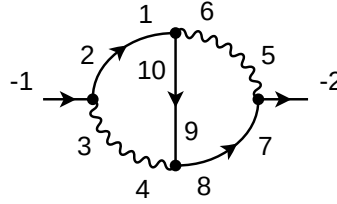


Fig. 16. A diagram (in QED) and its field-index numbering.

In F_1 , the vertex sequential order is irrelevant because each vertex is (for this purpose) a commuting quantity, comprising an even number of anti-commuting fields. In contrast, the relative ordering of the anti-commuting fields in each vertex is clearly relevant. In any case, an (arbitrary) initial vertex ordering has to be chosen to do the computation, and after that choice is made no vertices will be permuted — only a sequence of simple transpositions, each one involving exactly two fields, will be performed. If one discards the commuting fields, which clearly play no role in the present computation, F_1 will be reduced to the ordered product

$$F_2 = \bar{\Psi}_2 \Psi_{-1} \bar{\Psi}_{-2} \Psi_7 \bar{\Psi}_{10} \Psi_1 \bar{\Psi}_8 \Psi_9$$

which may also be regarded as a sequence. The next point to consider is that, whatever the type of field, the propagator ordering chosen by the program is always of the form

$$\langle \Phi_{2k-1} \Phi_{2k} \rangle$$

where the subscripts denote (internally generated) field indices. Thus, in our example, the internal anti-commuting fields will be paired into the following sub-sequences

$$\langle \Psi_1 \bar{\Psi}_2 \rangle, \langle \Psi_7 \bar{\Psi}_8 \rangle, \langle \Psi_9 \bar{\Psi}_{10} \rangle.$$

For explanatory purposes, it is useful to constrain the field transpositions allowed in this part

of the computation; without loss of generality, we shall require that the propagator pairing be achieved without changing the relative ordering of the external fields. With this restriction, once F_1 is fixed the parity of the propagator pairing operation will be well defined for each diagram. If this step takes t_p transpositions, the contribution to the sign will be $(-1)^{t_p}$. The internal fields are then deleted since they are no longer needed. In our example, an odd number of transpositions is required to transform F_2 into the sequence

$$F_3 = \Psi_{-1} \bar{\Psi}_{-2} \Psi_1 \bar{\Psi}_2 \Psi_7 \bar{\Psi}_8 \Psi_9 \bar{\Psi}_{10}$$

and we obtain a factor equal to -1 .

The last part of the computation involves the external fields. Here, the program assigns (arbitrarily) a positive sign to its generic ‘reference’ sequence, namely

$$\Phi_{-2}^{out} \Phi_{-4}^{out} \dots \Phi_{-2s}^{out} \Phi_{-2r+1}^{in} \dots \Phi_{-3}^{in} \Phi_{-1}^{in},$$

where r and s denote the number of incoming and of outgoing fields, and the subscripts are the field-indices (which are negative, since the fields are external). Any occurring sequence of external fields obtained in the previous step is then compared with the reference sequence (divested of its commuting fields), and the parity of the number of transpositions t_e needed to convert one sequence into the other is determined. Hence a new factor $(-1)^{t_e}$ is generated, and the diagram sign is then defined as $(-1)^{t_p+t_e}$.

In our example, removing the internal fields from F_3 leaves the sequence

$$F_4 = \Psi_{-1} \bar{\Psi}_{-2}$$

which can be converted into the reference sequence using a single transposition. Therefore, as $(-1)^{t_p} = (-1)^{t_e} = -1$, a positive sign is produced for that diagram.

In comparison with a graphical rule, the preceding definition of diagram sign might seem (to a human, and concerning evaluations ‘by hand’) somewhat complex, even error-prone; nevertheless, it is closely related to Wick’s theorem, and there is no difficulty whatever in programming it. Still, users who habitually rely on graphical rules might find some of the consequences of that definition rather unexpected. For that reason, let us take a look at a number of problems that may arise out of some misunderstanding or mere inattention.

43.1 Problem 1

Suppose we have defined QED as follows,

```
% propagators
[ photon, photon, +1 ]
[ positron, electron, -1 ]

% vertex
[ positron, electron, photon ]
```

where **electron** corresponds to a Dirac field Ψ and **positron** to $\bar{\Psi}$. The usual statement for that fermionic propagator should be

```
[ electron, positron, -1 ]
```

as this is the one that corresponds to the ordered contraction $\langle \Psi \bar{\Psi} \rangle$. The reason why the

former statement can be problematic is that the program will then sort the internal anti-commuting fields in a different way, and may thus compute a different sign. Then, if each propagator expression in the program's output is replaced by the usual Feynman rule, the result can be an incorrect amplitude. Declaring the 'wrong' vertex may lead to similar problems.

In QED, as defined above, using the conventional propagator expression does not always lead to problems, due to the following: for a fixed scattering process, and a fixed order of perturbation theory, every diagram has the same number of fermionic propagators; moreover, the number of fermionic propagators in n -loop diagrams and in $(n+1)$ -loop diagrams differs by *two*. Nonetheless, there exist many models for which one can find two diagrams (for the same process) with the same sign if the fermionic propagators are declared in one way, and opposite sign in the other.

43.2 Problem 2

Suppose one wishes to compute the amplitude of some scattering process involving fermions (in the initial and/or final state), both at leading order and at some higher orders. One declares

```
in = electron[p1], positron[p2] ;
out = photon[q1], Z[q2] ;
loops = 0 ;
```

for the leading order, and

```
in = positron[p2], electron[p1] ;
out = photon[q1], Z[q2] ;
loops = 1 ;
```

for the next-to-leading order. If the two initial fermion states are permuted, the parity $(-1)^{t_e}$ changes (for *every* diagram). Oops, the next-to-leading order amplitude is to be added to the leading order amplitude, but now every relative sign between a 0-loop diagram and a 1-loop diagram will be wrong.

43.3 Problem 3

We have a model with both Dirac and Majorana fermions, say. Then we choose the scattering process and the order of perturbation theory, and have the program write down an expression for the corresponding 'amplitude'. Now we pick that (commuting) expression, which includes the diagram signs computed by the program, and decide to 're-orient' some fermionic propagators in that expression. For example, we make some substitutions such as

$$\text{prop}(\text{Psi}(3, \mathbf{k}1), \text{Psi}(6, -\mathbf{k}1)) \rightarrow \text{prop}(\text{Psi}(6, -\mathbf{k}1), \text{Psi}(3, \mathbf{k}1))$$

where each of these expressions denotes a propagator function for a Majorana fermion Psi . That can also be problematic since those signs were computed by assuming a certain field ordering (for the propagator fields, for instance). If the redefined propagators had been used instead then different signs might have been produced for some diagrams.

We might decide also to exchange some arguments describing that same fermion, this time in some sub-expressions representing vertex (Feynman) rules. For instance, given an

amplitude that includes a sub-expression such as

```
vertex( Psi(5,k1), Psi(2,-k1-k2), Phi(7,k2) )
```

where `Phi` denotes a bosonic field, we might (simply) exchange the arguments `Psi(5,k1)` and `Psi(2,-k1-k2)`. Since those fields are identical there is no problem, right? Not really, that exchange would have affected the computation of $(-1)^{t_p}$.

43.4 Problem 4

The Path Integral formulation of Quantum Mechanics ‘says’ that the amplitude for a given scattering process should include the contribution of every ‘path’ leading from the initial to the final state (in the corresponding interpretation, that is why one adds the contributions of multiple Feynman diagrams, of course). Let us consider an experiment that has various possible outcomes, and for which — as a result of the measuring apparatus not being precise enough, say — some of the outcomes are not distinguishable from one another. Then, the amplitude for a certain (measurable) final state can be obtained by adding the amplitudes for the individual processes compatible with that measurement. In this case, the relative sign between diagrams for (a priori) distinct processes becomes critical.

The emission of (real) low-energy photons in a high-energy collider provides a simple example, owing to the unavoidability of experimental cutoffs. Thus, a scattering process like

```
in = electron[p1], positron[p2] ;
out = muon_minus[q1], muon_plus[q2] ;
```

may often be indistinguishable from

```
in = electron[p1], positron[p2] ;
out = muon_minus[q1], muon_plus[q2], photon[q3] ;
```

for small 3-momentum $|\vec{q}_3|$. Here, the program seems to compute the correct relative sign between the (former) ‘non-radiative’ process and the (latter) ‘radiative’ process, *if* the respective statements match as above (it should be checked all the same, of course). Still, the program does not address that kind of problem and other means must be employed to determine an appropriate global sign for each process. Moreover, any necessary adjustments should preferably be determined at an early stage, to avoid having to redo as many runs as possible (or to introduce ad-hoc signs). The sign of an amplitude may often be adjusted by permuting two incoming (or outgoing) fermions, but that is a minor point.

43.5 Solutions

It is always possible to ignore the sign computed by the program (by omitting any reference to it in the style-file), although that will require the user to implement some substitute definition (completely, that is). That seems the best option if one wishes to implement some sign convention not supported by the program.

Alternatively, if one does not care much about which sign convention is actually used, but wants nonetheless to be able to perform some substitutions that correspond in practice to fermionic exchanges, there is another (possibly easier) type of solution which consists in fixing the problematic substitutions — instead of discarding QGRAF’s sign altogether. The basic idea is that any substitution that corresponds to an odd number of fermionic transpositions should also generate a compensating factor equal to -1 .

Solutions to Problem 1

There is more than one fix, apart from just using the conventional propagator statement. For instance, one may use (at the symbolic processing stage) a modified Feynman rule for the propagator, differing from the standard one by a factor -1 . Alternatively, the model could be redefined by letting Ψ describe the **positron** and $\bar{\Psi}$ describe the **electron**. This would have to be reflected on the form of the electromagnetic vertex, and one should also have to consider what would the electric charge constant e define. Similar comments apply to the non-conventional vertex statement. Innovation of this kind is seldom a good idea, as the task of comparing one's results with existing results can (very likely) become more difficult; code debugging may also be harder.

Solution to Problem 2

Only the loop order of the diagrams should be changed, obviously. The extended `loops` statement might help but the diagrams will then have to be selected afterwards, depending on the order of perturbation theory being computed.

Solution to Problem 3

For each fermionic transposition, whether in a propagator or in a vertex, there should be an additional -1 factor for the amplitude. Additionally, as $(-1)^2 = 1$, it may sometimes happen that *‘two wrongs make a right’* (and that also applies to the other problems).

43.6 Further comments

Graphical rules were invented at a time when Feynman diagrams were generated by hand, and they made it easy for a human to determine (‘visually’) a practical diagram sign. Nonetheless, while graphical rules for QED and QCD are quite simple, they are more complex for other types of models — pure graphical rules may not even exist. The approach used by the program can be employed in general, irrespective of the vertex degrees and numbers of anti-commuting fields in each vertex, though the output may have to be processed somewhat to obtain expressions of the desired form. This approach might actually be advantageous, specially if the input model includes vertices with more than two fermionic fields, because it does not tie the user to additional conventions.

Nowadays, with the use of automatic set-ups being the norm, graphical rules are becoming less relevant. In addition, ‘simplifying’ Wick’s theorem consists in practice in adopting some (further) convention — while one of the guidelines behind QGRAF has been the adoption of as few conventions as possible, specially if they are not general enough.

A continued reliance on graphical rules may also induce the belief that the diagram sign is something more rigid than it really is. By that, we mean that the sign derived from any fixed set of graphical rules depends in fact on some convention(s) and/or assumption(s) that those rules may hide. Lastly, graphical rules do not necessarily solve Problem 4 either.

44. Models with explicit propagator mixing

For diagram generation purposes, a model features explicit propagator mixing if there is at least one field appearing in two (or more) propagator statements. Although this type of model is not accepted (at least not yet), there is a way to obtain the corresponding Feynman diagrams — namely by replacing the original model by an appropriately transformed model, as described in the following paper.

Feynman graph generation and propagator mixing, I
 Comput. Phys. Commun. 269 (2021) 108103.
<https://doi.org/10.1016/j.cpc.2021.108103>

In general terms, that type of transformation can be described by an algorithm that involves

- introducing new (ancillary) ‘charged’ fields;
- modifying the original set of propagators, to eliminate explicit mixing;
- adding new interaction terms, similar to the existing ones, but depending on the new fields too.

Although (as originally described) that algorithm applies directly to models whose propagators do not contribute to the order of perturbation theory (since their Feynman rules are independent of the coupling constants), it should be possible to apply it to many models in which some of the propagators do contribute — namely by defining appropriate weights (here, *p*-functions) in the model-file and then using `psum` statements.

It is perhaps advisable to let the input process (specified by the incoming and outgoing fields) be defined in terms of the fields of the original (non-transformed) model only, keeping the ancillary fields as internal (ie appearing only in propagators). The reason is twofold: clearly, ancillary external fields are not needed for studying the original model, and using those fields adds an extra complication, to be described below, which may lead to errors.

Let us consider a Lagrangian density $\mathcal{L}(\phi_1, \phi_2)$ that depends on two self-conjugate (real) fields ϕ_1 and ϕ_2 , and whose quadratic part includes a mixing term involving both fields, for example

$$\mathcal{L} = \frac{1}{2} \partial_\mu \phi_1 \partial^\mu \phi_1 + \frac{1}{2} \partial_\mu \phi_2 \partial^\mu \phi_2 + a \partial_\mu \phi_1 \partial^\mu \phi_2 + \dots$$

In this case, as shown in Example 4.1 of the above mentioned paper, the transformed model depends also on a conjugate pair of ancillary fields, to be denoted by ϕ_3 and ϕ_4 . To convert the diagrams obtained for the transformed model into the diagrams of the original model, the *internal* ancillary fields ϕ_3 and ϕ_4 should be replaced as follows:

$$\phi_3 \rightarrow \phi_1, \quad \phi_4 \rightarrow \phi_2.$$

In that paper, there is no discussion concerning the distinction between incoming and outgoing fields. Implicitly, the definition of *u*-product entails that every external field is an incoming field (the opposite convention is allowed, but it is perhaps not as common). As QGRAF requires the incoming and the outgoing fields to be properly identified, however, one may find that the previous substitution rules do not necessarily apply to external fields. In fact, while they are still valid for incoming fields, for outgoing fields one has instead

$$\phi_3 \rightarrow \phi_2, \quad \phi_4 \rightarrow \phi_1.$$

For example, since ϕ_3 and ϕ_4 constitute a conjugate pair, the input process

$$\begin{aligned} \text{in} &= \text{phi1}[p1] \ ; \\ \text{out} &= \text{phi3}[q1] \ ; \end{aligned}$$

is ‘equivalent’ to

$$\begin{aligned} \text{in} &= \text{phi1}[p1], \text{phi4}[p2] \ ; \\ \text{out} &= \ ; \end{aligned}$$

which (by the former substitution rules) is ‘equivalent’ to

$$\begin{aligned} \text{in} &= \text{phi1}[p1], \text{phi2}[p2] \ ; \\ \text{out} &= \ ; \end{aligned}$$

which (as ϕ_2 is self-conjugate) is ‘equivalent’ to

$$\begin{aligned} \text{in} &= \text{phi1}[p1] \ ; \\ \text{out} &= \text{phi2}[q1] \ ; \end{aligned}$$

which should be compared with the initial process. These processes are *essentially equivalent*, in the sense that the set of diagrams generated for each process can be easily transformed into the set of diagrams for any other equivalent process; in particular, those sets have the same number of diagrams. In contrast, the scattering process

$$\begin{aligned} \text{in} &= \text{phi1}[p1] \ ; \\ \text{out} &= \text{phi4}[q1] \ ; \end{aligned}$$

is not ‘equivalent’ to the previous ones.

Part VI — Additional information

45. Automatic downloads, licensing

It might be useful to have a simple way of automatically downloading a patched version instead of an outdated version with known problems, or even download a new (minor) version instead of the previous one. The following ‘version-linking’ set-up has been implemented.

- any future release will be numbered ‘x.y.z’, that is, using major, minor, and patch numbers (for instance, the first version of `qgraf-4.1` should be `qgraf-4.1.0`);
- a request to download ‘links/qgraf-x.y.tgz’ (with valid x and y) is converted into a request for the version `qgraf-x.y.z.tgz` with the highest (available) z; the end result should be the latest `qgraf-x.y.z` for the given x and y.
- a request to download ‘links/qgraf-x.tgz’ (with valid x) is converted first into a request for downloading ‘links/qgraf-x.y.tgz’ with the highest y, which will then be converted by the previous rule;
- a request to download ‘links/qgraf-x.s.tgz’ (with integer x and literal s) is converted into a request for downloading ‘links/qgraf-x.y.tgz’, with the highest y such that `qgraf-x.y` has been declared ‘stable’;
- other requests, including those for specific versions, will not be converted (irrespective of availability);
- automatic downloading requests should now refer to the directory ‘links’, for example


```
wget --quiet --user=anonymous --password=anonymous -O ./qgraf.tgz \
  http://qgraf.tecnico.ulisboa.pt/links/qgraf-4.0.tgz
```
- after each new release, there will be a ‘grace period’ (to be announced, possibly variable) before the corresponding conversions become effective; that would be coupled with an optional, automatic message system (see below).

This should not be contentious since it is optional and does not remove existing features. An ‘alert system’, which would allow ‘package owners’ and other users to automatically receive news and (above all) alerts about the program, is being considered.

Please do not implement or describe openly the (ie any) downloading method in some webpage, ready for ‘bot use’ (alas, there seems to be more than enough bot generated web traffic already). The information here provided is for ‘package-scripting’ only, ie for those packages that rely on this program.

Some versions that are no longer referenced on the website’s *Downloads* page might still be temporarily available in the above mentioned ‘links’ directory. At the time of the release of `qgraf-4.0.6` the convertible requests and their targets are as follows.

<code>links/qgraf-3.4.tgz</code>	→	<code>qgraf-3.4.2.tgz</code>
<code>links/qgraf-3.5.tgz</code>	→	<code>qgraf-3.5.3.tgz</code>
<code>links/qgraf-3.6.tgz</code>	→	<code>qgraf-3.6.10.tgz</code>
<code>links/qgraf-3.tgz</code>	→	<code>qgraf-3.6.tgz</code>
<code>links/qgraf-3.s.tgz</code>	→	<code>qgraf-3.6.tgz</code>
<code>links/qgraf-4.0.tgz</code>	→	<code>qgraf-4.0.6.tgz</code>
<code>links/qgraf-4.tgz</code>	→	<code>qgraf-4.0.tgz</code>

and the non-convertible requests are

```
links/qgraf-3.4.2.tgz
links/qgraf-3.5.3.tgz
links/qgraf-3.6.9.tgz
links/qgraf-3.6.10.tgz
links/qgraf-4.0.6.tgz
```

45.1 Stable versions and licensing

A *stable version* is a version that should be expected to be available and supported for a reasonably long period (five years, at least), within my ability and resources, and assuming that the present circumstances do not worsen in any relevant way, of course. Note that by *version* I mean something like **qgraf-x.y** where **x** and **y** denote the major and the minor version numbers. Since declaring a version as stable the moment it is released does not seem to be the best strategy, there will be a delay (of at least five months) so that (i) a better perspective on the program development may be obtained and (ii) there is an additional opportunity to find and eliminate any remaining bugs, or even to refine or add some feature. Once a version is declared stable, it should not be modified except for fixing some anomaly or unexpected behaviour, and even then in a minimal way; that includes both actual errors and potential errors, ie code that is already known to be erroneous and code thought likely to lead to problems.

Not every version will be declared as stable and, depending on its usefulness, a stable version may become unavailable once the appropriate period expires. One idea here is that some versions are ‘*more equal*’ than others, and that if some critical bug happens to be found then the stable versions will be prioritised whilst the non-stable versions could even be pulled out, temporarily or otherwise. That is why the licensing terms state that packages relying on QGRAF should be able to use a stable version — not necessarily in an exclusive way. The interpretation of that part of the licensing terms should be that *some available version of the package should be able to rely on an available, stable version* of this program. If the package dependence is introduced¹⁸ at a time when the latest version is not stable, and if no stable version fulfils the necessary requirements, then it could be acceptable to defer the fulfilment of that condition until the following stable version is declared — provided the inherent risks are assumed.

Non-stable versions should be expected to become unsupported, and unavailable for downloading, two years after the release of the subsequent stable version (but they might be pulled out for other reasons too).

There is now a provision for re-distributing officially *unavailable, stable* versions¹⁹ in two cases (see the header of one of the latest Fortran files for details), provided there is no legal impediment: (i) with (old) packages for which no upgrade is feasible, or (ii) with either new or old packages, should QGRAF’s official website close down. Nevertheless, in case (i) this provision might not be needed in practice as there is little backward-incompatibility in **qgraf-3**, specially at the level typically required by other packages.

¹⁸ The relevant date is the release date of that package version, obviously.

¹⁹ This means the latest patch version, of course; for example, **qgraf-3.4** is stable but, unless some other patch version is released in the meantime, **qgraf-3.4.2** would be the actual version to re-distribute.

The current list of stable versions and of their latest patch version (*lhs* and *rhs*, respectively) follows.

3.4	→	qgraf-3.4.2
3.6	→	qgraf-3.6.10

46. Implementing the new input specifications

Input files accepted by **qgraf-4** need not be compatible with **qgraf-3.6** (nor conversely), hence a file conversion tool could be useful; in fact, an integrated tool of that kind is available with **qgraf-4.0**, in auto-mode. The differences between the two ‘standards’ are described in Section 0. For some computer program (that uses the API) to rely on that tool, however, QGRAF should be installed also as ‘standalone’, so that its executable may be used for the file conversion.

It should often be possible to run **qgraf-4.0** using (indirectly) input files compatible with older, available versions of the program, mainly in the case of automatic set-ups. That would be achieved with a few brief, additional preliminary runs, each of which would convert an ‘oldish’ input file into its latest equivalent — namely by executing a command of one of the types described in some of the following subsections (one command for input file). The diagram generation would then take place in one further run, using those converted files as input. Alternatively, whole directories might be converted ‘in bulk’, beforehand, so that no ‘on-the-fly’ conversions would be necessary (Section 48). Obviously, any remaining constraints involving the control-file and described in p. 4 (rules 1 and 5, for instance) would have to be implemented by the user (or programmer).

In the first subsection, below, it is pointed out that it is very easy to address the changes entailed by rules 4 and 5 (described in p. 4). The subsequent subsections discuss the ‘forward-conversion’ of input files — that is to say, the conversion of files conforming to the previous input specifications to files that conform to the latest ones — and the (non-)insertion of a UTF-8 BOM in the converted file. That conversion may have to include (for example) the following basic operations:

- substituting any annotation-mark by another such mark (a fixed one, that is);
- adding any required statement-continuation marks.

46.1 Implementing rules 4 and 5, easily

Some of the recently introduced backward-incompatibilities — involving deleted defaults of some kind — can easily be dealt with, even (or perhaps *specially*) in the case of automatic set-ups. In addition, the simplest solution is, in each case, independent of the program’s version; for thoroughness only, they are detailed below.

If the default external momenta are (implicitly) being relied on, rule 4 can be satisfied by making the corresponding (explicit) declarations in the **in** and **out** statements, say

```
in = electron[p1], positron[p2] ;
out = photon[q1], photon[q2] ;
```

whether needed or not (this last bit applies also to the other two cases). Moreover, if the default integration momenta prefix is being used (implicitly), it is sufficient to always include the following statement in the control-file.

```
loop_momentum = k ;
```

In auto-mode, if the default filename for the control-file is being relied upon, rule 5 can be satisfied by launching the program with a command-line argument equal to that default (possibly using a shell alias, if convenient), for example

```
qgraf qgraf.dat
```

This should work even with `qgraf-3.4`, in which case that argument will *not* be read. The point here is that there is a simple generic solution, which may simplify the update of related software. It is also possible to detect the version of the program being used and address the various cases one by one, of course. In API-mode, that filename has to be supplied in the initialization call.

46.2 Forward-conversion of model-files

The conversion of model-files should be the most relevant, owing to the larger number of files of that type that are likely to be included in a ‘collection’ of ancillary files. That conversion can be performed with a command of the following kind.

```
qgraf --m3to4 models3/qcd models4/qcd --hash
```

The first filename (here, ‘models3/qcd’) should be the name of the file to convert, and the second the name of the file to create. The two options ‘--hash’ and ‘--percent’ define the annotation-mark (respectively ‘#’ and ‘%’) to be used in the new file; one of those options is required. Apart from two aforementioned basic operations, the program adds any required statements declaring the constants and functions of the input model-file (Section 26.3).

46.3 Forward-conversion of style-files

The conversion of a style-file can be performed with a similar command, for example

```
qgraf --s3to4 styles3/a1.sty styles4/a1.sty --percent
```

46.4 Basic forward-conversion of control-files

The (full) conversion of control-files has not been implemented since it presents a number of difficulties. In particular, there are non-negligible chances of failure, for example when the filenames are not appropriately restricted, or when some options are used. Also, that type of conversion appears to be the least relevant type of conversion: programs that rely on QGRAF usually create their own control-files, and users that run the program directly also create a control-file for each occasion.

What has been implemented is a minimal type of conversion that performs only the two above mentioned (itemized) basic operations involving annotation-marks and statement-continuation marks. Nonetheless, that basic conversion is often sufficient, provided that the statements included in the input file are not forward-incompatible. The respective command is similar to those to be used with the other types of input files, for example

```
qgraf --b3to4 control.dat q.in --percent
```

where ‘control.dat’ should be an input file without statement-continuation marks, and ‘q.in’ the control-file to be created.

46.5 Adding (or not adding) a byte-order-mark

It is possible to control the (non-)insertion of a UTF-8 BOM in the converted file by using one of the command options

```
--utf8_bom
--no_utf8_bom
```

which should appear last. If no such option is used, a BOM will be inserted if and only if the input file has one. An example follows.

```
qgraf -s3to4 styles3/a1.sty styles4/a1.sty -hash -no_utf8_bom
```

46.6 A few remarks

The sequence of the arguments of the above type of commands is fixed: the main option first, then the two filenames followed by any other ‘options’.

On the execution of the above type of commands, an empty line may be added at the start and at the end of the new file; trailing spaces (from any input line) will not be copied.

To some extent, the file to convert may be assumed to be a valid file of the expected type (that is, its contents may not be fully checked).

47. Backward-conversion of input files

The conversion of model-files accepted by **qgraf-4.0** to files compatible with either **qgraf-3.6** or **qgraf-3.4** (to be dubbed ‘backward-conversion’) is usually possible, but the keyword **internal** may be (quietly) discarded since it is not recognized by **qgraf-3.4**. The backward-conversion of style-files is not possible unless such files do not include any of the recently introduced (irreducible) keyword(s).

Even then, the backward-conversion may fail in a few supposedly rare cases. One such case is that in which an identifier or a style-keyword²⁰ does not fit in a ‘non-wide’ line (which may have at most 80 characters); still, that case can be easily avoided, simply by not defining such long strings (which are almost surely never defined, anyway). An error will also occur if the defaults that are to be made explicit (in a propagator or vertex statement) lead to a very long statement whose length exceeds the maximum allowed statement length. If the conversion fails for any reason, no output file will be saved.

It is to be expected that a user may prefer to have a single collection of input files, conforming to only one ‘standard’. From that point of view, the forward-conversion may be useful to continue using, now with **qgraf-4.0**, input files that conform to the previous specifications. The backward-conversion, while not perfect in general, should provide a way for **qgraf-3** to use, also indirectly, input files compatible with **qgraf-4.0** that do not employ certain recent features. The following features characterize the basic backward-conversion (when applicable):

- annotations are dropped;
- statement-continuation marks are dropped;
- trailing space characters are dropped;
- the number of consecutive, non-relevant empty (output) lines is limited to two;
- lines may be re-arranged (split into two, for instance) so as not to exceed the 80 character limit.

These features (except the last) will not be repeated below. Each of the command options ‘**--utf8_bom**’ and ‘**--no_utf8_bom**’ can be used in the backward-conversion too.

47.1 Backward-conversion of model-files

Whilst the forward-conversion copies the contents of the input file in (nearly) the exact same format, the backward-conversion not only ignores that formatting but may even discard some statements. Moreover, if the input model-file describes some submodel(s) then one such submodel must be selected; the converted file will describe a single model, with no sectors and no defaults. If the input model-file does not define any submodel, the command

```
qgraf --m4to3 models4/model_a models3/model_b
```

should convert file ‘models4/model_a’ to file ‘models3/model_b’. The actual kind of conversion is determined by the main option, as follows, where the version numbers (*rhs*) refer to the compatibility of the input file and of the output file, respectively.

```
--m4to2    means    4.0 → 3.4    (option ‘--subm’ required)
--m4to3    means    4.0 → 3.6
```

Those two options are almost equivalent, differing only in how they deal with the keyword

²⁰ That particular problem may be circumvented for encoded strings.

`internal` (which is kept if ‘`--m4to3`’ is used but not if ‘`--m4to2`’ is used). There is no option to specify an annotation-mark but there is an option to select a submodel, which may be required if the input file defines any submodel; in that case the appropriate command should be of the form

```
qgraf --m4to2 models4/model_a models3/model_b --subm model0
```

where ‘`model0`’ is the submodel to be ‘extracted’. The argument sequence is the same as that of the forward-conversion, with the exception that the submodel specification (which did not appear in the latter case) follows the filenames, preceding any other options.

To create the converted file, the program reads the input model-file and does the following:

- deletes every statement of the types introduced with `qgraf-3.6` (taking the relevant ones into account); for example, it deletes every submodel block, alien sector block, and statement that defines some default;
- determines the necessary (effective) definition of any submodel-dependent constants and moves (or adds) the respective statements to zone 2;
- adds the necessary function (ie image) definitions to propagator and vertex statements (that is, those images defined by some default);
- erases every occurrence of the keyword `internal` in propagator statements if option ‘`--m4to2`’ is used;
- splits every ‘long’ encoded string into two or more pieces, ready for concatenation;
- splits every long statement into a number of pieces that fit in ‘non-wide’ lines.

Thus, some statements may be rewritten and others omitted.

Note that the command-line option ‘`--extract`’ (Section 5.4), is also available, and will remain a permanent feature. The respective output file may not be backward-compatible, but then the conversion tool can be used if needed.

47.2 Backward-conversion of style-files

The command for the style-file conversion is similar, for example

```
qgraf --s4to3 styles4/a1.sty styles3/a1.sty --utf8_bom
```

There are two possibilities for the main option, namely `--s4to3` and `--s4to2`.

47.3 Basic backward-conversion of control-files

Also in this case, the file conversion is a minimal one; only the generic transformations itemized in the introduction of this section are performed. An example of the respective type of command follows.

```
qgraf --b4to3 control.in q.dat
```

The basic backward-conversion is always possible, but its use is rather limited.

48. Conversion in bulk of input files

A script file included in the distributed pack may facilitate the forward-conversion of an entire directory (including sub-directories) containing a single class of input files; that is not always the case for the backward-conversion, for which some limitations exist. Executing a plain `'ls'` command in subdirectory `'tools/q4fc'` should produce the following listing, possibly not in the same format.

```
models3/      q4fcs      styles4/
models4/      scripts/   styles_out/
models_out/   styles3/   tmp_dir/
```

The file `'q4fcs'` is a `bash` script and every other is a directory, of which `'scripts/'` contains two other `bash` scripts used by `'q4fcs'`. Assume that `'models3'` may contain only model-files compatible with either `qgraf-3.4` or `qgraf-3.6`, and `'models4'` model-files compatible with `qgraf-4.0`. A similar property should be assumed for directories `'styles3'` and `'styles4'`, this time concerning style-files, of course. Assume further that the directories `'models_out'` and `'styles_out'` are overwritable, meaning that the script `'q4fcs'` is allowed to change their contents and even delete them altogether (those two directories should be *reserved* to the files created by that script).

To perform the file conversion, start by placing a *copy* of any files to be converted in the appropriate directories, that is, `'models3'`, `'models4'`, `'styles3'` and `'styles4'`. Sub-directories may be created at more than one level (or depth). Edit the script `'q4fcs'` and define appropriately the variables `QGRAF`, `MODELSO` and `STYLESO`, whose declaration (possibly commented out) appears near to the top of that file. Other variables may be redefined if different directories are preferred, although that will also require some corresponding adjustments on the file system (ie, renaming or creation of directories); for simplicity, it will be assumed in what follows that those names were not changed. When that configuration has been properly adjusted, use `chmod` to allow file execution, for instance

```
\chmod u+x q4fcs
```

Then, to convert the model-files in `'models3'` to model-files compatible with `qgraf-4.0`, for example, execute the following command in the above mentioned directory.

```
./q4fcs --m3to4 --hash --no_utf8_bom
```

The converted files will be left in the directory defined by `MODELSO` (ie `'models_out'`, by default) and, assuming that directory to be empty at the time the script is launched, the exact same directory structure of `'models3'` will be re-created in the former directory. A BOM option is optional and should come last; the other two options (either those, or others of the same types) are required.

An important point: if two or more conversions of the same type are to be performed with script `q4fcs`, and if the corresponding sets of files should be kept separate, then the respective output directory should be emptied before re-running that script (otherwise the result will probably look like a confusing file assortment). The first of the commands

```
./q4fcs --mclean
./q4fcs --sclean
```

should clean `'models_out/'` or, more precisely, delete the entire directory and then create a

new (empty) one; the directory ‘`styles_out/`’ is ‘cleaned’ by the subsequent command. There are no options to clean input directories but it is possible to extend the script ‘`q4fcs`’, of course.

Any file conversion option described in the two previous sections can be used, but files that cannot be converted will not give rise to a new file. A forward-conversion requires an ‘option’ specifying an annotation-mark, a backward-conversion does not. A command option specifying the (non-)insertion of a **UTF-8** BOM is truly optional.

Concerning the backward-conversion of model-files, those that define one or more submodels will not be converted in bulk if the option ‘`--m4to2`’ is used. Such files will have to be converted separately, extracting one submodel at a time.

The foregoing discussion applies to style-files as well, changing what should obviously be changed. For example, the next command converts the style-files in directory ‘`styles4`’ to files compatible with `qgraf-3.6`, leaving the new files in ‘`styles_out`’.

```
./q4fcs --s4to3
```

49. Obsolescent features

This section may change in future versions of this manual, since it shall describe some changes that ought to become mandatory with **qgraf-5** (assuming that that version will exist at some point). The aim of such announcements is to try to preclude abrupt changes — as there will be then a long period ahead of their enforcement (years, in all likelihood) for the implementation of the respective specifications. Changes that are still not fully determined will be included, provided they *are* planned. The current list follows.

- (1) option **topol** will be deleted (option **new_topology** can be used instead);
- (2) options **simple** and **notsimple** will be deleted (option **simple** can be replaced by the conjunction ‘**nodiloop, noselfloop**’, and thus option **notsimple** is equivalent to an inclusive disjunction of the options **diloop** and **selfloop**, which can be treated using the method discussed in Section 17);
- (3) statements with an ‘unbounded’ number of arguments (Section 23) will be deleted; in their place there will be slightly more general statements depending on a single function, which will not have to be non-negative (even now any **iprop** statement can be replaced by a **psum** statement based on an appropriate p-function).

The ‘definite’ changes, that is, those for which some sort of equivalent exists already, should be alerted in **verbose** mode if the obsolescent features are used.

50. Changelog for qgraf-4

4.0.0 (May 2024)

New features: the ‘`-no_extra_chars`’ command-line option (p. 16); a statement-continuation mark, longer input lines (p. 19); the config option `delete` (p. 26); the `model_dir`, `style_dir`, `output_dir` and `separator` statements (pp. 30 *f*); the options `new_elinks`, `new_loops`, `new_partition`, `new_topology`, `noedge` and its dual (p. 39); the style-keywords `<loops1>`, `<loops2>`, `<local_symmetry_number>`, `<nonlocal_symmetry_number>` (p. 81 *f*), as well as `<field_sub>` and its dual (p. 94); an extended scope for the prologue and epilogue sections (p. 94); a preliminary version of the file conversion tool (p. 125).

Previously unannounced features: option `norbridge` and (duplicated) options `nosbridge`, `nobridge`, as well as their duals (p. 39 *f*), all of which introduced with `qgraf-3.3`; the ability to use commutation *numbers* (instead of *signs*) in propagator statements (p. 67), introduced (also quietly) with `qgraf-3.6`.

Other changes: some backward-incompatibilities were introduced (p. 4, rules 1–3) and others announced (p. 4, rules 8–9); the options and `loop_momentum` statements are now optional;²¹ the display-output may include the numbers of propagator-type and of vertex-type sectors of the input submodel, and the diagram count for each loop order (pp. 100 *f*); not for the first time, a few parameters were increased.

4.0.1 (May 2024)

Bug fix: a problem with function values.

An additional warning about Fortran module files was added to this manual (p. 11).

4.0.2 (June 2024)

New features: the `<index_offset>` style-keyword (p. 81).

Bug fix: re-activates option `topol`.

Other changes: occasionally, a minor speed-up may be observed; the file conversion tool was improved a bit (p. 125); a tentative *interface specification* was added to the respective version of this manual (and was later replaced by the ‘definitive’ specification).

4.0.3 (June 2024)

Bug fix: a problem with `vsum` that may occur in the presence of duplicate vertices.

4.0.4 (July 2024)

New features: the ability to exclude a fixed external momentum term from some of the expressions for the diagram’s momenta (p. 34); the `zero_momentum` statement (p. 46) and the style-keyword `<zero_momentum>` (p. 81); the momentum-loop construct, based on the style-keywords `<momentum_loop>`, `<momentum_term>` and its dual (p. 89).

Bug fixes: a problem with the internal momenta (which were not always computed), and additional problems with `<full_time>`, `<raw_time>` and `<index_offset>`.

²¹ That change has essentially been reversed for the `loop_momentum` statement (with `qgraf-4.0.5`).

4.0.5 (September 2024)

New features: config options `flush` and `no_ntnls` (p. 27); the `count_to` statement (p. 45); the style-keywords `<loop_momentum>` and `<diagram_counter>` (p. 81); an implementation of the ‘definitive’ API (pp. 105 *ff*).

Bug fixes: some non-decomposable models were deemed decomposable; wrong statements produced by `<command_data>` in multiple-style configurations; a problem involving config option `noinfo`, which could cut the diagram generation short, apart from not blocking some of the display-output; reverts the redefinition of `<diagram_index>` in the epilogue section.

Other changes: further backward-incompatibilities were either introduced (p. 4, rules 4–5) or announced (p. 4, rules 6–7); the source code will include directives from now on, and some additional options are thus needed for compiling `qgraf-4` (p. 11); the rule for the implicit display-mode has been modified (p. 28); the file conversion tool has been improved a bit further (p. 125).

4.0.6 (February 2026)

New features: the command-line option `--extract` (p. 17); the configuration options `debug` (a display-mode), `utf8_bom` and its dual (pp. 28–29); the language for describing models has been minimally extended to allow, for some (additional) statement types, multiple statements of each type (p. 62); the style-keyword `<field_stype>` (p. 95).

Bug fixes: incomplete input validation; filenames subjected to unexpected constraints; a non-existent compilation option (in a script file); broken error reporting; an empty message-file could be saved at the end of the run.

Other changes: implementation of the announced (remaining) input specification changes, including a ‘new’ rule (p. 4); a few API adjustments (p. 6); an internal buffer for storing a limited number of messages (p. 32); in auto-mode, the control-file may now contain zero `output` statements (p. 33); a slightly more detailed display-output (p. 100), and the redirection of the ‘diagnostic’ display-output to the proper standard stream (p. 103); last but not least, the (likely) completion of the file conversion tools, which may be of use for the ‘forward-conversion’, ‘backward-conversion’ and conversion ‘in bulk’ (pp. 125 *ff*).